OOP

---

1. Polymorphism
2. Classes and object
3. Static keywords
4. Constructors
5. Inheritance
6. Abstraction and encapsulation
7. Interface
8. Dynamic polymorphism
9. Tight coupling and loose coupling
10. constructor chaining and its way
11. Adapter pattern ,single pattern, factor pattern, builder pattern
12. Final keyword
13. How to create user defined classes as immutable classes
14. Solid principal

Polymorphism

Q. What is polymorphism?

Polymorphism means when the same thing performs different behaviour in different situations according to a requirement called polymorphism.

Example: Mobile is example of polymorphism
Because mobile can use as phone, can use as TV, can use camera, can use as Calculator etc

Example: Person

Persons can change its behaviour according his requirement like as suppose we have person and his name is ram
Ram is manager in some company then ram behaviour in front his team as boss and in front of his senior or his manager as employee , when ram with his wife behave as husband , with child behave as parent etc

How to implement polymorphism practically?

If we want to implement the polymorphism practically we have two ways

a. **Using compile time polymorphism technique**

   Q. What is compile time polymorphism?

   Compile time polymorphism means when functionality/function bounded with object at program compile time called as compile time polymorphism

   How to implement the compile time polymorphism practically?

   If we want to implement the compile time polymorphism practically we have to use function overloading technique.

b. **Using runtime polymorphism technique**

Q. What is runtime polymorphism/dynamic polymorphism/late binding?

When functionality bounded with object at program runtime called as runtime polymorphism

How to implement the runtime polymorphism?

If we want to implement the runtime polymorphism we have to use
Function overriding technique

Note: we will discuss this topic in inheritance chapter

Function overloading concept

Q. What is function overloading?

Function overloading means when we have multiple functions with the same name , with different data types with a different parameter list with a different parameter sequence called function overloading.

Example:

```
void add(int x,int y)
{
}
void add(float x,float y)
{

}
```

```
void add(float x,int y)
{

}
void add(int x,float y)
{

}
```

Suppose if we want to work with overloading we have some important points.

a. Function name must be same
b. Return type is not considered in function overloading means developer give different return type with every function definition
c. Function parameter list or parameter type or sequence of parameter must be different
d. Function calling is dependent on number of parameter pass in it,its sequence and data type

```
public class OverloadingApp
{   public static void main(String x[])
    {
        add(10,20);
        float result=add(5.4f,2.5f);
        System.out.println("Addition is "+result);
    }
    public static void add(int x,int y)
    {
          System.out.println("Addition is "+(x+y));
    }
    public static float add(float x,float y)
    { return x+y;
    }
```

Talking: Adinath Giri

35:55

Example with function overloading

```
public class OverloadingApp
{  public static void main(String x[])
   {
      addInteger(10,20);
            float result=addFloat(5.4f,2.5f);
            System.out.println("Addition is "+result);
   }
   public static void add(int x,int y)
   {
        System.out.println("Addition is "+(x+y));
   }
   public static float add(float x,float y)
   { return x+y;
   }
}
```

Example without function overloading

```
public class OverloadingApp
{  public static void main(String x[])
   {
              addInteger(10,20);
            float result=addFloat(5.4f,2.5f);
            System.out.println("Addition is "+result);
   }
   public static void addInteger(int x,int y)
   {
        System.out.println("Addition is "+(x+y));
   }
   public static float addFloat(float x,float y)
   { return x+y;
   }
}
```

Note: suppose if you think about above code there is no function overloading implementation we defined two function first is addInteger(int,int), addFloat(float,float)
When we run this code we get same answer like as function overloading

Q. What is the benefit of function overloading or why use function overloading?

If we have multiple logic under the same domain or same logics required multiple time with different input parameters then we required to define multiple function for implement that logic then we provide different function names for every function logic but at the time of reuse function as developer we need to remember all function name and it is little bit tedious task for avoiding this situation OOP suggest give same name to all function if function has different data type or parameter list or parameter sequence so as developer we not need to

remember different function name just you can remember single function and function execution is dependent on function calling according rule of overloading

Note: given example explain the above scenario

```
public class OverloadingApp
{  public static void main(String x[])
  {    addInteger(10,20);
            float result=addFloat(5.4f,2.5f);
            System.out.println("Addition is "+result);
  }
  public static void addInteger(int x,int y)
  {
       System.out.println("Addition is "+(x+y));
  }
  public static float addFloat(float x,float y)
  { return x+y;
  }
} //addition  is same logic
```

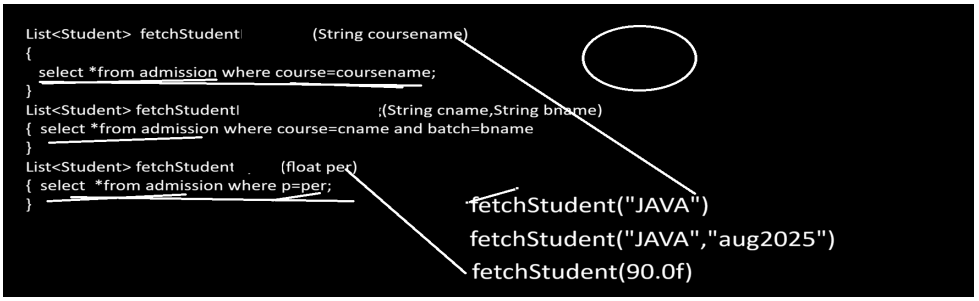
Description of above code: if we think about above code we define two functions addInteger(int,int),addFloat(float,float)
But both function contain same logic like as x+y but in both function required different input parameter so we define two function with two different names but as developer when i want to use these function i need to remember different function names so OOP suggest dont't remember different function name if so give all function name same with different parameter list or parameter type or sequence of parameter and call function by same name i will decide which function should call dependent on your input parameter from calling

Q. Did you use function overloading in your project?
Yes i used function overloading in my project

Q. Explain scenarios where you use function overloading or give context where you use function overloading?

Suppose consider we develop the admission process And we want to generate reports like as

a. **Fetch student list by course**
b. **Fetch student list by batch & course**
c. **Fetch students list by batch & passing year**
d. **Fetch student list by per**



Suppose consider : we are working in banking application a customer may be transfer money by different ways

a. **Using online banking**
b. **Using mobile number**
c. **Transfer using currency conversion**

Here we required following functions with different method names
transferByAccount(int accno,String bname,String ifsc,int amount);
transferByMobile(int contact,int amount)
transferByInternational(int acc,String bname,String cname,int amount,String cctype,int crate)

If we think about above code we write three different function name with a different parameter list , with a different data type with a different parameter sequence
But goal of every function logics is same i.e transfer money but behaviour of every function is different for achieve that logics so better way writing different function name is not good approach you can give same name to all functions called as function overloading or compile time polymorphism.

Example with source code

---

transfer(int accno,String bname,String ifsc,int amount);
transfer(int contact,int amount)
transfer(int acc,String bname,String cname,int amount,String cctype,int crate)

Example: we want to design the API for sorting for integer data , character data etc

```
public class SortingApp
{
  public static void main(String x[])
  {
    int a[]=new int[]{5,2,4,1,3};
        char c[]=new char[]{'d','a','b','c','e'};

        System.out.println("\nDisplay integer array before sorting");
        for(int i=0; i<a.length; i++)
        {  System.out.printf("%d\t",a[i]);
        }
        System.out.println("\nDisplay character array before sorting");
        for(int i=0; i<c.length; i++)
        { System.out.printf("%c\t",c[i]);
        }
        sort(a);//integer sort is executed
        sort(c); // character sort is executed
        System.out.println("\nDisplay integer array after sorting");
        for(int i=0; i<a.length; i++)
        { System.out.printf("%d\t",a[i]);
```

```
        }
        System.out.println("\nDisplay character array after sorting");
        for(int i=0; i<c.length; i++)
        {  System.out.printf("%c\t",c[i]);
        }
  }
  public static void sort(int a[])
  {
    for(int i=0; i<a.length; i++)
        {
          for(int j=(i+1);j<a.length; j++)
              {
                if(a[i]>a[j])
                    {
                      int temp=a[i];
                      a[i]=a[j];
                      a[j]=temp;
                    }
              }
        }
  }
  public static void sort(char ch[])
  {
    for(int i=0; i<ch.length; i++)
        {
          for(int j=(i+1);j<ch.length; j++)
              {
                if(ch[i]>ch[j])
                    {
                      char temp=ch[i];
                       ch[i]=ch[j];
                       ch[j]=temp;
                    }
              }
        }

  }
}
```

Example using a function overloading

Suppose consider we want to design HR recruitment or hiring application for consultancy or for company we have following types of hiring categories

a. Fresher
b. Experience
c. FreeLancer

```
void hire(String name,Strig clgname,float tenPer,float twlPer,String grad,String skills)
{
}
void hire(String name,String prevComp,int prevSal,int expSalary,int expInyear,String
prevUAN,String prevSalary)
{
}
void hire(String name,String projName,Date startDate,Date endDate,int perHourRate)
{
}
```

Classes & Objects

Q. What is class?

---

class is a combination of state and behaviour
Here state means a variable declared within class and behaviour means function defined within class.

How to declare the class in JAVA?

```
access specifier class classname
{
    access specifier data type variablename;
          access specifier data type variablename;
          access specifier return type  functionname(datatype variablename)
          {
          }
}
```

Example:

```
public class Employee
{
          private int id;             //data or state
          private String name;
          private long salary;

          public void setDetails(){   //behaviour or method
            //here store data in employee object
          }
          public void showDetails()
          { //here display the employee data
          }
}
```

Code description

---

public class Employee :
public : public is access specifier in java
Access specifier are the some keywords which help us to apply restrictions on class and its member

Types of access specifier in JAVA

a. **public :** public access specifier indicates class or its members can access outside of class within the same package or outside of package.

b. **private :** private access specifier means member cannot access outside of class within the same package as well as outside of package.

c. **protected :** protected access specifier means member can access only in child class of the same package or outside of the package also.

d. **default :** default access specifier means member can access outside of class but within package not outside of package.
Note: when we not specify any access specifier with package then java by default use access specifier name as default

In depth definition of class : class is a combination of instance variable, class variable, method, constructor , instance initializer, static initializer and nested classes

Instance variable: instance variable means a variable declared within class but without a static keyword called as instance variable.

Class Variable: class variable means a variable declared within class with a static keyword called as class variable.

Method: method means a function define within class called as method in java

Constructor: function same name as class name called as constructor

Instance initializer: instance initializer means block without static keyword called as instance initializer.

Static initializer: static initializer means block with static keyword called as static initializer.

Nested class: class within class called nested class.

Example with source code

```
class ABC
{
  int x;//instance variable
  static int y; // class variable

  void show(){  //method
```

```
  }

  ABC(){
   //constructor
  }

  {
    //instance initializer
  }

  static{
    //static initializer
  }
  class MNO{ // this is your nested class
  }
}
```

Q. Why use class or What are the benefits of class?

---

a. **Ability to store different types of data**

```
public class Employee
{
        private int id; //data
        private String name;
        private long salary;

        public void setDetails(){   //behaviour or method
          //here store data in employee object
        }
        public void showDetails()
        { //here display the employee data
        }
}
```

b. **Reusability :** reusability means we can declare the class only once and can reuse it more than one time.

There are two ways to reuse the class

1. By creating object of class
2. **By using inheritance -** we will discuss later in OOP concept

Q. What is an object?

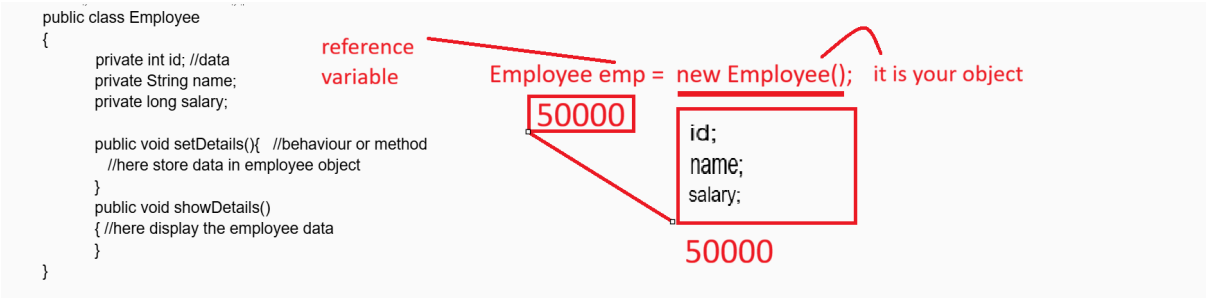
An object is an instance of a class or it is a runtime entity but if we think about an object we can say object is a block of memory where class data store.

How to create objects in JAVA?

If we want to create objects in java we have to use following syntax

Syntax: classname ref = new classname();
Example: Employee emp = new Employee();
If we think about above statement here emp is reference variable and new Employee() is actual object

Q. What is the difference between reference and object?
Reference is a variable which holds the address of object and object is a block of memory where we store class data.



Note: we will discuss above question after 4 to 5 examples

If we want to work with any class we have some important steps.

1. **Declare the class**
```
class Add  //step1 - declare class
{     private int a,b;

}
```

2. **Define the method within class**
```
import java.util.*;
class Add  //step1 - declare class
{
  private int a,b;
  Scanner xyz =new Scanner(System.in);
  void setValue() //step2 - define method within class
  { System.out.println("Enter two values");
    a=xyz.nextInt();
        b=xyz.nextInt();
  }
  void showAdd(){ //step2- define method within class
    System.out.println("Addition is  "+(a+b));
  }
}
```

3. **Create object of class**
   Add ad =new Add(); /
4. **Call its method using object or reference**

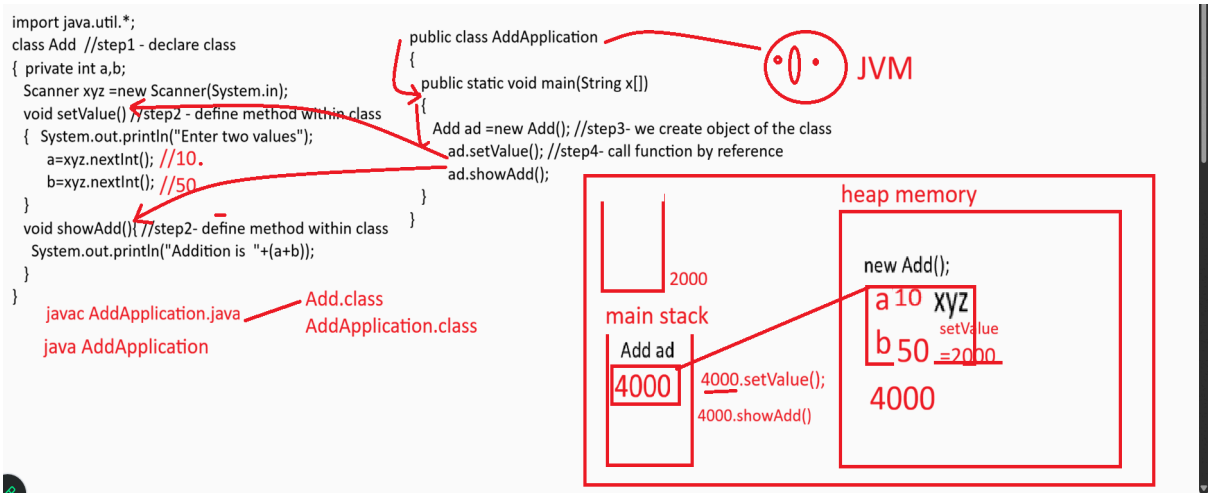
```
        ad.setValue(); //step4- call function by reference
                ad.showAdd();
```

Example with complete source code

AddApplication.java

```
import java.util.*;
class Add  //step1 - declare class
{
  private int a,b;
  Scanner xyz =new Scanner(System.in);
  void setValue() //step2 - define method within class
  { System.out.println("Enter two values");
    a=xyz.nextInt();
          b=xyz.nextInt();
  }
  void showAdd(){ //step2- define method within class
    System.out.println("Addition is  "+(a+b));
  }
}
public class AddApplication
{
  public static void main(String x[])
  {
     Add ad =new Add(); //step3- we create object of the class
          ad.setValue(); //step4- call function by reference
          ad.showAdd();
  }
}
```

Following diagram represent internal working of above code



Example: WAP to create class name as Cube with two methods

void setValue(): this function is used for accept input from keyboard

int getResult(): this function return cube at function calling point

```java
import java.util.*;
class Cube  //step1 - declare the class
{
   int no;
   Scanner xyz= new Scanner(System.in);
   void setValue()//step2 - define function within class
   { System.out.println("Enter number from keyboard");
      no=xyz.nextInt();
   }
   int getResult()
   {  return  no*no*no;
   }
}
public class CubeApplication
{ public static void main(String x[])
   {
      Cube c = new Cube(); //step3 - creating object of class
      c.setValue();//step4- call function
      int result =c.getResult();
      System.out.println("Cube is "+(result));
   }
```

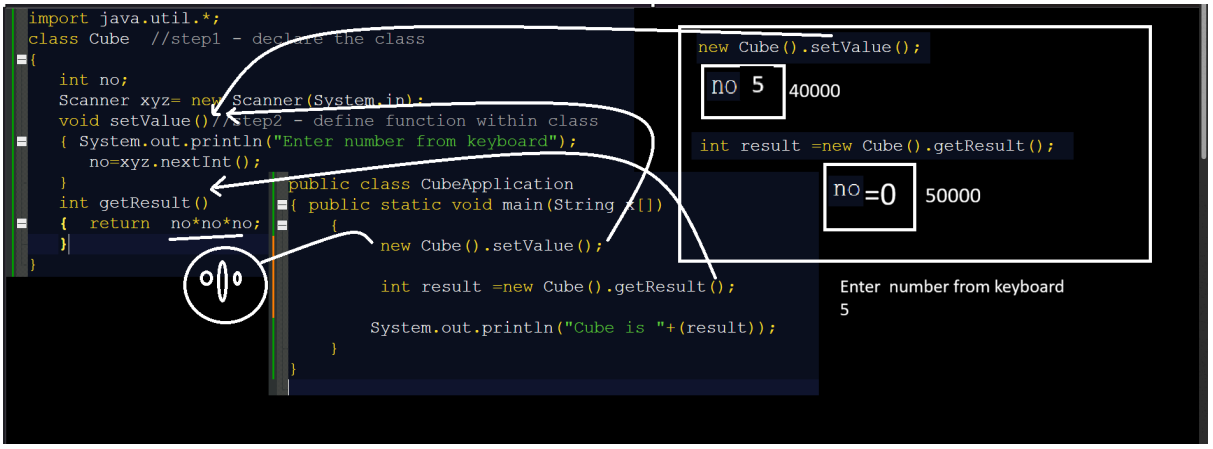
```
C:\Program Files\Java\jdk-24\bin>javac CubeApplication.java

C:\Program Files\Java\jdk-24\bin>java CubeApplication
Enter number from keyboard
5
Cube is 125

C:\Program Files\Java\jdk-24\bin>
```

Q. What happens when we do not use the reference with an object?

If we not use reference with object then JVM create new object every time in memory means we can use same object only once cannot use it more than one time shown in following diagram



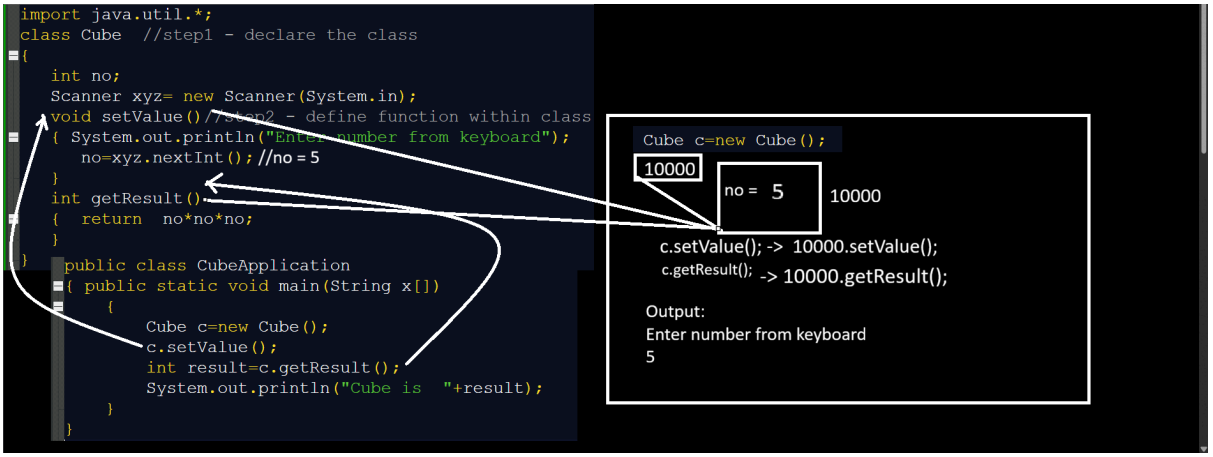
Suppose if we think about above diagram we have statement in mai method new Cube().setValue() - means here we create object in heap memory whose address is 40000 and we call function name as setValue() and we input value in no variable i.e no=5 but this value stored in no variable which is present in object whose address is 40000 Again we have one more statement: int result =new Cube().getResult() means we create second object in memory whose address is 50000 and we call getResult() function from object whose address is 50000 in getResult() we use no from 50000 block and we not set or store value in 50000 block memory so the default value is 0 so no*no*no is 0 so we get final answer 0

Conclusion: The conclusion of the above concept is we should not use references with objects we cannot use the same object more than one time so we can use it single time and without reference object known as anonymous object so it is recommended when we want to use object only once.

Q. Why use references with objects?

We can use reference with object in following cases

1. When you want to reuse same object more than one time
2. When we pass object as parameter or class data as parameter in some another classes

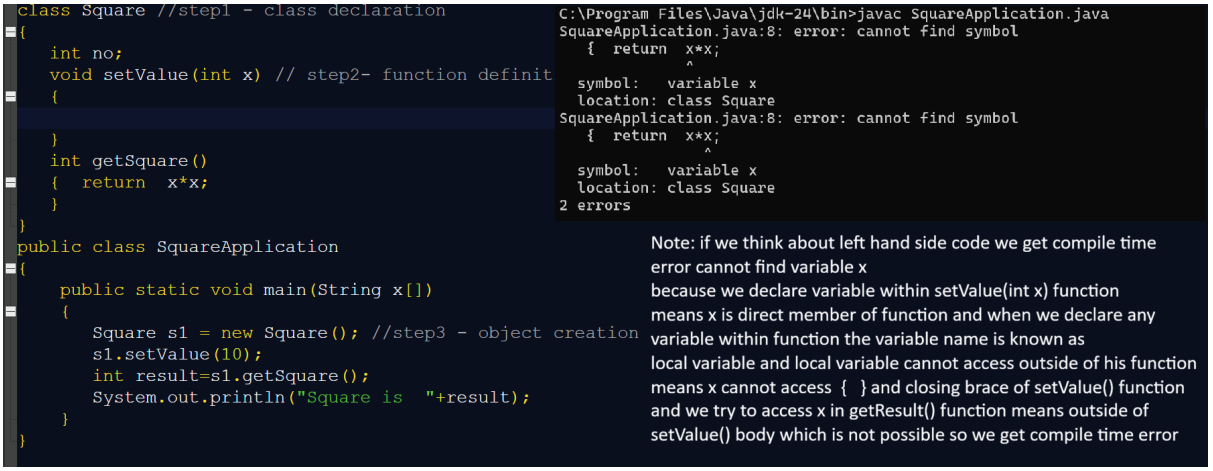


If we think about above diagram we have statement in main method Cube c = new Cube() Here c is reference variable which contain 10000 address which is memory address of object new Cube() and we call function c.setValue() meaning is 10000.setValue() means input value of no variable i.e 5 and stored in object whose address is 10000
Again i have statement c.getResult() means 10000.getResult() so here we not create new object just we reuse object whose address is 10000 by using a reference variable c
So we can say we can reuse same object more than once by using reference variable

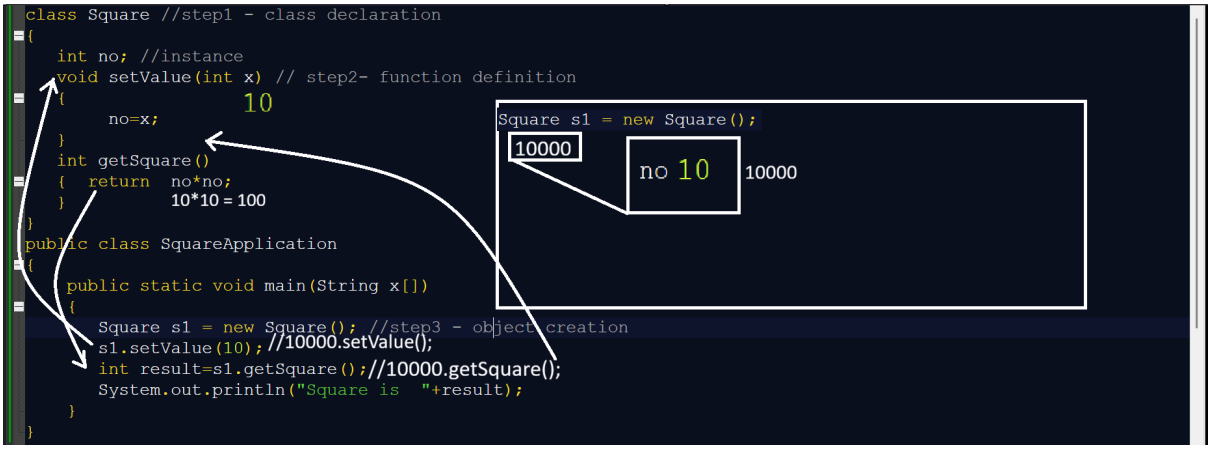
How to pass parameter to class function

Example: WAP to create class name as Square with two functions

1. void setValue(int x): this function accept x as integer parameter value
2. int getSquare(): this function calculates square of number and returns it.



Note: if we want to solve above error which we mention in screenshot we have to copy the value of local variable into instance variable
Instance variable is directly part of class means instance variable can access in all functions of class.



Example: WAP to create class name as Table with two functions
void setValue(int x): this function is used for accept number as parameter
void showTable(): this function is used for print table of number

```
class Table //step1- declare class
{   int no;
   void setValue(int x) //step2 - define function
   {  no=x;
   }
   void showTable()
   {
      for(int i=1; i<=10; i++)
            { System.out.printf("%d\n",no*i);
             }
   }
}
public class TableApplication
{
    public static void main(String x[])
          {
              Table t1 = new Table(); //step3 - create object of class
                    t1.setValue(10);  // step4- call function
                    t1.showTable();
          }
}
```

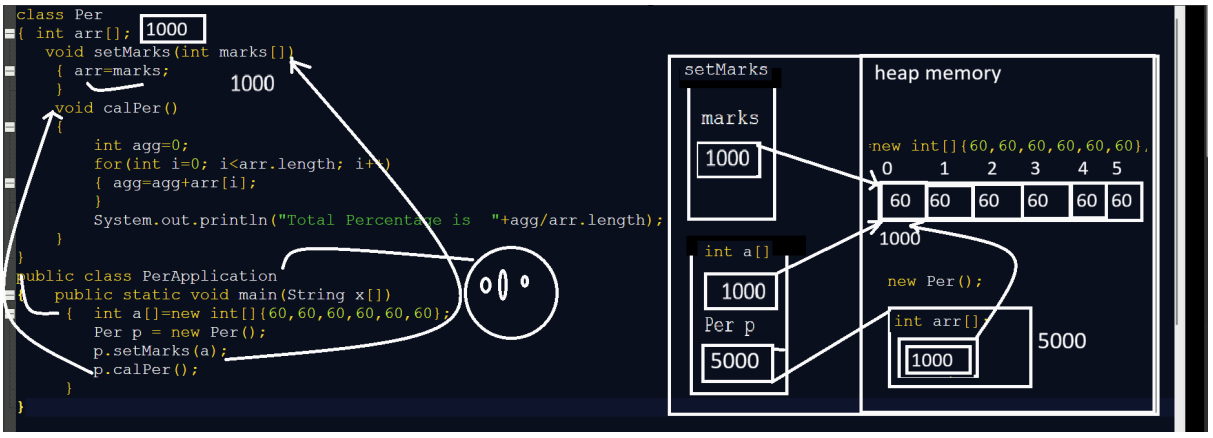
Example: WAP to create class name as Per with two functions

void setMarks(int s1,int s2,int s3,int s4,int s5,int s6): this function accept marks of six subjects
void calPer(): this function is used for calculate percentage of six subject marks and display it

Example with source code

```
class Per
{  int sub1,sub2,sub3,sub4,sub5,sub6;
    void setMarks(int s1,int s2,int s3,int s4,int s5,int s6)
          {
            sub1=s1;
            sub2=s2;
            sub3=s3;
            sub4=s4;
            sub5=s5;
            sub6=s6;
          }
          void calPer()
          {
            int agg=sub1+sub2+sub3+sub4+sub5+sub6;
            int per=agg/6;
            System.out.println("Percentage is  "+per);
          }
}
public class PerApplication
{
    public static void main(String x[])
          {
            Per p = new Per();
                  p.setMarks(60,60,60,60,60,60);
                  p.calPer();
          }
}
```

Note: if we think about above code we create object of Per class in main method and call setMarks() method of Per class and pass 6 parameter of type integer in it
But if we have more than 3 parameter of same type in function then standard say pass array as parameter in function
Means according to our example you have to create array of size six store all subjects marks in that array and pass base address or reference variable of array as parameter in function



Example: Create class name as UpperCase with following methods

void setString(String data): this method is used for accept string as parameter
String getUpperCase(): this method can convert your lower case string to upper case and return it.

Example with source code

```
import java.util.*;
class UpperCase
{
  String str;
  void setString(String data)
  {str=data;
  }
  String getUpperCase(){
    String s="";
          char ch=' ';
          for(int i=0; i<str.length(); i++)
          {
              if(str.charAt(i)>='a' && str.charAt(i)<='z')
                    {    ch=(char) ((int)str.charAt(i)-32);
                    }
                   s=s+""+ch;
          }
          return s;
  }
}
public class UpperCaseApplication
{
   public static void main(String x[])
         {
            UpperCase uc=new UpperCase();
                  Scanner xyz = new Scanner(System.in);
                  System.out.println("Enter string data");
                  String d=xyz.nextLine();
                  uc.setString(d);

                  String result=uc.getUpperCase();
                  System.out.println(result);
         }
}
```

Example: WAP to create class name as Count words from string
You have to Create class name as WordCounter with two methods

void setValue(String str): this method is used for accept String as parameter
int getWordCount(): this method can count word and return word count

```
class WordCounter //step1 - declare class
{
   String str;
        void setValue(String data) //step2 - define function within class
        { str=data;
        }
        int getWordCount()
        {
          String words[]=str.split(" ");
          return words.length;
        }
}
public class WordCounterApplication
{
   public static void main(String x[])
        {
          WordCounter wc=new WordCounter();
          wc.setValue("good morning india");

          int numWord=wc.getWordCount();
          System.out.println(numWord);
        }
}
```

Example: WAP to find max value from Array
Create class name as MaxFinder with two methods
void setArray(int a[]): this function accept array as parameter
int getMax(): this function finds the max value and returns it.

```
import java.util.*;
class MaxFinder //step1- declare class
{   int m[];
   void setArray(int arr[]) //step2 -define function within class
        {  m=arr;
        }
        int getMax()
        {  int max=m[0];
          for(int i=0; i<m.length; i++)
          {
               if(m[i]>max)
                         {  max=m[i];
                         }
          }
          return max;
        }
}
public class MaxFinderApp
```

```
{
   public static void main(String x[])
         {  int a[]=new int[5];
         Scanner xyz=new Scanner(System.in);
         System.out.println("Enter values in array");
         for(int i=0; i<a.length; i++)
         {  a[i]=xyz.nextInt();
         }
           MaxFinder mf = new MaxFinder();
           mf.setArray(a);//base address of array in function
           int maxValue=mf.getMax();
           System.out.println("Max value is  "+maxValue);
         }
}
```

POJO class concept

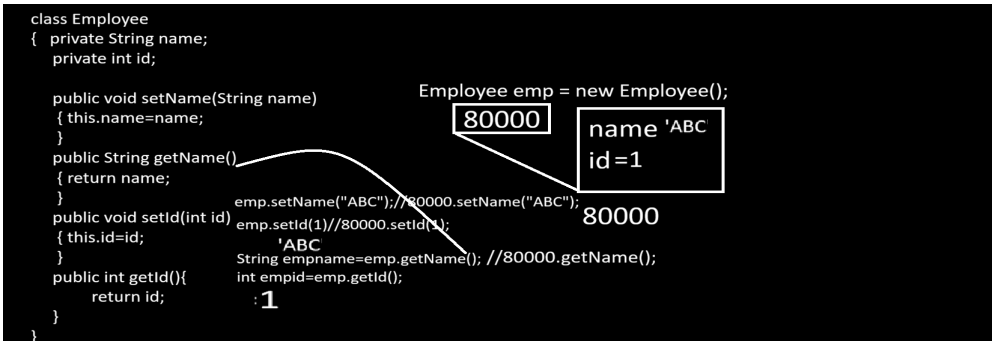
POJO stands for plain old java objects - basically it is Java class which contain setter and getter methods
This class work as container class
Using a setter method we can store data in objects and using a getter method we can retrieve data from objects.
The standard of Writing POJO class is use set and get keyword with variable at the time of function definition and declare class variable as private

Standard rule of writing POJO class

---

1. Variable must be private
2. Use set and get keyword with variable name when we define method
3. Setter method is used for store data in object
4. Must be pass single parameter in setter method & return type of setter method should be void
5. Getter method must be return some data or value from method
6. Parameter type of setter method and return type of getter method must be same
7. When your method return type is boolean then method name should be start with is or has keyword like as isStatus() or hasStatus(), isStudent(),isAdmin() etc

Note: these are bean standard set by JAVA - bean indicate class



```
import java.util.*;
class Employee
{  private int id;
   private String name;

   public void setName(String name)
   { this.name=name;
   }
   public String getName()
   { return name;
   }
   public void setId(int id)
   { this.id=id;
   }
   public int getId()
   { return id;
   }
}
public class POJOAPP
{  public static void main(String x[])
   {      Employee emp = new Employee();
              emp.setId(1);
              emp.setName("ABC");
              String empname=emp.getName();
              int empid=emp.getId();
              System.out.println("Name is  "+empname);
              System.out.println("Id is   "+empid);
   }
}
```

Example: WAP POJO class name as Player with field id,name and run store data in Player object and retrieve data from player object.

```
class Player
{
   private int id;
   private String name;
   private int run;

   public void setId(int id)
   { this.id=id;
   }
   public int getId()
   { return id;
   }
   public void setName(String name)
   { this.name=name;
   }
   public String getName(){
    return name;
```

```
  }
  public void setRun(int run)
  { this.run=run;
  }
  public int getRun()
  { return run;
  }
}
public class PlayerApp
{
  public static void main(String x[])
  {
    Player p1=new Player();
          p1.setId(1);
          p1.setName("ABC");
          p1.setRun(10000);

         System.out.println(p1.getId()+"\t"+p1.getName()+"\t"+p1.getRun());
  }
}
```

Q. Why use POJO class?

Suppose we have method which contain different types of parameters and if parameter length is more than three then passing individual parameter is not good approach so better way you store all parameters in POJO class and pass POJO class reference as parameter and it is known as model class in project

Example: we have class name as Company with method
void addNewEmployee(String name,int id,int sal,String address,String qual): this method is used for accept employee details
void showEmployeeDetail(): this method is used for display the employee details

Example with source code

```
class Company
{  private String name;
  private int id;
  private int sal;
  private String address;
  private String qual;
  void addNewEmployee(String name,int id,int sal,String address,String qual)
  {
     this.name=name;
          this.id=id;
          this.sal=sal;
          this.address=address;
          this.qual=qual;
```

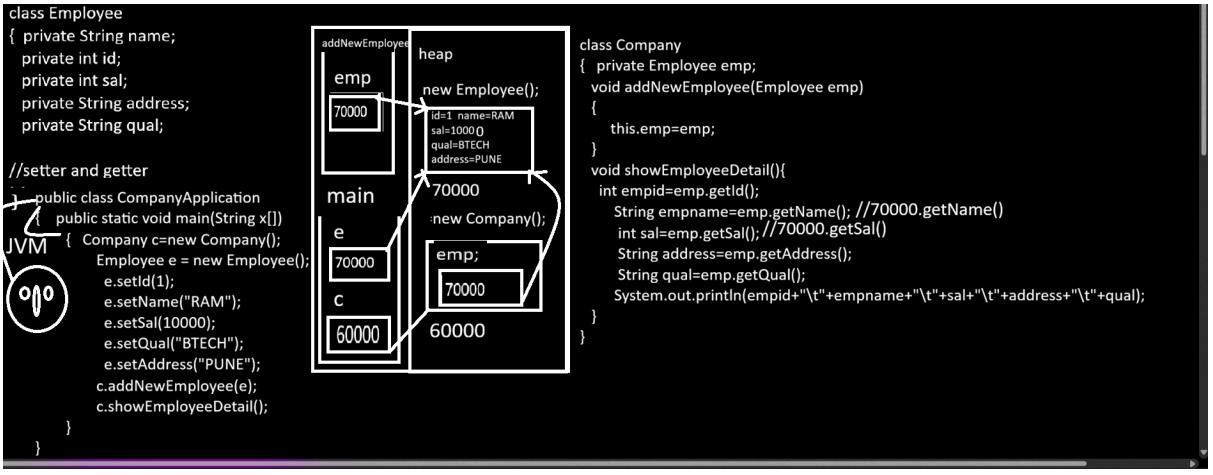
```
  }
  void showEmployeeDetail(){
   System.out.println(name+"\t"+id+"\t"+sal+"\t"+address+"\t"+qual);
  }
}
public class CompanyApplication
{
   public static void main(String x[])
        {
           Company c=new Company();
                c.addNewEmployee("RAM",1,10000,"PUNE","MTECH");
                c.showEmployeeDetail();
        }
}
```

Note: if we think about above code there is some limitations

1. If we think about addNewEmployee() it contain 5 parameters of different types as developer when we call this method we need to remember all parameters list, its type and sequence of parameter
2. Need to maintain parameter sequence and its type from calling point
3. We must have to pass all parameter to function definition from calling we cannot skip it if we try to skip it then we get compile time error

If we want to solve this problem java suggest us store all parameters in single class i.e POJO class and Pass POJO class as reference in method where we want to use these parameters

Means according to our example we have 5 parameters name,id,sal,qual and address we have to store these all parameters in single class e.g Employee class and pass employee class reference as parameter in addNewEmployee() method show in following




```
class Employee
{  private String name;
   private int id;
   private int sal;
   private String address;
```

```
   private String qual;

   public void setId(int id)
   { this.id=id;
   }
   public int getId(){
               return id;
   }
   public void setName(String name)
   { this.name=name;
   }
   public String getName()
   { return name;
   }
   public void setSal(int sal)
   { this.sal=sal;
   }
   public int getSal()
   { return sal;
   }
   public void setQual(String qual)
   { this.qual=qual;
   }
   public String getQual()
   { return qual;
   }
   public void setAddress(String address)
   { this.address=address;
   }
   public String getAddress()
   { return address;
   }
}
class Company
{   private Employee emp;
   void addNewEmployee(Employee emp)
   {
       this.emp=emp;
   }
   void showEmployeeDetail(){
     int empid=emp.getId();
            String empname=emp.getName();
             int sal=emp.getSal();
             String address=emp.getAddress();
             String qual=emp.getQual();
            System.out.println(empid+"\t"+empname+"\t"+sal+"\t"+address+"\t"+qual);
   }
}
```

```
public class CompanyApplication
{    public static void main(String x[])
        {   Company c=new Company();
                Employee e = new Employee();
                  e.setId(1);
                  e.setName("RAM");
                  e.setSal(10000);
                  e.setQual("BTECH");
                  e.setAddress("PUNE");
                c.addNewEmployee(e);
                c.showEmployeeDetail();
        }
}
```

Array of objects

Array of objects is used to store more than one object data in a single name reference called an array of objects.

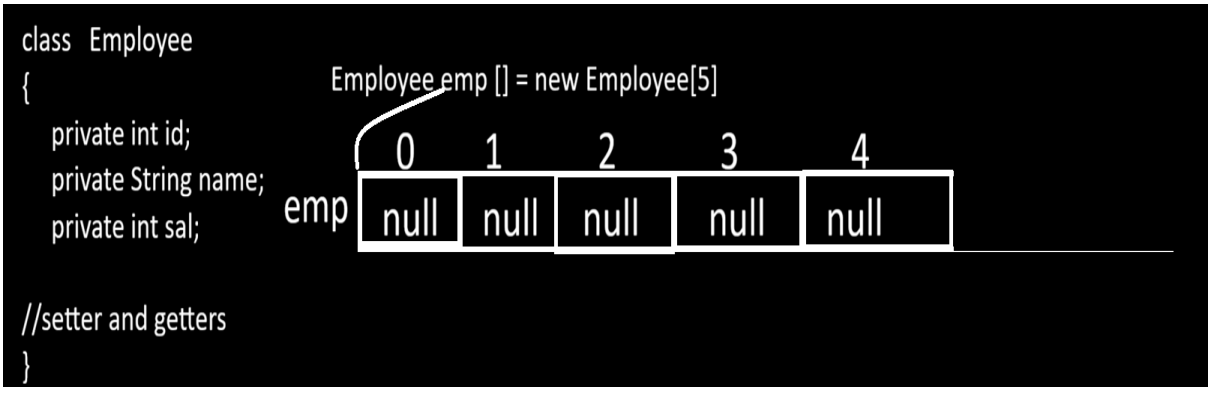
Example: Suppose we have Employee class and we need 5 employee data then we have to create an object of 5 employees then we need to create 5 objects manually. A better way to create 5 objects separately we can create an array of objects and store 5 employees data in it.

How to create array of objects

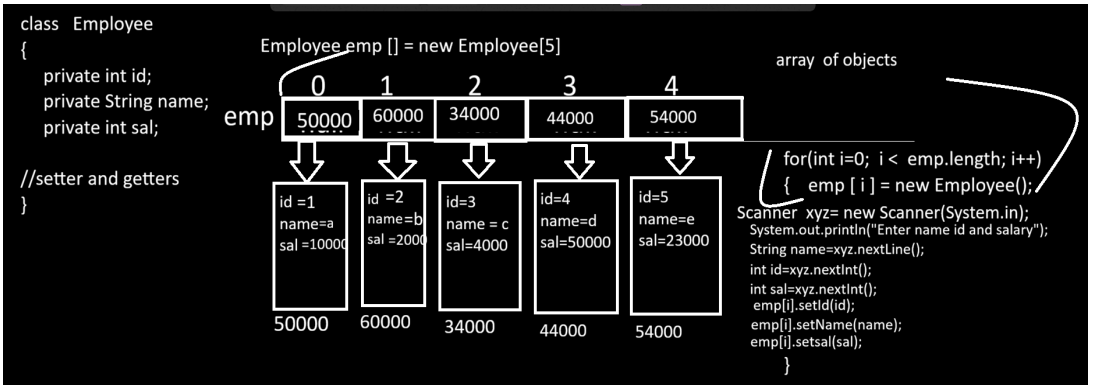
There are two steps involved in creating array of objects

1. **Create array of references :** we need to create references or declare the references because references help us to hold the address of objects and if we have an array object means we need an array of references also for holding objects address.

   Syntax: classname ref[]=new classname[size];//array of references
   Example: Employee emp[]=new Employee[5];



2. **Create array of objects :** here we need to create array of objects and store every object address in employee references index



Example with source code

```
import java.util.*;
class Employee
{  private int id;
   private String name;
   private int sal;
   public void setId(int id)
   { this.id=id;
   }
   public int getId()
   { return id;
   }
   public void setName(String name)
   { this.name=name;
   }
   public String getName()
   { return name;
   }
   public void setSal(int sal)
   { this.sal=sal;
   }
   public int getSal()
   { return sal;
   }
}
public class EmployeeApplication
{
    public static void main(String x[])
          {   Scanner xyz = new Scanner(System.in);
              Employee emp[]=new Employee[5];//array of references
                    for(int i=0; i<emp.length; i++)
                    {  emp[i]=new Employee();//array objects
                    System.out.println("Enter name id and salary of employee");
                    String name=xyz.nextLine();
```

```
                    int id=xyz.nextInt();
                    int sal=xyz.nextInt();
                    emp[i].setName(name);
                    emp[i].setId(id);
                    emp[i].setSal(sal);
                    xyz.nextLine();
                   }
                   System.out.println("Display the all employee data");
                   for(int i=0; i<emp.length; i++)
                   {
System.out.println(emp[i].getId()+"\t"+emp[i].getName()+"\t"+emp[i].getSal());
                   }

         }
}
```

Example: WAP to create class name as Player with field id,name and run and we have create 5 objects of player and sort player data by using id of player

```
import java.util.*;
class Player
{
  private int id;
  private String name;
  private int run;

  public void setId(int id)
  { this.id=id;
  }
  public int getId()
  { return id;
  }
  public void setName(String name)
  { this.name=name;
  }
  public String getName()
  { return name;
  }
  public void setRun(int run)
  { this.run=run;
  }
  public int getRun()
  { return run;
  }
}
public class PlayerApplication
{   public static void main(String x[])
    {
```

```
        Player p[]=new Player[5];//array of references
            for(int i=0; i<p.length; i++)
            { Scanner xyz =new Scanner(System.in);
              System.out.println("enter name id and run of player");
              String name=xyz.nextLine();
              int id=xyz.nextInt();
              int run=xyz.nextInt();
              p[i]=new Player();//array of objects
              p[i].setId(id);
              p[i].setName(name);
              p[i].setRun(run);
            }
            System.out.println("display the all player before sorting");
            for(int i=0; i<p.length; i++)
            {
               System.out.println(p[i].getId()+"\t"+p[i].getName()+"\t"+p[i].getRun());
            }
            //sorting logics
            for(int i=0;i<p.length; i++)
            {
                    for(int j=(i+1); j<p.length; j++)
                    {
                            if(p[i].getId()>p[j].getId())
                            {
                                    Player temp=p[i];
                                    p[i]=p[j];
                                    p[j]=temp;
                            }
                    }
            }
        System.out.println("display the all player after sorting");
            for(int i=0; i<p.length; i++)
            {
               System.out.println(p[i].getId()+"\t"+p[i].getName()+"\t"+p[i].getRun());
            }
    }
}
```

Example: WAP to create Product class with field id,name and price of product and create array of objects and give sum of all products or stock total amount

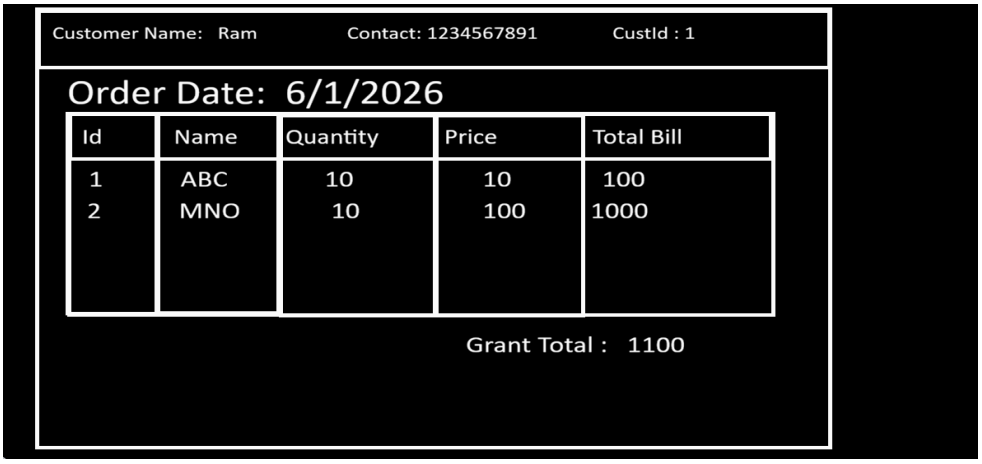
```
import java.util.*;
class Product
{
  private int id;
  private String name;
  private int price;

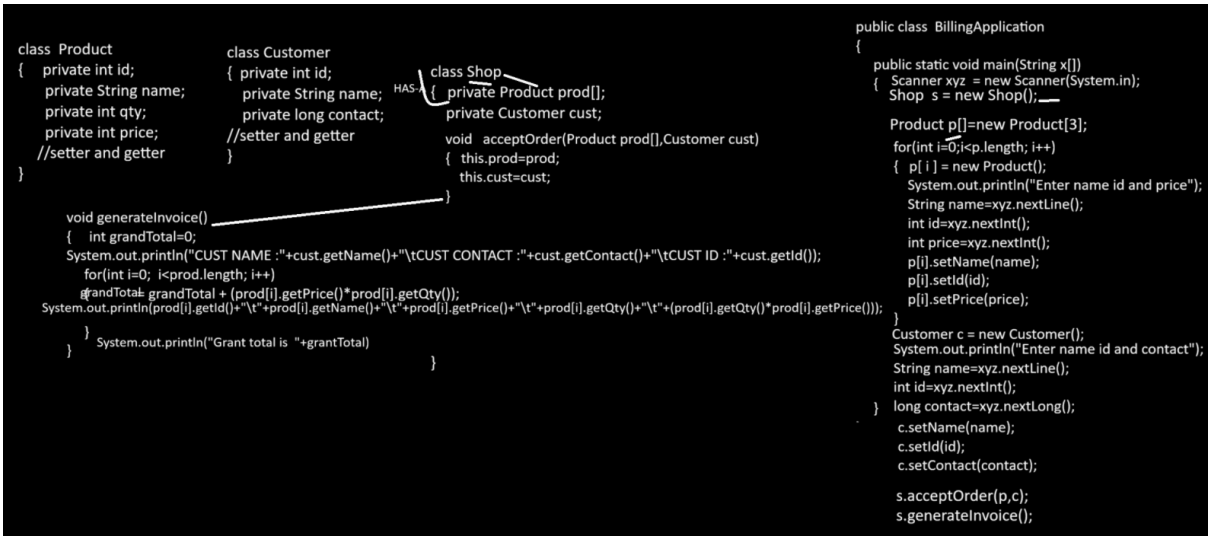
  public void setId(int id)
```

```
  { this.id=id;
  }
  public int getId()
  { return id;
  }
  public void setName(String name)
  { this.name=name;
  }
  public String getName()
  { return name;
  }
  public void setPrice(int price)
  { this.price=price;
  }
  public int getPrice()
  { return price;
  }
}
public class ProdApplication
{
   public static void main(String x[])
        {
          Product p[]=new Product[5];
              for(int i=0; i<p.length; i++)
              {
                p[i]=new Product(); //array of objects
                Scanner xyz  = new Scanner(System.in);
                System.out.println("enter name id and price of the product");
                String name=xyz.nextLine();
                int id=xyz.nextInt();
                int price=xyz.nextInt();
                p[i].setName(name);
                p[i].setId(id);
                p[i].setPrice(price);
              }
              System.out.println("display the data of products");
              int sum=0;
              for(int i=0; i<p.length;i++)
              {   sum =sum+p[i].getPrice();
                System.out.println(p[i].getId()+"\t"+p[i].getName()+"\t"+p[i].getPrice());
              }
              System.out.println("Your Total Stock value is  "+sum);
        }
}
```

Example: Billing Application using Array of objects concept

Objective : The objective of this assignments is to generate the invoice for customer like as





Example with complete source code

```
import java.util.*;
class Product
{
  private int id;
  private String name;
  private int price;
  private int qty;

  public void setQty(int qty)
  { this.qty=qty;
  }
  public int getQty()
  { return qty;
  }

  public void setId(int id)
```

```
  { this.id=id;
  }
  public int getId()
  { return id;
  }
  public void setName(String name)
  { this.name=name;
  }
  public String getName()
  { return name;
  }
  public void setPrice(int price)
  { this.price=price;
  }
  public int getPrice()
  { return price;
  }
}
class Customer
{
   private int id;
   private String name;
   private long contact;

   public void setId(int id)
   { this.id=id;
   }
   public int getId()
   { return id;
   }
   public void setName(String name)
   { this.name=name;
   }
   public String getName()
   { return name;
   }
   public void setContact(long contact)
   { this.contact=contact;
   }
   public long getContact(){
    return contact;
          }
}
class Shop
{
   Product prod[];
   Customer cust;
   void acceptOrder(Product prod[],Customer cust)
```

```
  { this.prod=prod;
    this.cust=cust;
  }
  void generateInvoice()
  {
    int grandTotal=0;
          System.out.println("Cust Name :"+cust.getName()+"\tCust Id :"+cust.getId()+"\tCust
Contact "+cust.getContact());

System.out.println("==========================================================
================");
          System.out.println("\nID\tNAME\tQuantity\tPrice\tTotal\n");
          for(int i=0; i<prod.length; i++)
          {
            grandTotal=grandTotal+(prod[i].getQty()*prod[i].getPrice());

System.out.println(prod[i].getId()+"\t"+prod[i].getName()+"\t"+prod[i].getQty()+"\t\t"+prod[i].get
Price()+"\t\t"+(prod[i].getQty()*prod[i].getPrice()));
          }

System.out.println("==========================================================
================");
          System.out.println("\t\t\t\tGrand Total is  "+grandTotal);
  }
}
public class BillingApplication
{   public static void main(String x[])
  {  Scanner xyz=new Scanner(System.in);
     Shop s=new Shop();
          Product p[]=new Product[3];
          for(int i=0; i<p.length; i++)
          { System.out.println("Enter name id and price ,quantity of product");
            String name=xyz.nextLine();
                int id=xyz.nextInt();
                int price=xyz.nextInt();
                int qty=xyz.nextInt();
                p[i]=new Product();
                 p[i].setName(name);
                 p[i].setId(id);
                 p[i].setPrice(price);
                 p[i].setQty(qty);
                 xyz.nextLine();
          }
          Customer c=new Customer();
          System.out.println("Enter name id and contact of customer");
          String custname=xyz.nextLine();
          int id=xyz.nextInt();
          long contact=xyz.nextLong();
```

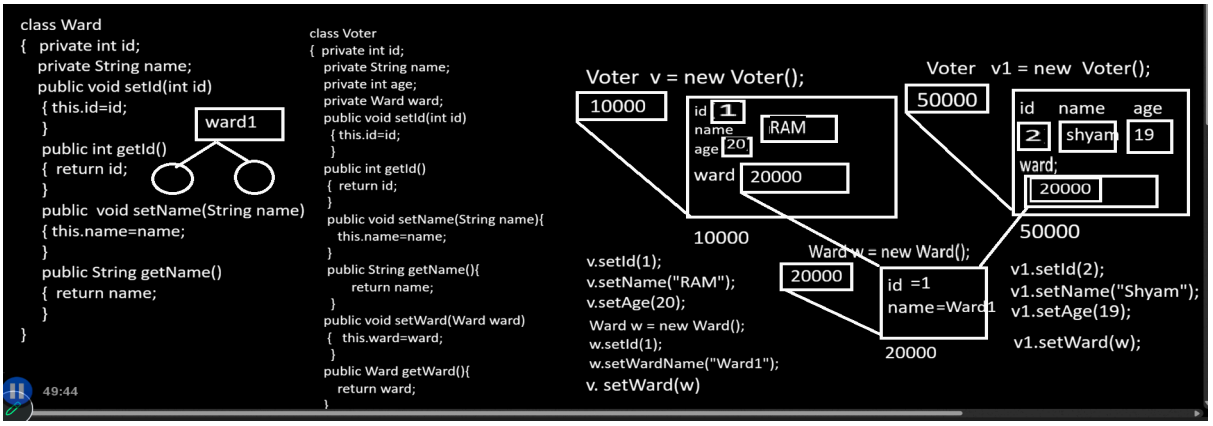
```
            c.setName(custname);
            c.setId(id);
            c.setContact(contact);

            s.acceptOrder(p,c);
            s.generateInvoice();
    }
}
```

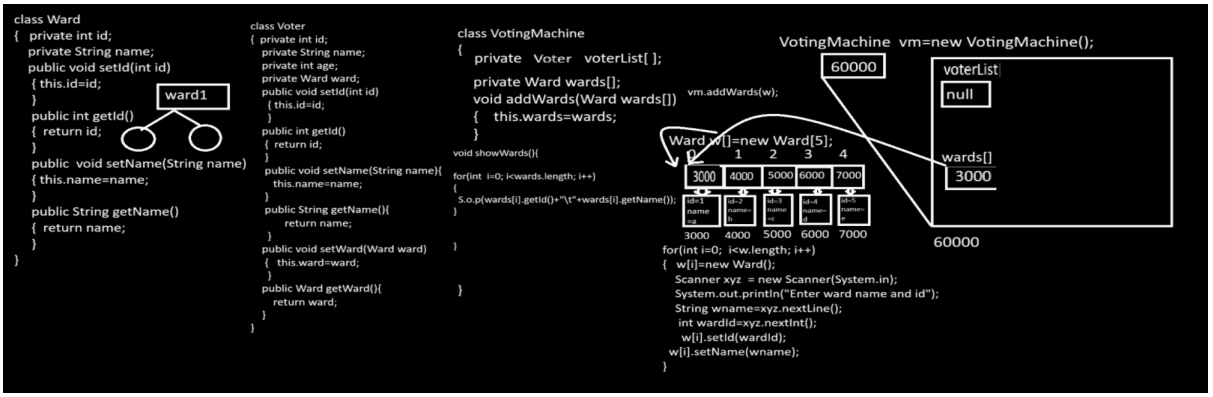
Example: Suppose consider we are working on VotingApplication and we have following task

1. **Add new voter :** here we have voter details but voter should accept by machine if his age is greater than 18.
2. **View all voters :** display the all voters
3. **Delete voter :** delete voter by using voter id
4. **Update voter :** update voter by using voter id
5. **Search voter** : search voter by using its id
6. **Count voter**
7. Arrange voter info by ward
8. Arrange voter details by age

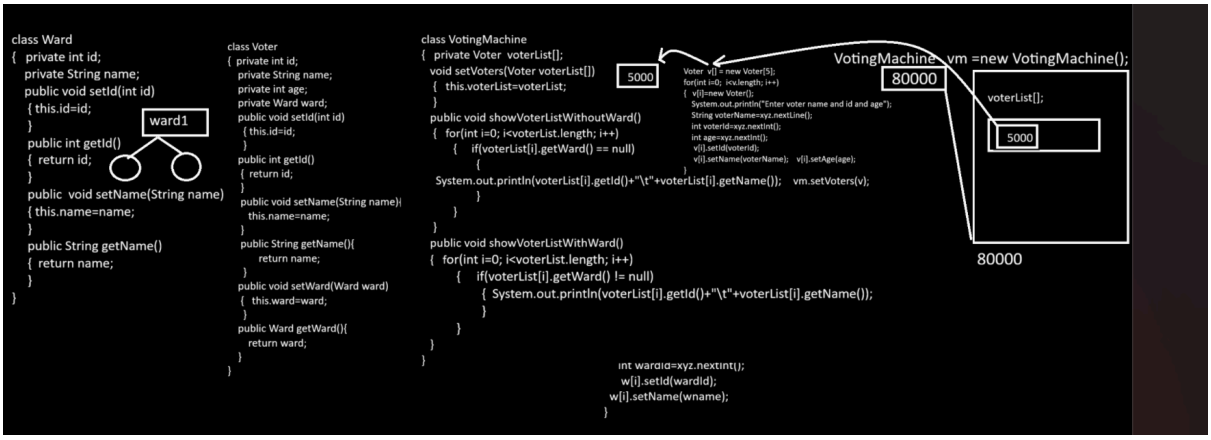
Basic class structure for above problem statement (class structure 1.0)



Following diagram shows the add ward to voting machine and display its data



Following diagram show the add voter to VotingMachine



Example with source code

```
import java.util.*;
class Ward
{
  private int id;
  private String name;
  public void setId(int id)
  { this.id=id;
  }
  public int getId()
  { return id;
  }
  public void setName(String name)
  { this.name=name;
  }
  public String getName(){
   return name;
  }
}
class Voter
{
        private int id;
        private String name;
        private int age;
        private Ward ward;
        private boolean status;
        private boolean voteStatus;

        public void setVoteStatus(boolean voteStatus)
        { this.voteStatus=voteStatus;
        }
        public boolean isVoterStatus()
        {  return voteStatus;
        }
```

```
        public void setId(int id)
        { this.id=id;
        }
        public int getId()
        { return id;
        }
        public void setName(String name)
        { this.name=name;
        }
        public String getName(){
                return name;
        }
        public void setAge(int age)
        { this.age=age;
        }
        public int getAge(){
                return age;
        }
        public void setWard(Ward ward)
        { this.ward=ward;
        }
        public Ward getWard(){
                return ward;
        }
        public void setStatus(boolean status)
        { this.status=status;
        }
        public boolean isStatus(){
                return status;
        }

}
class VotingMachine
{
   private Ward wards[];
   private Voter voterList[];
   public void setWards(Ward wards[])
   { this.wards=wards;
   }
   public void showWards()
   {  System.out.println("ID\tNAME");
      for(int i=0;i<wards.length; i++)
        { System.out.println(wards[i].getId()+"\t"+wards[i].getName());
        }
   }
   public void setVoters(Voter voterList[])
   { this.voterList=voterList;
   }
```

```
  public void showVoterListWithoutWard()
  {  boolean flag=true;
     for(int i=0;i<voterList.length; i++)
           {
         if(voterList[i].getWard()==null)
                    {  flag=false;
System.out.println(voterList[i].getId()+"\t"+voterList[i].getName()+"\t"+voterList[i].getAge());
                    }
           }
           if(flag)
           { System.out.println("no voter without ward");
           }
  } //5  - 5
   public void showVoterListWithWard()
  {   int count=1;
     for(int i=0;i<voterList.length; i++)
           {
         if(voterList[i].isStatus())
                    {  ++count;
                     Ward w=voterList[i].getWard();
System.out.println(voterList[i].getId()+"\t"+voterList[i].getName()+"\t"+voterList[i].getAge()+"\t"
+w.getName());
                    }
           }
           if(count==1)
           { System.out.println("All voters without ward");
           }
           else if(count<voterList.length)
           {System.out.println("some voters without ward");
           }

  }

  void allocateWardToVoter(){
           System.out.println("WardID\tWARDNAME\n");
           for(int i=0; i<wards.length; i++)
           {
       System.out.println(wards[i].getId()+"\t"+wards[i].getName());
           }
           System.out.println("VID\tVNAME\tVAGE");
           for(int i=0; i<voterList.length; i++)
           {
System.out.println(voterList[i].getId()+"\t"+voterList[i].getName()+"\t"+voterList[i].getAge()+"\t"
+voterList[i].isStatus());
           }
           Scanner xyz  = new Scanner(System.in);
           //ward allocation logics
           System.out.println("Enter Ward id");
```

```
            int wid=xyz.nextInt();
            System.out.println("Enter  Voter id");
            int vid=xyz.nextInt();
            boolean flag=true;
            Ward ward=wards[0];
            for(int i=0;i<wards.length;i++)
            {
                        if(wards[i].getId()==wid)
                              {  flag=false;
                                        ward=wards[i];
                                 break;
                              }
            }
            for(int i=0; i<voterList.length;i++)
            {

                        if(voterList[i].getId()==vid && ward.getId()==wid)
                       {  flag=false;
                                 Ward w=ward;
                                 voterList[i].setWard(w);//
                                 voterList[i].setStatus(true);
                       }
            }
            if(flag)
            { System.out.println("No ward or voter may be present");
            }
  }
  void vote(int voterId)
  {
             for(int i=0; i<voterList.length; i++)
                   {
                              if(voterList[i].getId()==voterId && voterList[i].isVoterStatus()==false
&& voterList[i].isStatus()==true)
                              {
                                        voterList[i].setVoteStatus(true);
                              }
                   }
  }
  void checkVoterAttendance(){
             for(int i=0;i<voterList.length;i++)
             {
                        if(voterList[i].isVoterStatus())
                        {
             Ward w=voterList[i].getWard();
           System.out.println(voterList[i].getId()+"\t"+voterList[i].getName()+"\t"+w.getName());
                        }
             }
  }
```

```
}
public class VotingApplication
{
    public static void main(String x[])
          { VotingMachine vm = new VotingMachine();
            do{
              Scanner xyz =new Scanner(System.in);
                    System.out.println("1:Add New Wards");
                    System.out.println("2:View Wards");
                    System.out.println("3:Add New Voters");
                    System.out.println("4:Show voter without ward");
                    System.out.println("5:Show Voter with ward");
                    System.out.println("6:Allocate ward to Voters");
                    System.out.println("7:Vote");
                    System.out.println("8:Check voter attendance");
                    System.out.println("9:Exit");
                    System.out.println("Enter your choice");
                    int choice=xyz.nextInt();
                    xyz.nextLine();
                    switch(choice)
                     {
                       case 1:
                              Ward w[]=new Ward[2];
                              for(int i=0;i<w.length;i++)
                              { w[i]=new Ward();
                                System.out.println("Enter ward name and id");
                                String wardName=xyz.nextLine();
                                int wardId=xyz.nextInt();
                                w[i].setName(wardName);
                                w[i].setId(wardId);
                                 xyz.nextLine();
                              }
                              vm.setWards(w);//set wards to voting machine
                             break;
                             case 2:
                               vm.showWards();
                             break;
                             case 3:
                              Voter voterList[]=new Voter[5];
                              for(int i=0; i<voterList.length;i++)
                              {
                                        voterList[i]=new Voter();
                                        System.out.println("Enter voter name id and age");
                                        String voterName=xyz.nextLine();
                                        int voterId=xyz.nextInt();
                                        int voterAge=xyz.nextInt();
                                        voterList[i].setId(voterId);
                                        voterList[i].setName(voterName);
```

```
                    voterList[i].setAge(voterAge);
                    xyz.nextLine();
                }
                vm.setVoters(voterList);
                break;
                case 4:
                 vm.showVoterListWithoutWard();
                break;
                case 5:
                 vm.showVoterListWithWard();
                break;
                case 6:
                vm.allocateWardToVoter();
                break;
                case 7:
                System.out.println("Enter voter id");
                int voterId=xyz.nextInt();
                vm.vote(voterId);
                break;
                case 8:
                vm.checkVoterAttendance();
               case 9:
                System.exit(0);
               break;
               default:
                System.out.println("Wrong choice");
            }
        }while(true);//infinite loop
    }
}
```

Q. What is variable argument concept or var -args?

Variable argument is facility in java where we can pass infinite parameter to the methods and it is denoted by …

Syntax: return type methodname(datatype …variablename)
{ write here your logics
}

Note: … indicate dynamic array at runtime internally

```
class Sum
{   int s=0;
    void calSum(int ...x)
    {
        for(int i=0; i<x.length; i++)
        {
          s=s+x[i];
        }
        System.out.printf("Sum is %d\n",s);
    }
}
public class CalSum
{
    public static void main(String x[])
    {
        Sum s =new Sum();
        s.calSum(10,20,30,40,50);
    }
}
```

10 20 30 40 50

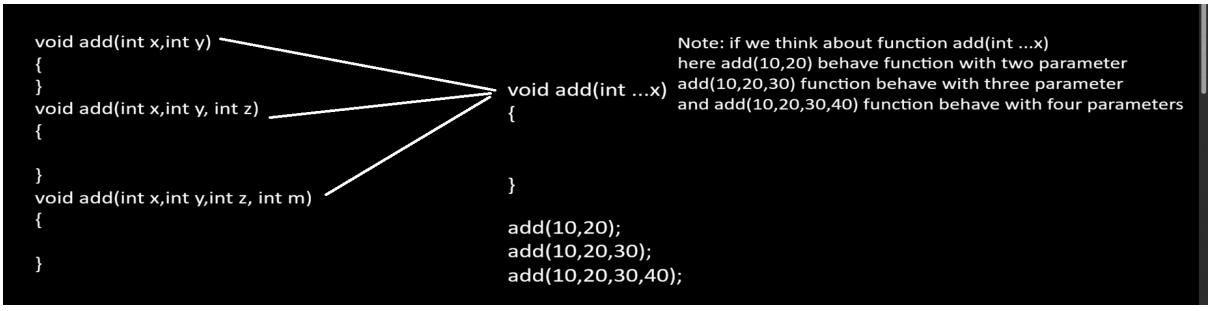
4:36

Q. What is the benefit of the variable arguments concept?

---

1. Pass infinite parameter
2. Variable arguments help us to overcome problem of function overloading
   Note: if we have function overloading with same data type then we need to define multiple function for that identify list of parameter so better way we can variable arguments

   Example: we want to calculate of two values, three values, four values etc



If we want to work with variable arguments we have some important rules

1. … must be left hand side variable
2. Cannot pass more than one variable in single function
3. If we have some simple parameter and variable argument in single function then variable argument must be the last parameter

```
class Sum
{   int s=0;
    void calSum(String name,int ...x)
    { System.out.println("Name is "+name);
        for(int i=0; i<x.length; i++)
        {
          s=s+x[i];
        }
        System.out.printf("Sum is %d\n",s);
    }
}
public class CalSum
{
    public static void main(String x[])
    {
        Sum s =new Sum();
        s.calSum("ABC",10,20,30,40,50);
    }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac CalSum.java

C:\Program Files\Java\jdk-24\bin>java CalSum
Name is ABC
Sum is 150

C:\Program Files\Java\jdk-24\bin>
```

21:00

Q. How can we pass more than one variable argument to a single function?

If we want to pass multiple variables to the function then store all parameters in POJO class and pass POJO class as parameter and use variable argument with a POJO class.

```
class Employee
{   private int id;
    private String name;
    private int sal;
 //setter and getter
}
class Company{

     void addEmployee(Employee  ...emp)
      {
      }
}
```

Note: if we think about addEmployee(Employee ...emp) in this method we pass infinite parameter to Employee and if we think about employee class object it contain three field id,name and sal
means when we pass single employee data we pass id,name and sal
and when we pass infinite employee data then we pass infinite ids, infinite name and infinite salaries
means indirectly we use multiple type variable argument in same function

Example with source code

```
import java.util.*;
class Employee
{
  private int id;
  private String name;
  private int sal;

  public void setId(int id)
  { this.id=id;
  }
  public int getId()
  { return id;
  }
  public void setName(String name)
  { this.name=name;
  }
  public String getName()
  { return name;
  }
  public void setSal(int sal)
  { this.sal=sal;
  }
  public  int getSal()
  { return sal;
  }
}
class Company
{
   void addEmployee(Employee ...emp)
        {
           for(int i=0; i<emp.length; i++)
                {
```

```
                    System.out.println(emp[i].getId()+"\t"+emp[i].getName()+"\t"+emp[i].getSal());
                   }
          }
}
public class VarPOJOAPP
{
    public static void main(String ...x)
          {
            Company c=new Company();

             Employee e[]=new Employee[3];
              for(int i=0;i<e.length; i++)
                    { Scanner xyz  =new Scanner(System.in);
                              e[i]=new Employee();//
                              System.out.println("Enter name id and salary of employee");
                              String name=xyz.nextLine();
                              int id=xyz.nextInt();
                              int sal=xyz.nextInt();
                              e[i].setId(id);
                              e[i].setName(name);
                              e[i].setSal(sal);
                    }
            c.addEmployee(e);

          }
}
```

Example with integer array with variable arguments

```
class Test
{
   void show(int ...x)
   {
       for(int i=0;i<x.length; i++)
              { System.out.println(x[i]);
              }
   }
}
public class TestVarApp
{
  public static void  main(String x[])
  {
      Test t=new Test();
            int m[]=new int[]{10,20,30,40,50};
            t.show(m);
  }
}
```

Pass array as infinite parameter



static keyword

---

static is a non access specifier in java we can use with variable, function and nested class

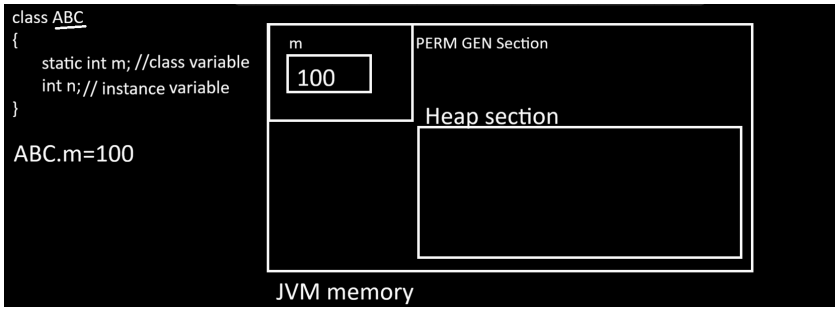
There are 8 non access specifier in java

1. static 2. final 3. Abstract 4. Synchronized 5. Transient 6. Volatile 7. Strictfp 8. Assert

static variable: static variable known as class level variable

Important points related with a static variable

1. static variable is not part of objects means when we create object of class then static variable not present in object
2. static variable allocate memory before creating object of class so we can use static variable by object also and without object also means we can use static variable by classname



Note: if we think about above screenshot it represent two above mention points
Here we we use the static variable by using a classname and we show static present before creating object of class and static variable get stored in permanent generation section of memory
In class ABC we have variable int n and this variable allocate memory after creating object of class means this variable stored within object and object known as instance means we can say n is part of object means n is part of instance means object has n or instance has n so n is variable and we cannot use it using class name must be use it using object

```java
class ABC
{
   static int m; //static variable or class level variable
   int n;//instance variable
}
public class ABCAPP
{
   public static void main(String x[])
   {
      ABC.m=100;
      System.out.println("M is  "+ABC.m);
     ABC a1 = new ABC();
     a1.n=200;
     System.out.println("N is  "+a1.n);
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac ABCAPP.java

C:\Program Files\Java\jdk-24\bin>java ABCAPP
M is  100
N is  200
```

Note: static variable can use by object also shown in following diagram

```java
class ABC
{
   static int m; //static variable or class level variable
   int n;//instance variable
}
public class ABCAPP
{
   public static void main(String x[])
   {
      ABC.m=100;
      System.out.println("M is  "+ABC.m);
     ABC a1 = new ABC();
     a1.n=200;
     System.out.println("N is  "+a1.n);
     System.out.println("M by using object "+a1.m);
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac ABCAPP.java

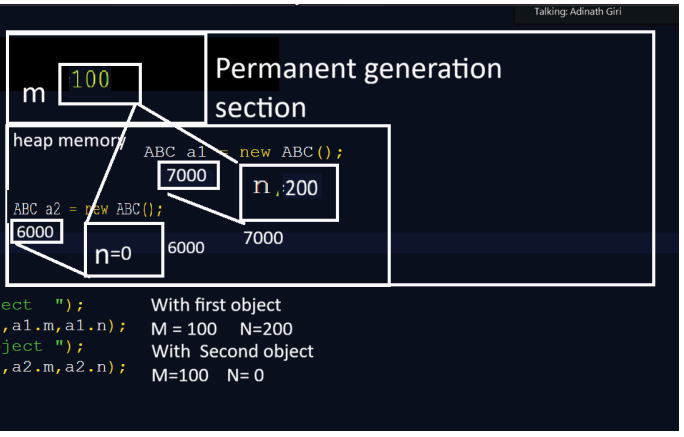
C:\Program Files\Java\jdk-24\bin>java ABCAPP
M is  100
N is  200
M by using object 100

C:\Program Files\Java\jdk-24\bin>
```

3. Static variable share his common copy between more than one object of same class

```java
class ABC
{
   static int m; //static varia
   int n;//instance variable
}
public class ABCAPP
{
   public static void main(String x[])
   {
      ABC a1 = new ABC();
      a1.m=100;
      a1.n=200;//7000.n=200

      ABC a2 = new ABC();
      System.out.println("With first object  ");
      System.out.printf("\nM=%d\tN=%d\n",a1.m,a1.n);
      System.out.println("With second object ");
      System.out.printf("\nM=%d\tN=%d\n",a2.m,a2.n);
   }
}
```



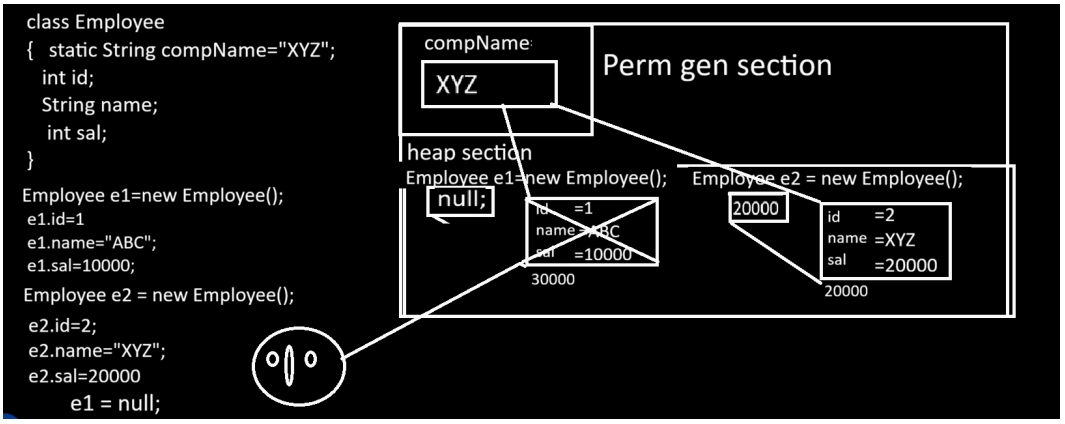
If we think about above diagram we have static int m which stored in permanent generation section of memory and we have one instance int n and if we think about main method
We have statement
ABC a1 = new ABC() means here we create object of ABC class and we imagine 7000 is memory address of object and we store this 7000 in reference variable a1
a1.m=100 means we store 100 in static variable which is not part of object but shared by object and again we have statement a1.n=200 means we store 200 in object whose address is 7000 i.e point by a1 reference so n=200 in 7000 memory block
We have one more statement ABC a2 = new ABC(); we have new object in memory we image address of object is 6000 and store in a reference

After we have statement a1.m it will print 100 and a2.n it will 200 as well as we have one more statement a2.m it will print 200 because a1 and a2 use m commonly because it is a static variable but a2.n give 0 means a2.n access own variable and we initialize value to the n present within a2 so the default of value n in a2 is 0 so we get 0 as output

4. Static variable life present whenever application present in memory but instance variable life present whenever object present in memory

   Q. Whenever objects are present in memory?

   Whenever object used by particular reference then object present in memory but when object not use by reference then JVM delete
   Memory of object automatically called as garbage collection
   When JVM delete object in directly instance variable also get deleted because instance variable is part of object but static variable is not part of object so static variable present in memory still JVM delete object

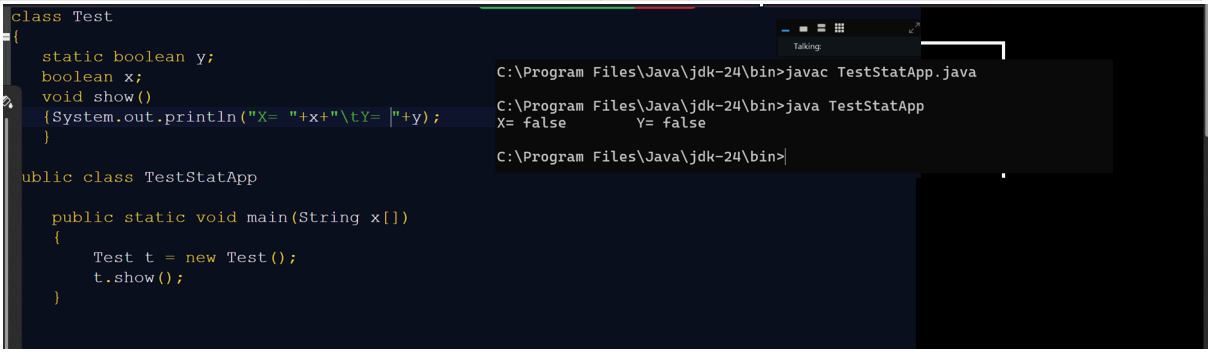


If we think about above diagram we have one variable name as compName=XYZ and we have three instance variable id,name and sal and we create two objects e1 and e2 here we consider e1 contain address 30000 and e2 contain 20000 and e1 object contain data id=1,name=ABC and sal=10000 and e2 contain id=2 name=xyz and sal =20000
And both objects access compName because it is static and when we mark e1=null means reference e1 not point to object whose address is 30000 so JVM object with address 30000 used by some another reference or not and if JVM find object not use any another reference then JVM delete object from memory this process known as garbage collection and when we delete object from memory instance get delete automatically because they are part of object But static is not part of object so they still memory even object deleted as well as they accessible by multiple object as common data so if one object get deleted static variable need for other objects so JVM not delete and persist in memory still program in execution

5.static variable stored in permanent generation section of memory i.e Meta section after JDK 1.8
6. Static variable and instance variable has default values according to their data types

Datatype	Default value
int	0

Float	0.0
Double	0.0
Long	0
Short	0
String	null
boolean	False
char	blank

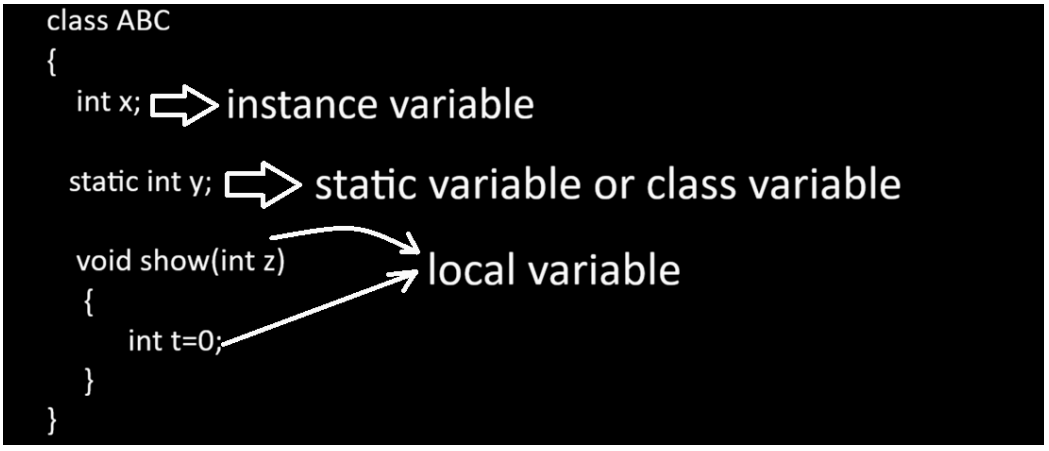


Local Variables

Local variables is declared within class function and they are not directly part of class they are part of function and function is part of class

Following diagram shows the static variable ,instance variable and local variable

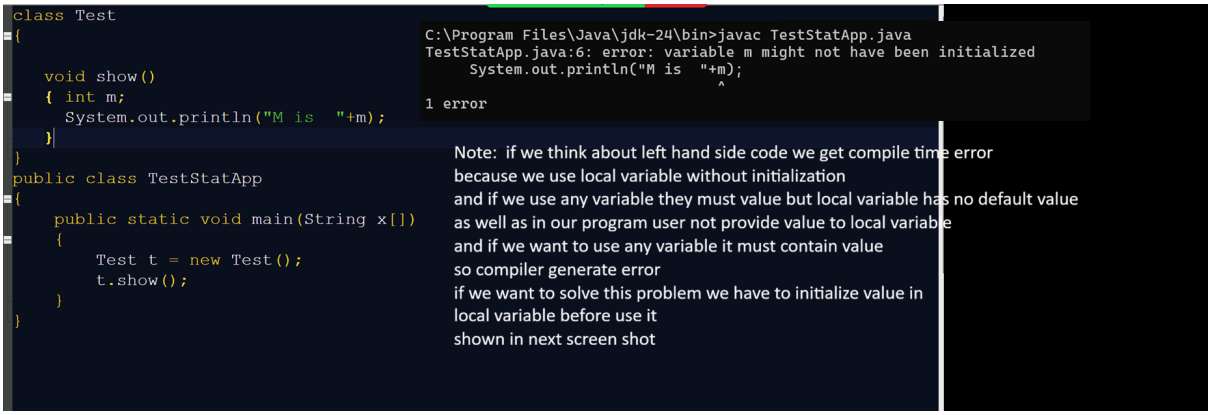
---



If we want to work with local variable we have some important points

1. **Local variable cannot access outside of his block**
   Because local variable declared within a function has opening and closing braces i.e they boundary to access we cannot access local variable outside of his closing brace so we have to copy the value of local variable in some instance variable and use in other functions of class

2. **Local variable cannot have default values even garbage**
   This statement indicate if we want to use any local variable in code they must have initialize value by developer otherwise we will get compile time error shown in following screenshot



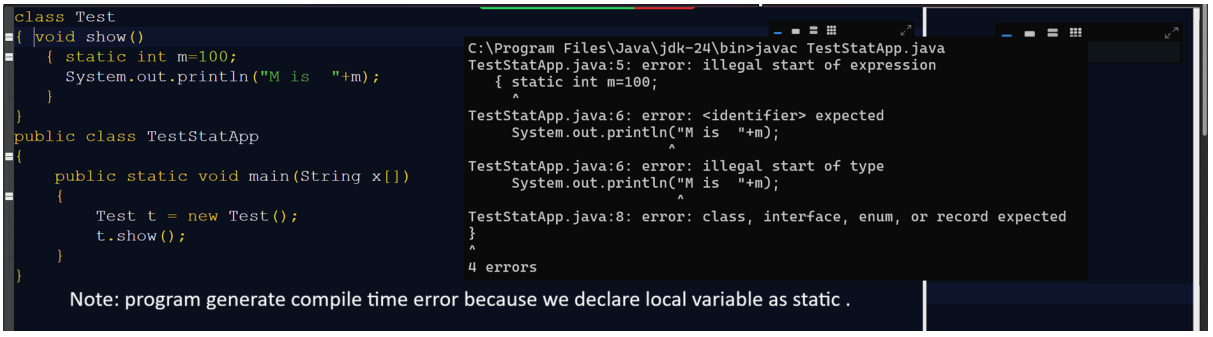


3. **Local variable cannot mark as static : if we try to declare the local variable as static then we get compile time error**

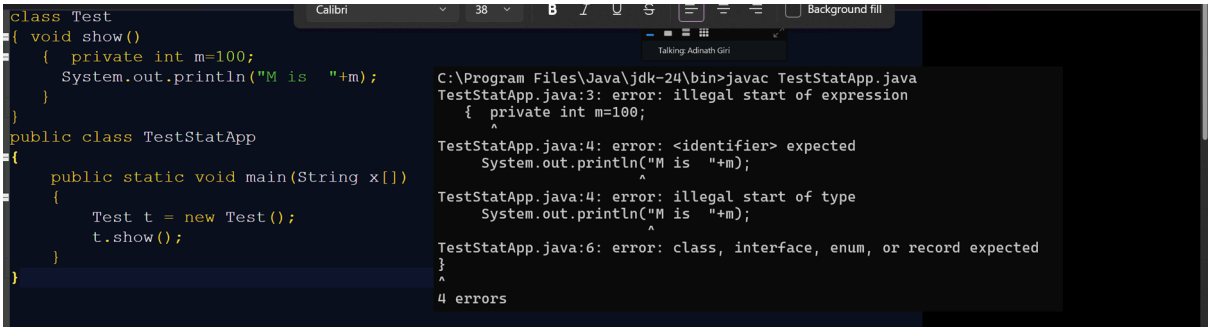
   Q. Why can we not declare local variables as static?

---

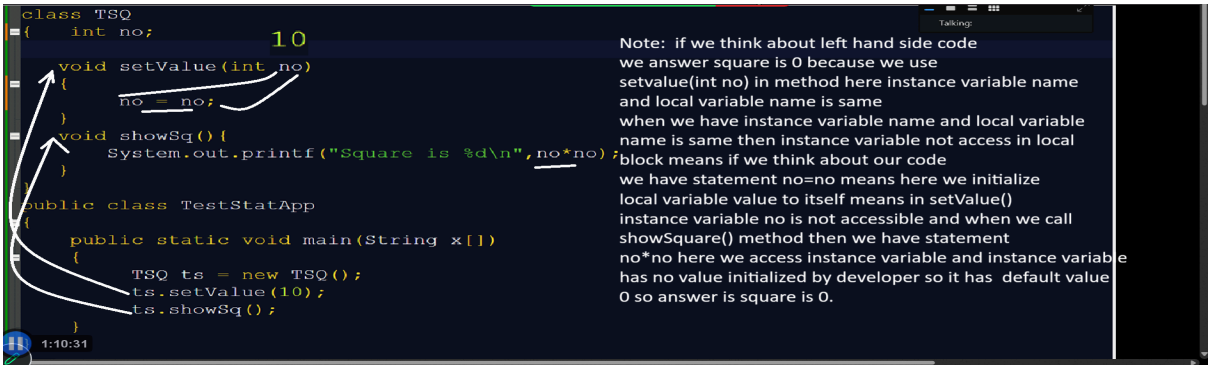
   a. Local variable life present whenever block in execution or function in execution and static variable life present whenever program running in memory
   b. Local variable allocate memory when we call function and static variable allocate memory at the time of class loading means without any object or function
   c. Local variable present in stack memory and static variable present in perm gen section or meta section of memory



4. **Cannot apply any access specifier on local variable**
   Means we cannot use private ,public or protected access specifier
   Access specifier is used for apply security restriction or decide the security level on class and its member but if we think about local variable it is already secure not access outside of block and local variable is not part of class so we cannot use any access specifier on local variable
   If we try it then the compiler will generate a compile time error.



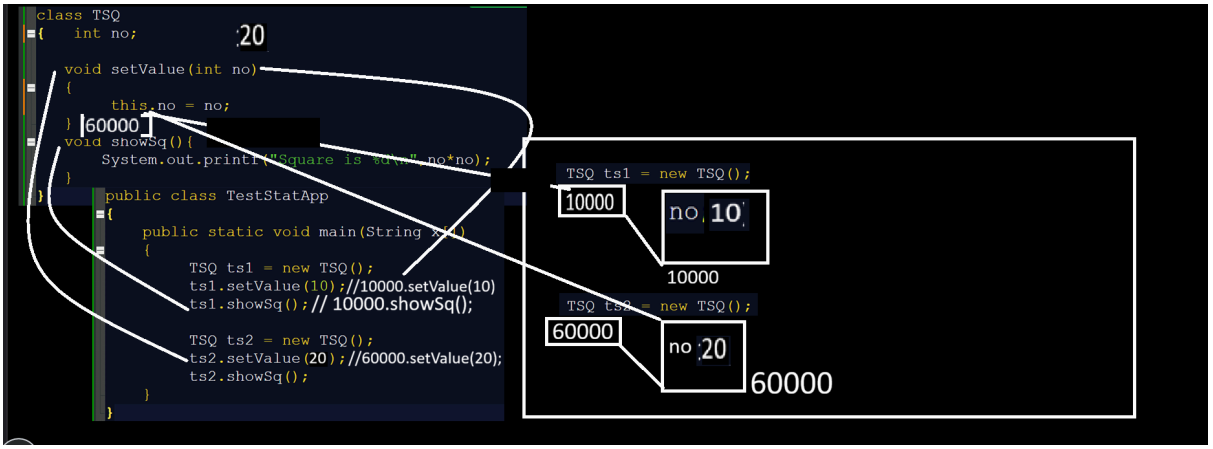
5. If local variable name and instance variable name is same then instance variable not access within local block where local variable is name



   If we want to solve this problem we have to use this reference variable in function where local variable name and instance variable name is same

   Q. What is this reference?

   This is an internal reference present in every class which is used to point to the current running object in memory.
   Function calling object work as current object



Encapsulation

Q. What is encapsulation?

Encapsulation is an object oriented programming concept that means binding data (variables) and methods (functions) together in a single class and restricting direct access to the data.

It is achieved by

1. Declaring variables as private
2. Providing public getter and setter methods to access and modify the data

Example: we have Employee data with field id,name and salary and we want to provide salary to employee and salary amount should be greater than 0.

```
class Employee
{ private int id;
  private String name;
  private int sal;
  public void setId(int id)
  { this.id=id;
  }
  public int getId()
  { return id;
  }
  public void setName(String name)
  { this.name=name;
  }
  public String getName()
  { return name;
  }
  public void setSal(int sal)
  {  if(sal>0){
       this.sal=sal;
            }else{
              System.out.println("Give some salary to employee");
            }
  }
  public int getSal()
  { return sal;
  }
}
public class EncApplication
{  public static void main(String x[])
   {              Employee e=new Employee();
                    e.setId(1);
                    e.setName("Ram");
                    e.setSal(10000);
                    System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
```

```
  }
}
```

Example: Suppose consider we are working on banking application we have class name as BankAccount and we want to give deposit and withDraw functionality in BankAccount class But we want to deposit or withdraw functionality must be allowed after login

Example with source code

```
import java.util.*;
class BankAccount
{
  private int balance;
  private String username="";
  private String password="";
  public int getBalance(){
          return balance;
  }
  public boolean isUser(String username,String password)
  {     this.username=username;
        this.password=password;
     if(username.equals("ram") && password.equals("ram"))
          { return true;
          }
          else{
            return false;
          }
  }
  public void setDeposit(int amount)
  {       if(this.username.length()>0 &&this.password.length()>0)
          {    balance=balance+amount;
          }
          else{
                  System.out.println("login to system before deposit");
          }
  }
  public void setWithdraw(int amount)
  { balance=balance-amount;
  }
}
public class EncScenarioApp
{ public static void main(String x[])
  {  Scanner xyz = new Scanner(System.in);
     BankAccount account=new BankAccount();
           do{
            System.out.println("1:Deposit");
                System.out.println("2:Withdraw");
                System.out.println("Enter your choice");
```

```
                int choice=xyz.nextInt();
                xyz.nextLine();
                switch(choice)
                {      case 1:
                        System.out.println("Enter username and password");
                        String u=xyz.nextLine();
                        String p=xyz.nextLine();
                        boolean result = account.isUser(u,p);
                        if(result)
                        { System.out.println("Enter deposit amount");
                          int depAmt=xyz.nextInt();
                          account.setDeposit(depAmt);
            System.out.println("After deposit your balance is "+account.getBalance());
                        }
                        else{
                                System.out.println("Invalid user");
                        }
                        break;
                        case 2:
                        System.out.println("Enter username and password");
                          u=xyz.nextLine();
                          p=xyz.nextLine();
                          result = account.isUser(u,p);
                        if(result)
                        { System.out.println("Enter deposit amount");
                          int widAmt=xyz.nextInt();
                          account.setWithdraw(widAmt);
                System.out.println("After deposit your balance is "+account.getBalance());
                        }
                        else{
                                System.out.println("Invalid user");
                        }
                        break;
                        case 3:
                        System.exit(0);
                        break;
                        default:
                         System.out.println("Wrong choice");
                }
             }while(true);
   }
}
```

Constructor

---

constructor is a function same name as classname but without return type called as constructor

Example with source code

Syntax:

```
class classname
{
          classname()
         { //write here your logics
         }
}
```

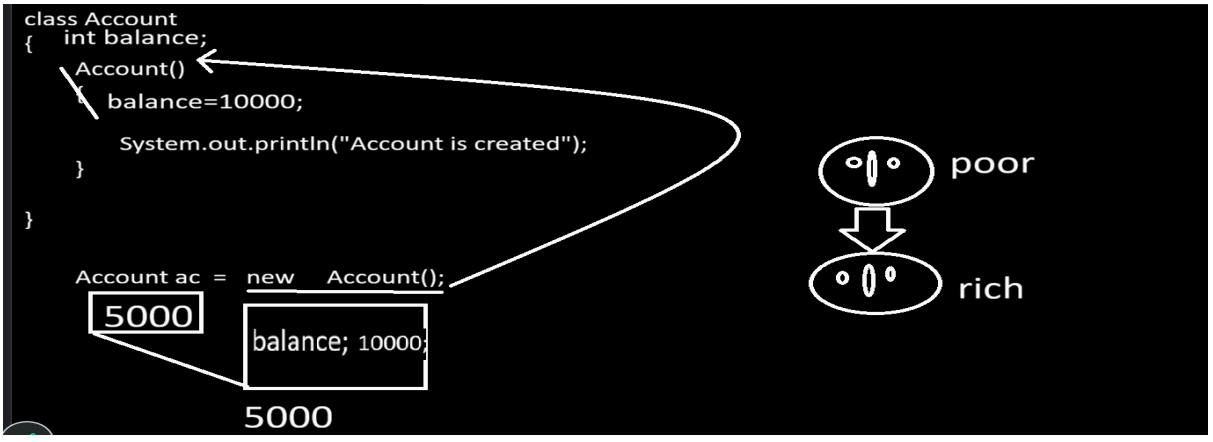
Example:

```
class Add
{
  Add()//this is your constructor
  { //
  }
}
```

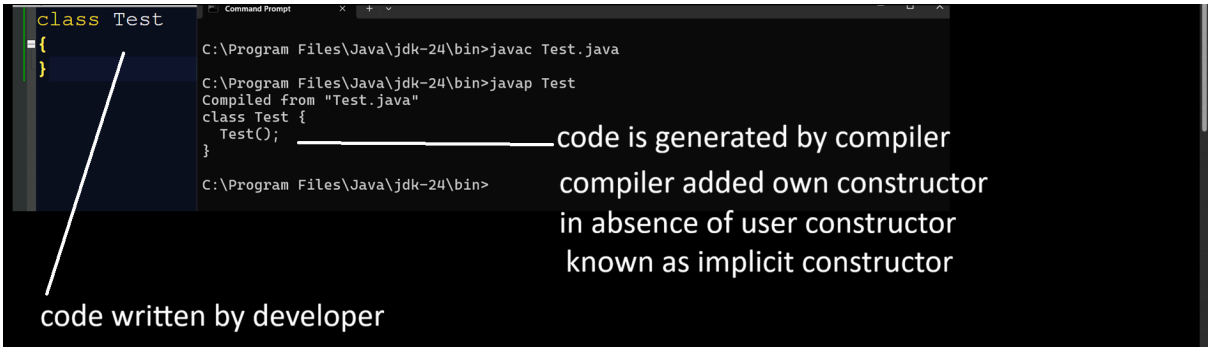
Q. What is the purpose of constructor or why use the constructor?

---

1. **To call function when class object get created :** when we create object of the class then class constructor get executed
2. **To initialize the object :** means we can initialize an instance variable when we create an object of the class.



Note: if we not define constructor within the class then every has a default constructor known as implicit constructor



Types of constructor

a. **Default constructor :** default constructor is basically user defined constructor without parameter in it called as default constructor

```
class Test
{
    Test(){
     System.out.println("I am default constructor");
    }
}
public class ImplConsApp
{
   public static void main(String x[])
   {
       Test t=new  Test();
   }
}
```

Note: default constructor normally used when we want to initialize value to instance variable at the time of object creation

Example: WAP to create class name as Circle and with two methods
void setRadius(float radius): this function is used for accept radius of circle
float getArea(): this function is used to calculate the area of a circle and return it.

```
class Circle
{  float radius,area,PI;
   Circle(){
      PI=3.14f;
   }
   void setRadius(float radius)
   { this.radius=radius;
   }
   float getArea()
   {  return radius*radius*PI;
   }
}
public  class CircleAreaApp
{  public static void main(String x[])
   {   Circle c=new Circle();
       c.setRadius(3.0f);
     float result=c.getArea();
     System.out.println("Area of circle is "+result);

   }
```

```
Circle c=new Circle();
70000
radius
area
PI 3.14
70000
```

Example: WAP to create class name as Account and we have two methods
void deposit(int amount): this method help us to deposit money in account
void withDraw(int amount): this method can withdraw money from account
int getBalance(): this method return balance of the account
Note: when user open then user must have deposit account opening balance 10000

```
class Account
{  private int balance;
   Account()
   { balance=10000;
   }
   void deposit(int amount)
   {  balance=balance+amount;
   }
   void withDraw(int amount)
   { balance=balance-amount;
   }
   int getBalance()
   {  return balance;
   }
}
public class AccountApp
{  public static void main(String x[])
   {    Account user1=new Account();
        user1.deposit(20000);
       System.out.println("User Balance is "+user1.getBalance());
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac AccountApp.java

C:\Program Files\Java\jdk-24\bin>java AccountApp
User Balance is 30000

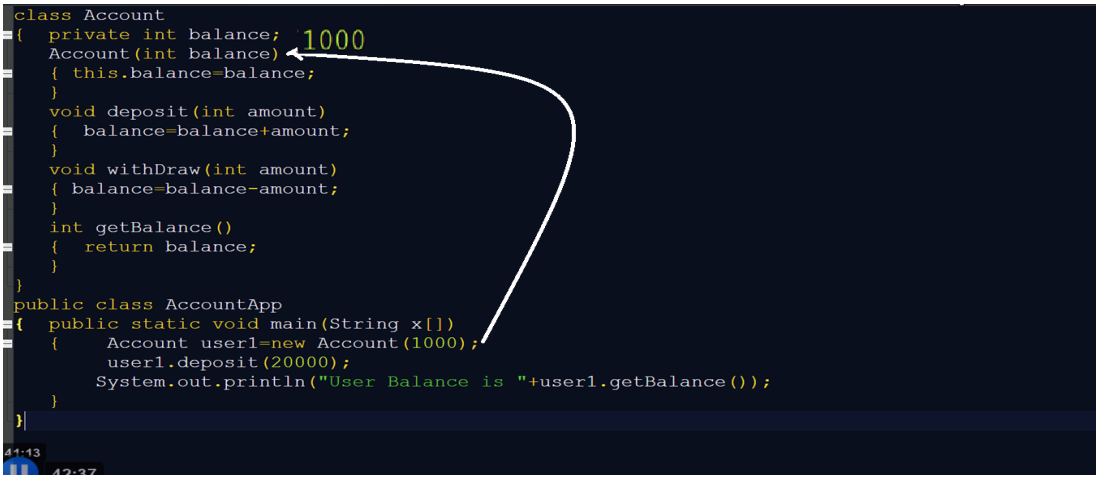
C:\Program Files\Java\jdk-24\bin>
```

b. **Parameterized constructor :** parameterized constructor means pass parameter to constructor like as function
Parameterized constructor is recommended when user want to initializer instance variable or static variable at the time of object creation means user can pass any value at runtime not fixed values

Example with source code

```
classname  classname
{
        classname(datatype value)
        {
                //write here your logics
        }
}
```

Note: when we have parameterized constructor then we can pass parameter to the constructor where we create objects of the class.



c. **Overloaded constructor :** when we define multiple constructors by using the same name called as overloaded constructor.
In the case of constructor overloading, which constructor should execute is dependent on the number parameters in it and its data type and its sequence.

Example: we have scenario we are going to implement the Account opening module in banking system

There are different types of account

1. Saving account
2. Current account
3. Zero balance account

Note:

1. Need to provide account details and user details at the time account creation
2. Every account type must have give some account opening amount i.e balance
   Example: if current account minimum account opening balance 50000 and saving account minimum account opening balance 10000 and zero balance

account no minimum balance required but in every account user can give account opening balance more than minimum amount as per his budget

Example with source code

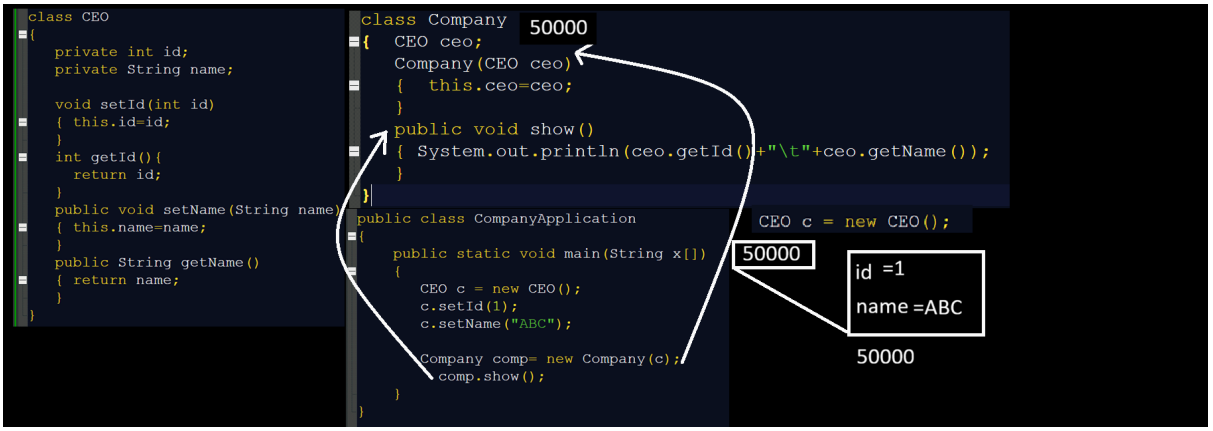
```
class Account
{  private int balance;
   private String accountType;
   Account(String accountType)
   {                this.accountType=accountType;
             if(accountType.equals("saving"))
                { balance=10000;
                }
                else if(accountType.equals("current"))
                { balance=50000;
                }
                else{
                        balance=0;
                }
   }
   Account(String accountType,int balance)
   {   this.accountType=accountType;
           if(accountType.equals("saving") && balance>=10000)
           {  this.balance=balance;
           }
           else if(accountType.equals("current"))
           { this.balance=balance;
           }
           else if(accountType.equals("zerobalance")){
                    this.balance=balance;
           }
           else{
                   System.out.println("Insufficient fund ");
           }
   }
   void deposit(int amount)
   {  balance=balance+amount;
   }
   void withDraw(int amount)
   { balance=balance-amount;
   }
   int getBalance()
   {  return balance;
   }
   String getAccountType(){
           return accountType;
   }
}
```

```
public class AccountApp
{  public static void main(String x[])
   {
      Account user1=new Account("saving",20000);
System.out.println(user1.getAccountType()+"\t"+user1.getBalance());
   }
}
```

Example array as parameter to constructor

```
import java.util.*;
class MinFinder
{    int arr[];
     int minValue=0;
    MinFinder(int arr[])
          { this.arr=arr;
          }
          int getMin()
          {   minValue=arr[0];
            for(int i=0; i<arr.length; i++)
            {
                if(arr[i]<minValue)
                              { minValue=arr[i];
                              }
            }
            return minValue;
          }
}
public class MinFinderApp
{
   public static void main(String x[])
   {
      Scanner xyz =new Scanner(System.in);
              int m[]=new int[5];
            System.out.println("Enter five values in array");
            for(int i=0; i<m.length; i++)
            {
              m[i]=xyz.nextInt();
            }
            MinFinder mf = new MinFinder(m);
      int result = mf.getMin();
            System.out.println("Minimum value  "+result);
   }
}
```

Note: we can pass POJO class as a parameter to the constructor also.



Or

```
class CEO
{
  private int id;
  private String name;

  CEO(){
          id=0;
               name="";
  }
  CEO(int id,String name)
  { this.id=id;
    this.name=name;
  }
  void setId(int id)
  { this.id=id;
  }
  int getId(){
    return id;
  }
  public void setName(String name)
  { this.name=name;
  }
  public String getName()
  { return name;
  }
}
class Company
{  CEO ceo;
  Company(CEO ceo)
  {  this.ceo=ceo;
  }
  public void show()
  { System.out.println(ceo.getId()+"\t"+ceo.getName());
  }
```

```
}
public class CompanyApplication
{
    public static void main(String x[])
          {
            CEO c = new CEO(1,"ABC");

            Company comp= new Company(c);
              comp.show();
          }
}
```

Or

```
class CEO
{
   private int id;
   private String name;

   CEO(){
              id=0;
                    name="";
   }
   CEO(int id,String name)
   { this.id=id;
     this.name=name;
   }
   void setId(int id)
   { this.id=id;
   }
   int getId(){
     return id;
   }
   public void setName(String name)
   { this.name=name;
   }
   public String getName()
   { return name;
   }
}
class Company
{  CEO ceo;
   Company(CEO ceo)
   {  this.ceo=ceo;
   }
   public void show()
   { System.out.println(ceo.getId()+"\t"+ceo.getName());
   }
}
```

```
public class CompanyApplication
{
   public static void main(String x[])
        {   new Company(new CEO(1,"ABC")).show();
        }
}
```

d. **this():** this constructor is used for constructor chaining purpose

Q.What is constructor chaining?

---

constructor chaining is technique where we can call one constructor from another constructor without creating manual object of class called as constructor chaining

There are two ways to perform constructor chaining?

---

1. **Using this() constructor**
2. **Using super() constructor**

Note: we will discuss super() constructor in inheritance chapter

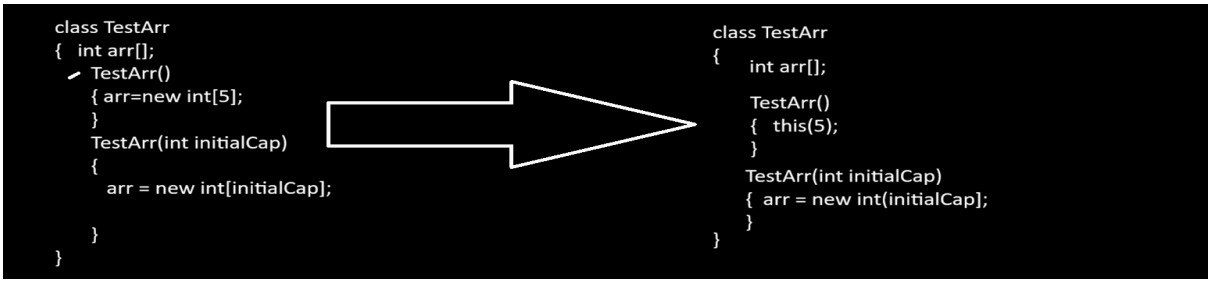
this() constructor : this() constructor is used to call the same class from another constructor normally this() or constructor chaining performs using constructor overloading technique.

Q. When constructor chaining is recommended?

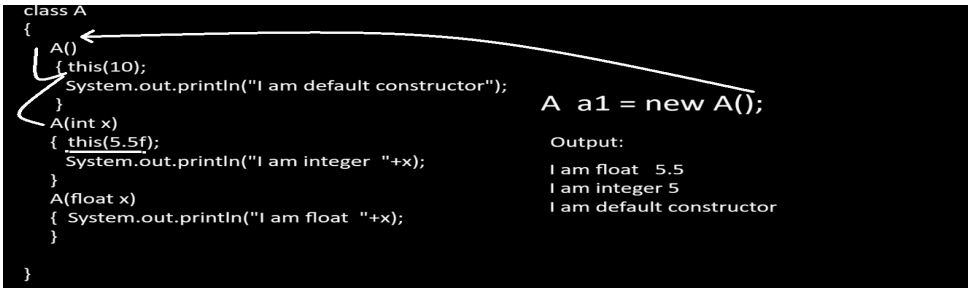
---

constructor chaining is recommended when we have same logic in different constructor but required different parameters for execute that logic then we can use constructor chaining technique

Example: suppose consider we have class and we want to create array in class with default capacity 5 and but user can change array size in future



Note: this constructor execute in last in first out format



```
class A
{
  A()
  {  this(10);
    System.out.println("I am default constructor");

  }
  A(int x)
  {  this(5.5f);
    System.out.println("I am integer constructor  "+x);
  }
  A(float x)
  { System.out.println("I am float constructor "+x);
  }
}
public class AAPP
{
   public static void main(String x[])
   {  A a1=new A();
   }
}
```

Note: this() constructor must be the first line of code in calling constructor
But after there feature -enable preview using that we can write this() constructor anywhere in code just your have to compiler your program like as

```
C:\Program Files\Java\jdk-24\bin>javac --enable-preview --release 24 AAPP.java
Note: AAPP.java uses preview features of Java SE 24.
Note: Recompile with -Xlint:preview for details.
```

Important points related with constructor

---

1. **Constructor cannot have return type**

   If we give the return type to constructor then java compiler consider it is a method Means in the case of java there is possibility function name and class name may be same

   ```
   class T
   {
     void T()
     { System.out.println("I am method");
     }
   }
   public class TAPP
   {
      public static void main(String x[])
      {
          T t1 = new T();
   ```

```
            t1.T();//method calling
    }
}
```

2. **Constructor cannot be declared as static**

If we try to declare constructor as static then it will generate compile time error to us constructor calling happen when we create object of the class and static member allocate memory before object creation



3. **Constructor not support to recursion** :



4. **Try to avoid declare as private**

When we declare the constructor as private then we cannot access or cannot create object of the class outside of class



Note: private constructor used in following cases

a. **To implement the singleton class**

Q. What is a singleton class?

Singleton class means a class can create its object only once in application
Means we cannot create more than one object of singleton class

How to create a singleton class in JAVA?

Steps to create singleton class in JAVA

1. **Declare class constructor as private**

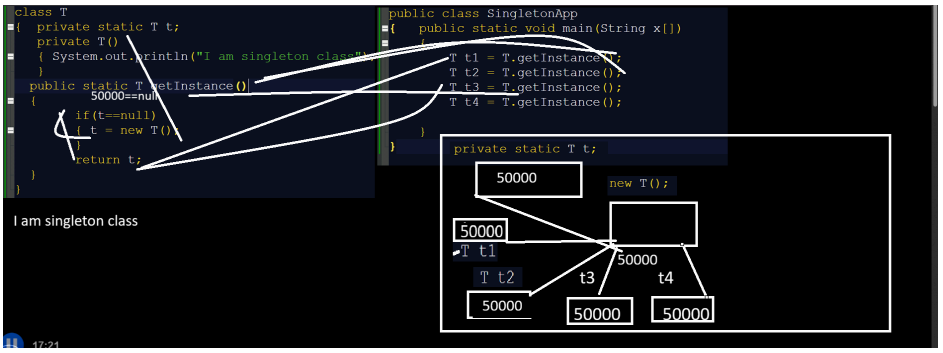
```
class T
{
   private T()
   {
   }
}
```

2. **Declare the reference of same class within class with a private and static**

```
class T
{  private static T t;
   private T()
   { System.out.println("I am singleton class");
   }
}
```

3. **Define one static method and return reference of same class in it**

```
class T
{
  private static T t;
   private T()
   { System.out.println("I am singleton class");
   }
  public static T getInstance()
  {
                    if(t==null)
                    { t = new T();
                    }
                    return t;
  }
}
```




```
class T
{  private static T t;
   private T()
   { System.out.println("I am singleton class");
   }
  public static T getInstance()
  {
                    if(t==null)
                    { t = new T();
           System.out.println("Hashcode of t is  "+System.identityHashCode(t));
                    }
                    return t;
  }
}
public class SingletonApp
{   public static void main(String x[])
    {
                    T t1 = T.getInstance();
          System.out.println("Hashcode of t1 is
"+System.identityHashCode(t1));
                    T t2 = T.getInstance();
          System.out.println("Hashcode of t2 is
"+System.identityHashCode(t2));

                    T t3 = T.getInstance();
          System.out.println("Hashcode of t3 is
"+System.identityHashCode(t3));

                    T t4 = T.getInstance();
          System.out.println("Hashcode of t4 is
"+System.identityHashCode(t4));

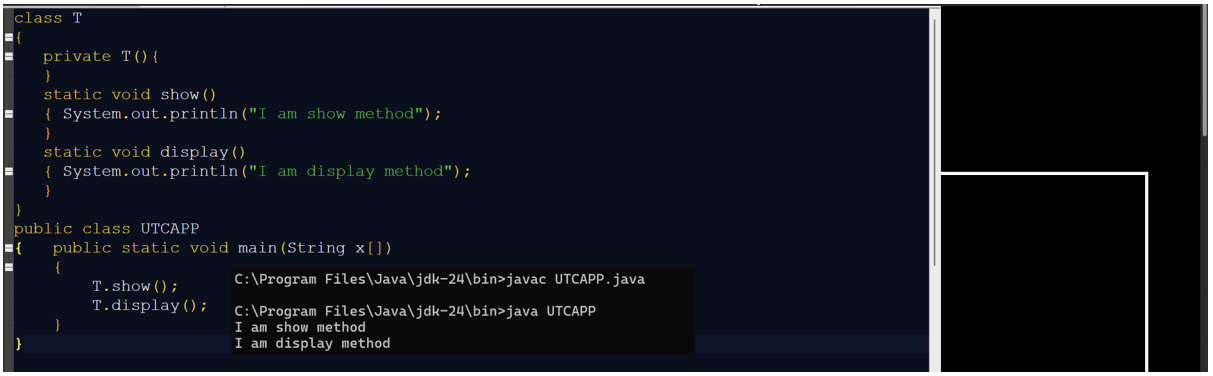
          }
}
```

b. To implement the utility class

Utility class means the class cannot create its object but contains only static methods called as utility class and it works as helper class.
E.g Collections is a utility

How to create utility class

1. **Declare class constructor as private and blank**
2. **Define static methods in it**



Q. What is the difference between function and constructor?

1. Constructor call when class object is created and function call when user call it
2. constructor cannot have return type but function have return type
3. constructor cannot support to recursion but function can support to recursion
4. constructor declare constructor as static but function can declare as static
5. constructor cannot mark as final but function can mark as final
6. Constructor cannot mark as abstract but function can mark as abstract
7. Constructor cannot override but function can override
8.

Note: we will discuss this topics in inheritance chapter

5. Cannot declare constructor as final
6. Cannot declare constructor as abstract
7. Cannot override the constructor

Etc

Nested classes

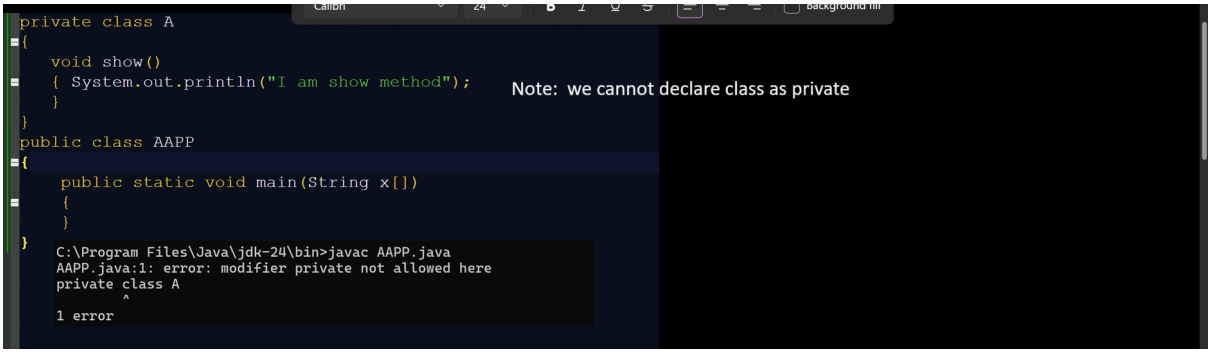
Nested classes means class within class called as Nested class

Q. Why do we need to mark nested classes or declare nested classes?

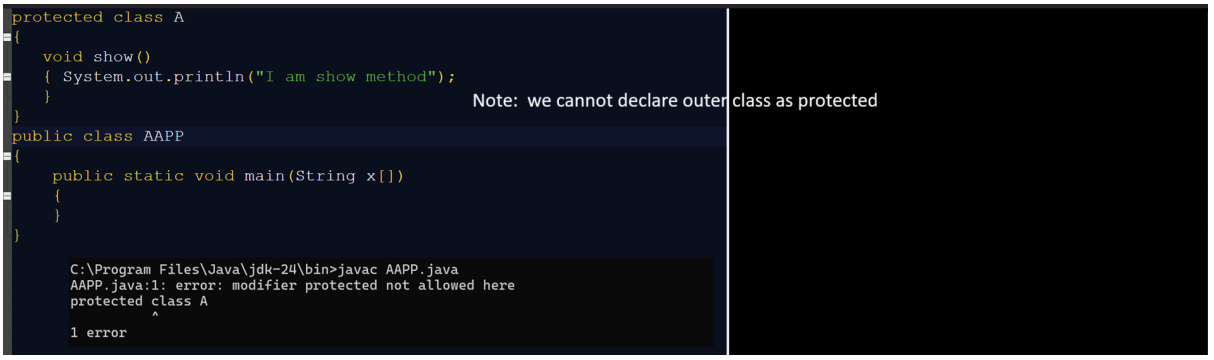
There are some reason

1. We cannot declare class as private, protected or static then we can declare the nested classes
   Meaning is we cannot declare any outer class as private, static or protected and if we declare it we have to declare as nested class

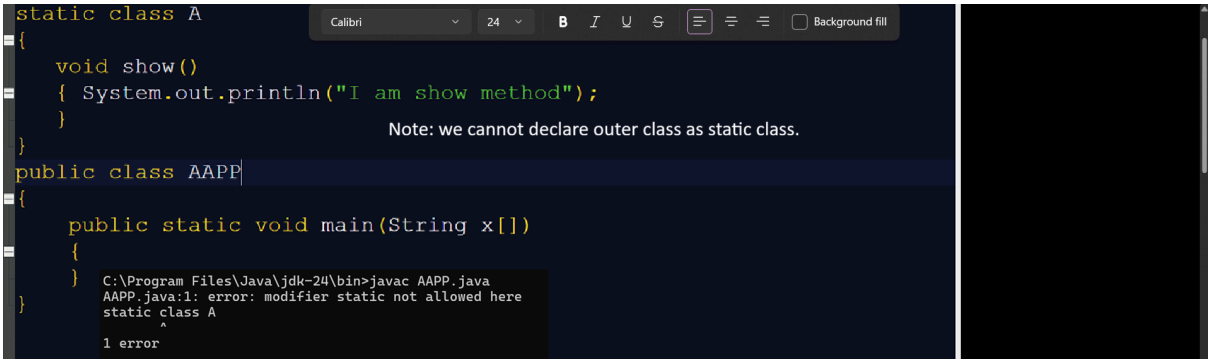
2. When we want to provide more security to classes or mark particular access within specified block then we can create nested classes



Or



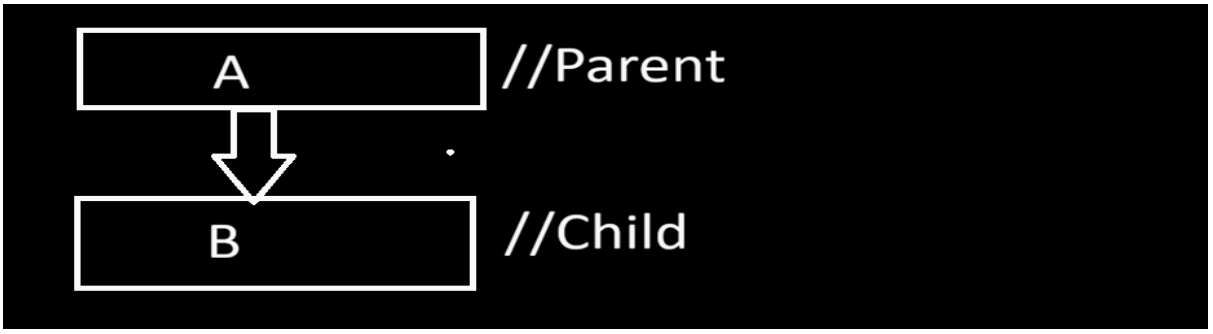
Or



Note: but we can declare nested class as private, static or protected

Inheritance

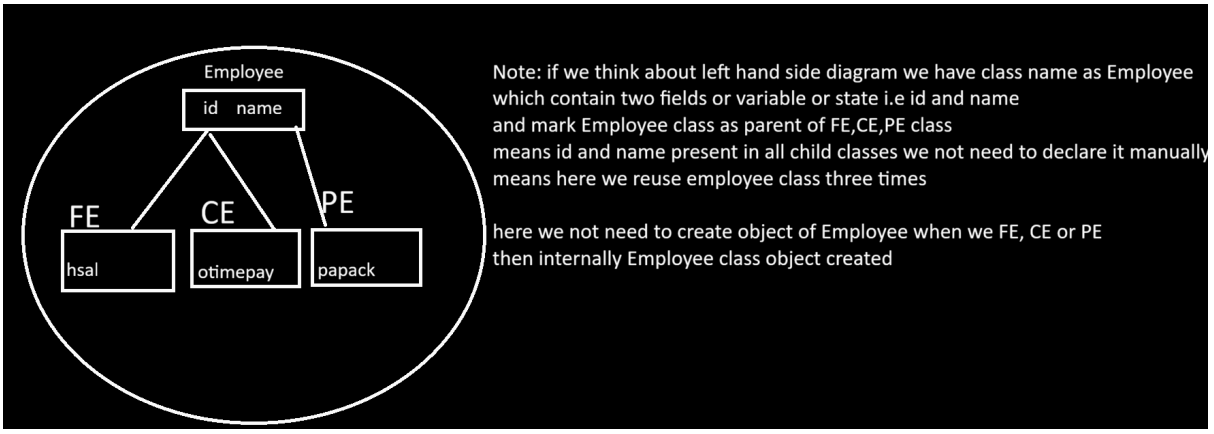
Inheritance means transferring the properties of one class to another class called inheritance.
The Property provider class known as parent and property receiver class known as child.



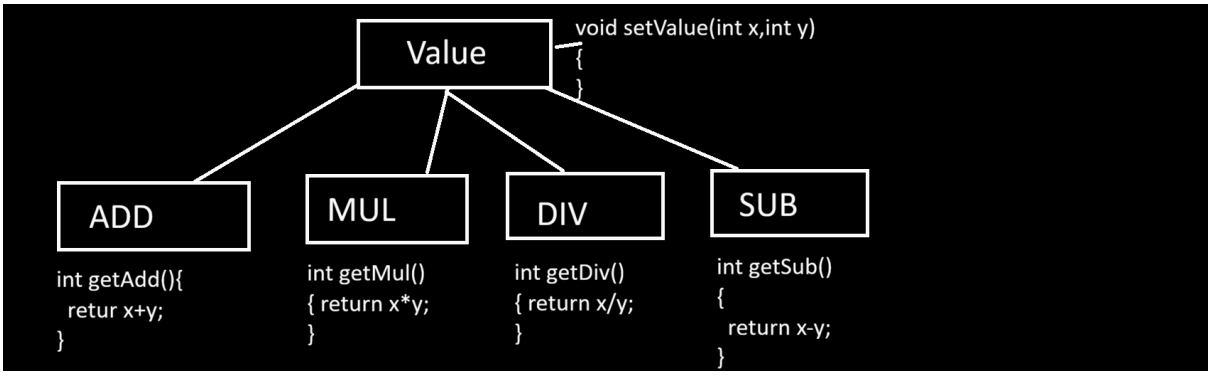
Q. Why do we use inheritance or What are the benefits of inheritance?

---

1. **Reusability of code :** we can define the logics only once and reuse it more than one time without creating manual object of class
   Means if we have some common logic required in multiple classes so better way you can write logic in particular class and mark class as parent to every class where we want to use that logic



2. **Extensibility of code :** child class can acquire the all properties of parent as well as child can define own properties also or methods also called as extensibility



If we think about above diagram we have Value class which contain setvalue() method which is required to every child i.e to Add, Mul,Div, Sub
So we create separate class name as Value and define setValue() method in it and inherit this method in child classes name as ADD,MUL,DIV, SUB means this present every class without manually define it but every class has own method like as int getAdd(), int getMul() means child use properties of parent and create new own properties called as extensibility

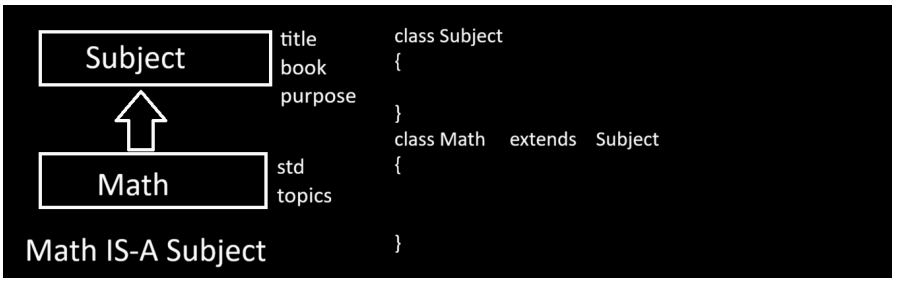
3. **Help to implement abstraction :** we have abstract class concept or interface concept which help us to implement the abstraction using a inheritance

4. **Help implement dynamic polymorphism :** we can implement dynamic polymorphism by using method overriding technique and upcasting technique
5. **Customize parent logic in child class according requirement of child:**
   Because using method overriding technique child class can modify or manipular logic of parent class according to required of child

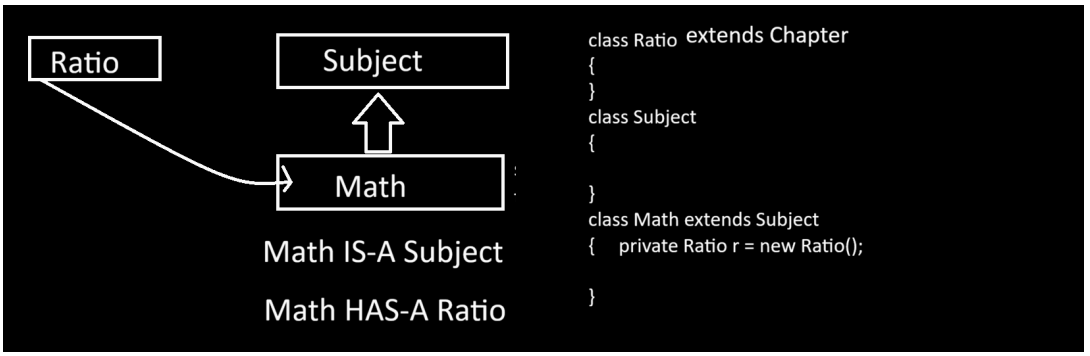
Etc

Important points (IS-A relation and HAS-A relation)

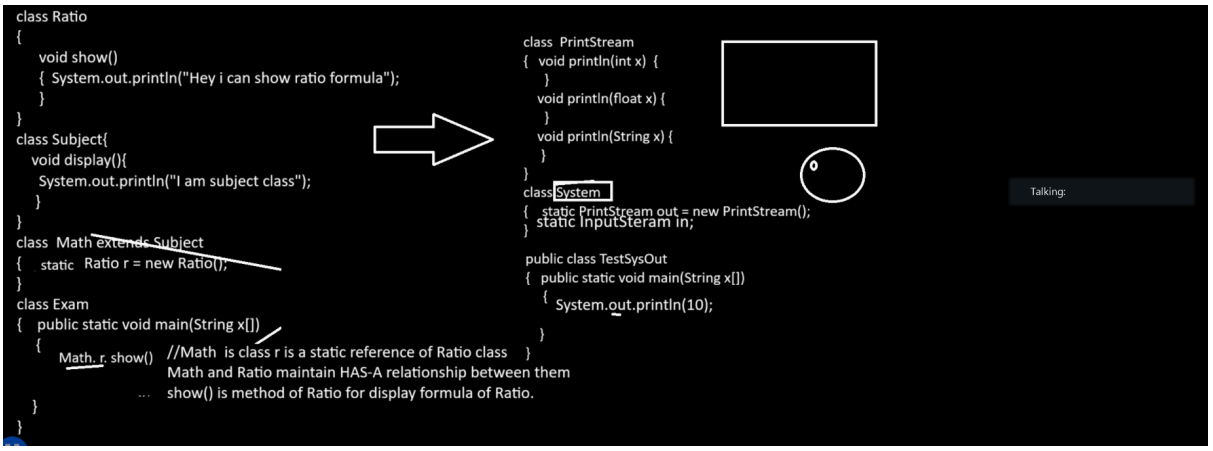
1. When we perform inheritance between two classes then classes bind in new relationship called as IS-A relationship



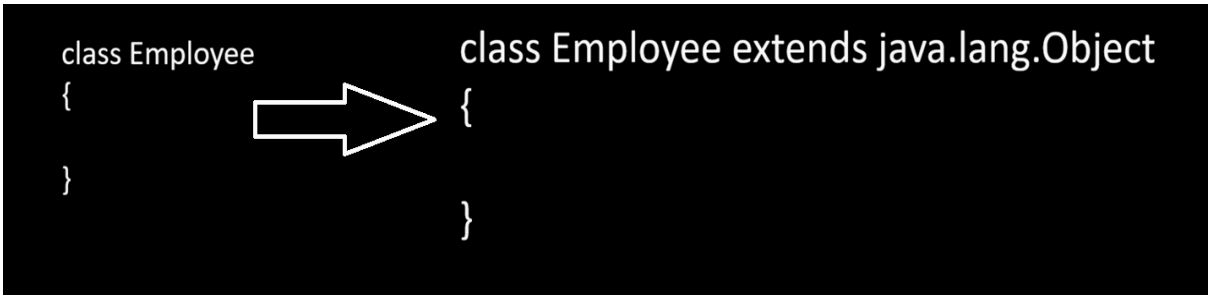
2. When we create object of one class in another class called as HAS-A relationship



Note: now we will understand meaning of System.out.printnln()
Meaning: System is class out is a reference of PrintStream class. System and PrintStream maintain a HAS-A relation between them and println() is an overloaded method for displaying output on the output screen.



Note: we cannot create any program without inheritance
Because in java there is one class from java.lang package named as Object class and it is the parent of every class in java.



Q. Why is the object class the parent of every class in JAVA?

Because Object class has a some inbuilt methods which required to every class for achieve day to day life

Methods of Object class

int hashCode()
boolean equals(Object)
String toString()
Object clone()
void wait()
void wait(int)
void notify()
void notifyAll()
void finalize()
Class getClass()

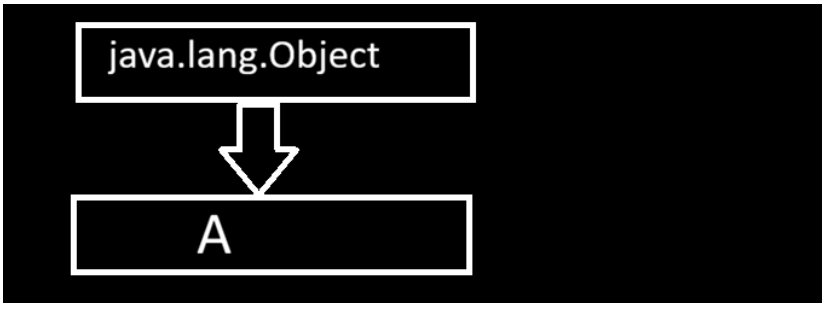
How to perform inheritance in JAVA?

If we want to inherit the one class content to another class in java we have extends keyword
Example with source code

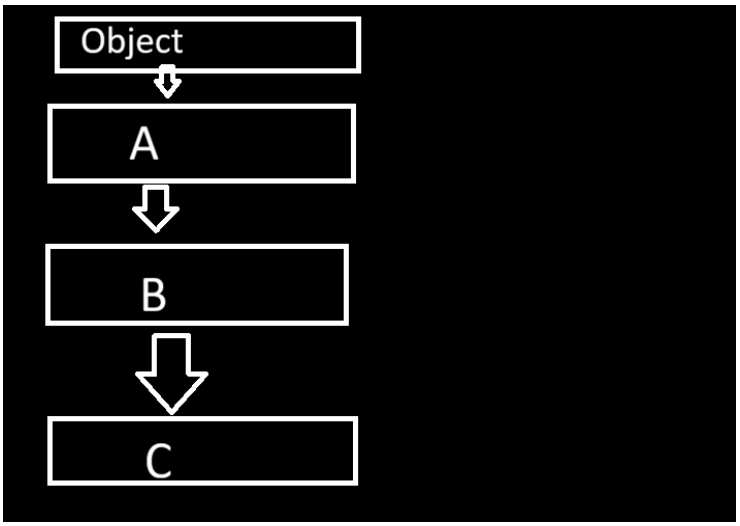
```
class childclassname extends parentclassname
{
}
```

Types of inheritance

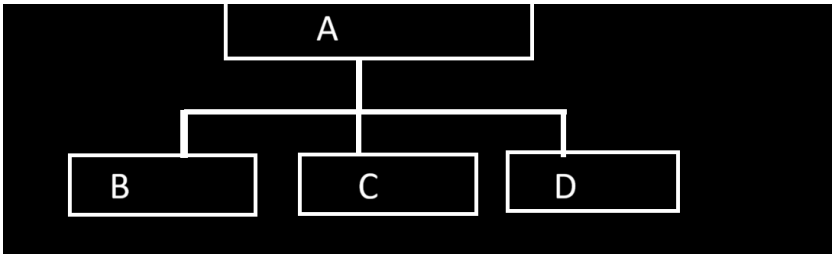
1. **Single level inheritance :** in the single level inheritance there is only one parent class and only one child class called as single level inheritance
   Note: if we think about java there inbuilt single level inheritance



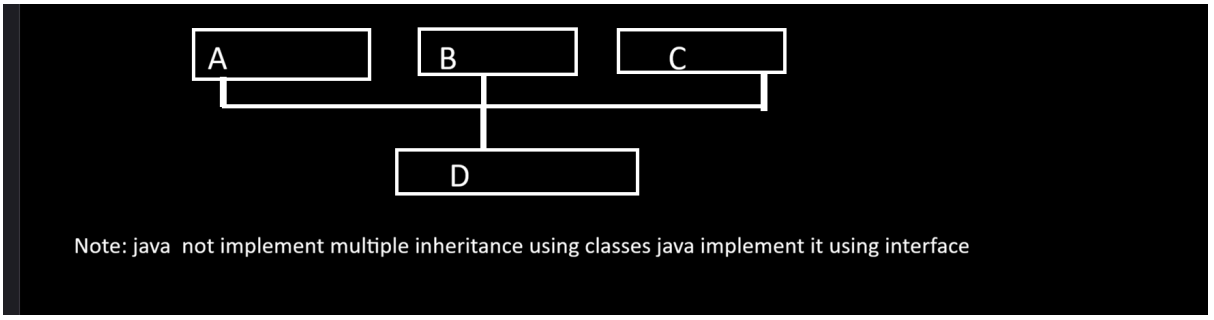
2. **Multilevel inheritance:** Multilevel inheritance means one class is parent of another and child of some other class class called as multilevel inheritance



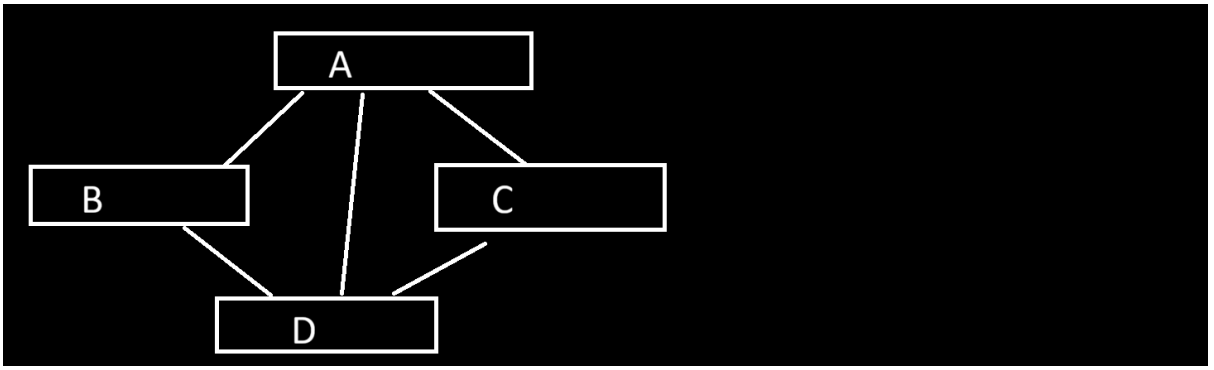
3. **Hierarchical inheritance** : Hierarchical inheritance means single parent and more than one child classes called hierarchical inheritance.



4. **Multiple inheritance :** Multiple inheritance means more than one parent and single child class called as multiple inheritance



5. **Hybrid inheritance: it is a combination of all types of inheritance**



Important point related with inheritance

In the case of inheritance we do not need to create an object of parent class.
We can create an object child class and can use the all properties of the parent class by using a child class object.

Example: Suppose consider we want to design the calculator with four operation
Case 1: ADD
Case 2: MUL
Case 3: DIV
Case 4: SUB

Example with source code

```
import java.util.*;
class Value
{  int a,b;
   void setValue(int x,int y)
   {  a=x;
      b=y;
   }
}
class Add extends Value
{
   int getAdd()
   { return a+b;
   }
}
class Mul extends Value
{   int getMul()
    { return a*b;
          }
}
class Div extends Value
{
   int getDiv()
   { return a/b;
   }

}
class Sub extends Value
{ int getSub()
  { return a-b;
  }
}
public class CalcApp
{
   public static void main(String x[])
   {
     Scanner xyz  = new Scanner(System.in);

              System.out.println("1:ADD");
                    System.out.println("2:MUL");
                    System.out.println("3:DIV");
```

```
            System.out.println("4:SUB");

            System.out.println("Enter your choice");
        int choice=xyz.nextInt();
            System.out.println("Enter two values");
            int a=xyz.nextInt();
            int b=xyz.nextInt();
            switch(choice)
            {
              case 1:
              Add ad = new Add();
                ad.setValue(a,b);
              int result=ad.getAdd();
              System.out.println("Addition is "+result);
              break;
              case 2:
              Mul m =new Mul();
               m.setValue(a,b);
                    result=m.getMul();
                    System.out.println("Multiplication is "+result);
              break;
              case 3:
              Div d= new Div();
              d.setValue(a,b);
              result=d.getDiv();
              System.out.println("Division is "+result);
              break;
              case 4:
              Sub s=new Sub();
              s.setValue(a,b);
              result=s.getSub();
              break;
              default:
                System.out.println("Wrong choice");
            }

  }
}
```

Constructor concept in inheritance

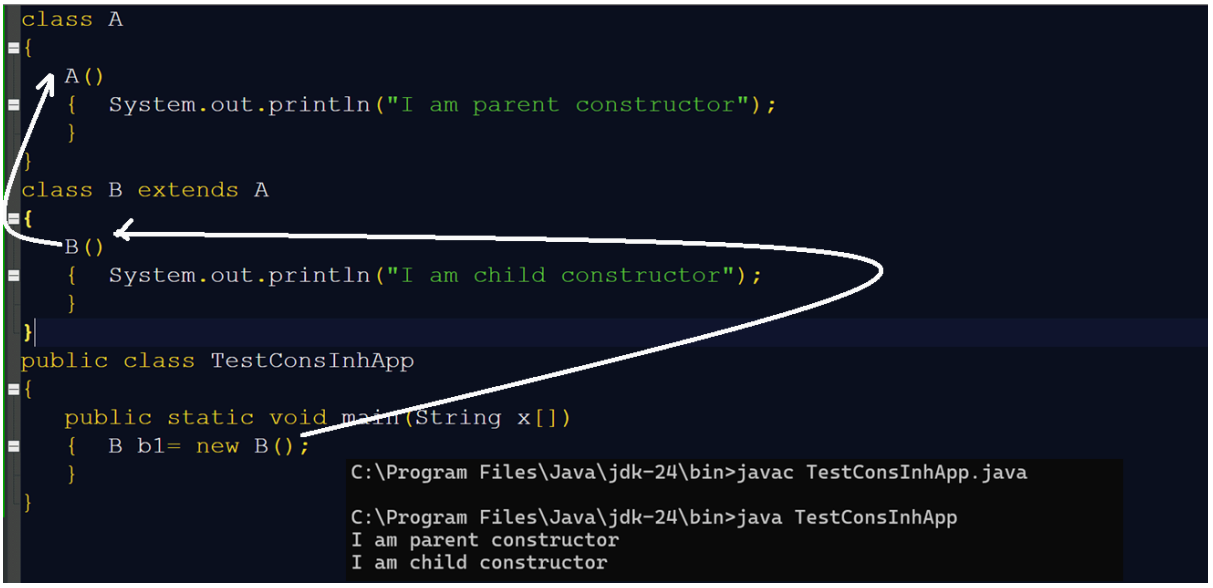
Important points related with constructor in inheritance

1. When parent has default constructor and child has constructor and if we create object of child class then parent constructor executed before child constructor automatically or internally

2. If parent has parameterized constructor and child class contains constructor and if we create object of child class then child constructor has responsibility

to pass parameter to parent constructor and for that we have super() constructor concept

Example:



```
class A
{
   A()
   {  System.out.println("I am parent constructor");
   }
}
class B extends A
{
   B()
   {  System.out.println("I am child constructor");
   }
}
public class TestConsInhApp
{
   public static void main(String x[])
   {  B b1= new B();
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac TestConsInhApp.java

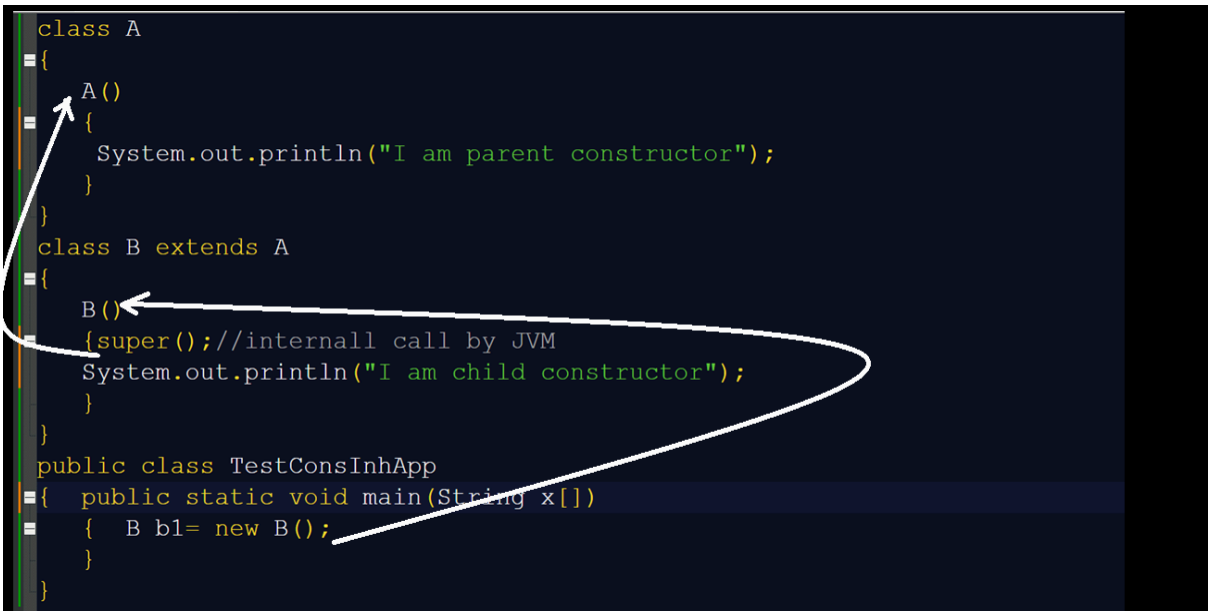
C:\Program Files\Java\jdk-24\bin>java TestConsInhApp
I am parent constructor
I am child constructor
```

Note: if we think about above code we have two classes named as class A and class B and in class A contain default constructor and we create object of child class internally java class first parent constructor and then child class

How JAVA JVM call internally parent constructor before child if parent has default constructor

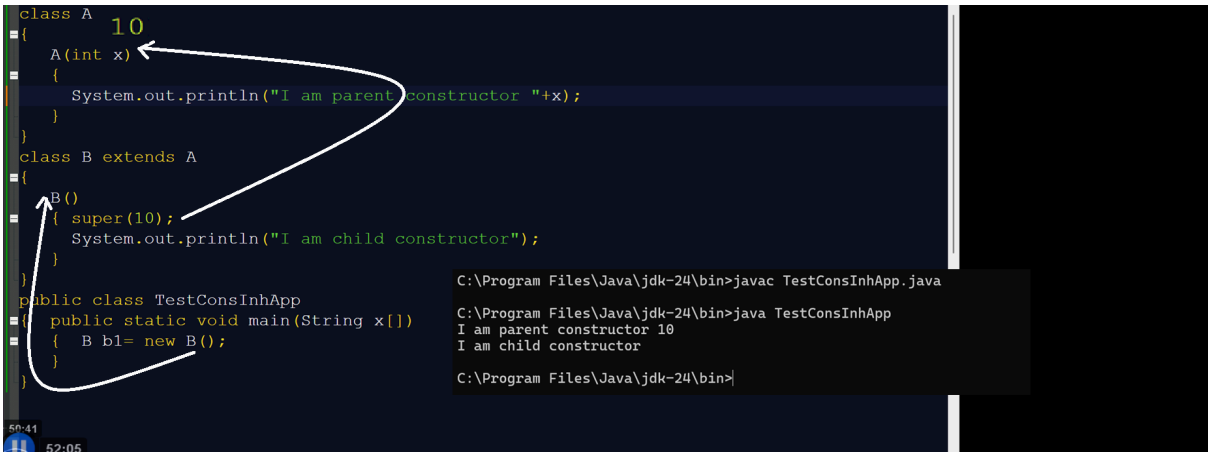
---

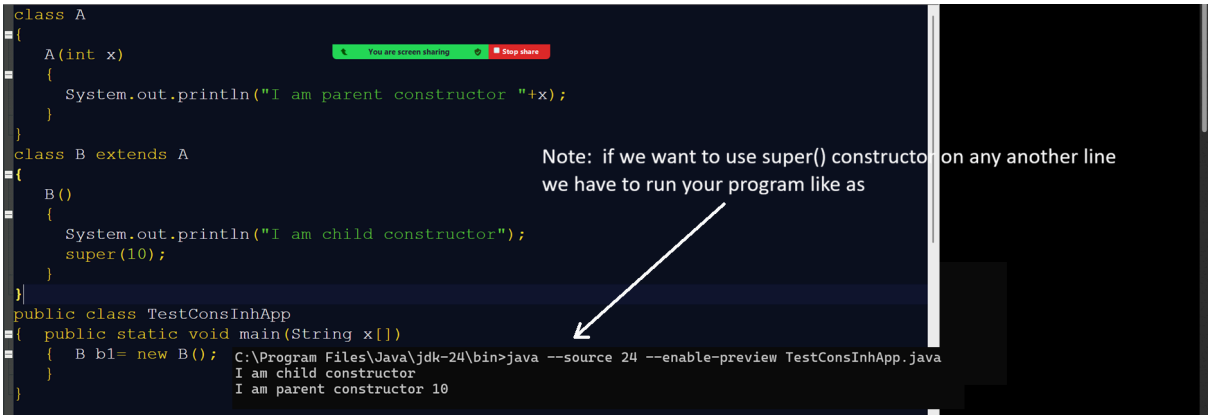
Internally java JVM call super() default constructor



```
class A
{
   A()
   {
    System.out.println("I am parent constructor");
   }
}
class B extends A
{
   B()
   {super();//internall call by JVM
   System.out.println("I am child constructor");
   }
}
public class TestConsInhApp
{  public static void main(String x[])
   {  B b1= new B();
   }
}
```

Note: when we have parameterized constructor in parent class then we must have to pass that parameter from child class constructor using inbuilt constructor called as super constructor





Method overriding concept

Method overriding means when we define a method in parent class and redefine the same method again in child class called method overriding.
In Method overriding signature of parent method and signature of child method must be same

Example

```
class P
{
  void show()
  { System.out.println("I am parent method");
  }
}
class C extends P
{
 void show()
 { System.out.println("I am child method");
 }
}
```

Note: In the case of method overriding when we create object of child class and try to call overridden method then child logic get executed by defaults

```java
class P
{
    void show()
    { System.out.println("I am parent method");
    }
}
class C extends P
{
   void show()
   { System.out.println("I am child method");
   }
}
public class OAPP
{   public static void main(String x[])
    {
          C c1 = new C();
          c1.show();
    }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac OAPP.java

C:\Program Files\Java\jdk-24\bin>java OAPP
I am child method

C:\Program Files\Java\jdk-24\bin>
```

Note: if we think about above code we get output I am child method
Means here we lost the parent parent logics using a method overriding

Q. Why do we use method overriding or What are the benefits of method overriding?

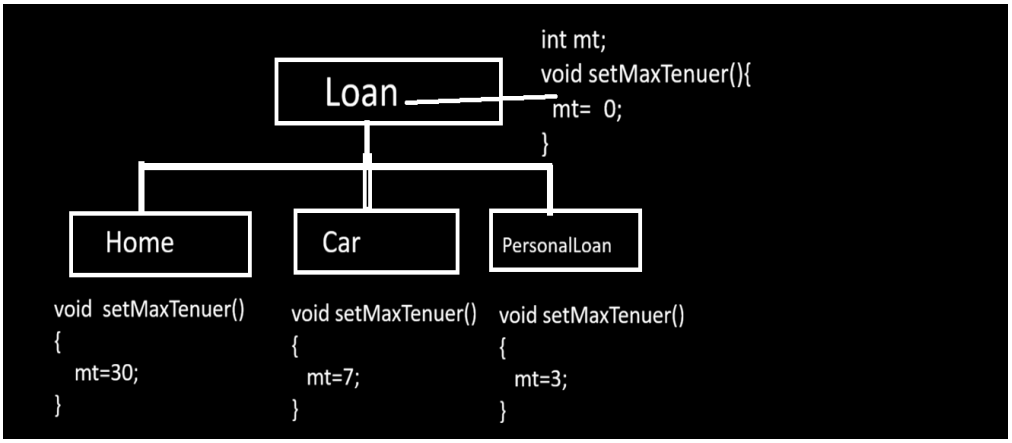
---

1. Child class can customize parent method logic according his requirements
2. Using a method overriding we can achieve dynamic polymorphism
3. Using a method overriding we can use abstraction or achieve abstraction

Child class can customize parent method logic according his requirements

---

Example: Suppose consider we want to develop the Banking System where we have one parent class name as Loan and we have different kinds of loan like as Home, Car, PersonalLoan etc
Loan Tenure is vary according to loan category



Note: if we think about the above diagram we have the parent class name as Loan with setMaxTenuer() method and there is no max tenure we set 0 but the loan re fundation period is dependent on the category of loan.
We have different types of loan classes like as Home, Car, PersonalLoan etc and we override the setMaxTenuer() method in every child class and write a different logic or set different tenure as per the loan category
Means here every child class modify the parent logic in every child as per his requirement

Example with source code

```
class Loan
{ int mt;
  void setMaxTenure()
  { mt=0;
  }
}
class Home extends Loan
{  void setMaxTenure()
   { mt=30;
   }
   int getMaxHomeLoanTenure(){
     return mt;
   }
}
class Car extends Loan
{ void setMaxTenure()
   { mt=7;
   }
   int getMaxCarLoanTenure()
   {  return mt;
   }

}
class Personal extends Loan
{ void setMaxTenure()
   { mt=3;
   }
   int getMaxPersonalTenure()
   {  return mt;
   }
}
public class LoanApplication
{
   public static void main(String x[])
   {
      Home h = new Home();
            h.setMaxTenure();
            int result =h.getMaxHomeLoanTenure();
            System.out.println("Home Loan Max Tenure is  "+result);

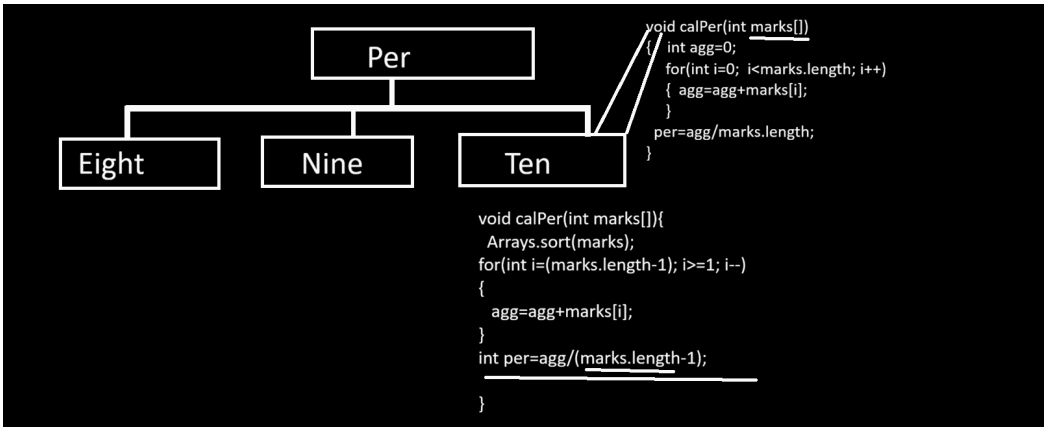
            Car c = new Car();
            c.setMaxTenure();
            result=c.getMaxCarLoanTenure();
            System.out.println("Max Car Loan Tenure is "+result);

            Personal p = new Personal();
```

```
        p.setMaxTenure();
        result=p.getMaxPersonalTenure();
        System.out.println("Max Personal Loan Tenure is "+result);

  }
}
```

Example: Suppose consider we want to develop the application for MarkSheet Generation Generate MarkSheet for three classes 8,9 and 10th there is requirement like as
Class Eight & Nine marksheet generate using best of six pattern and class Tenth percentage should be calculated by using best of 5
But there is one more rule if student admission before 2010 then student is not applicable for best of 5



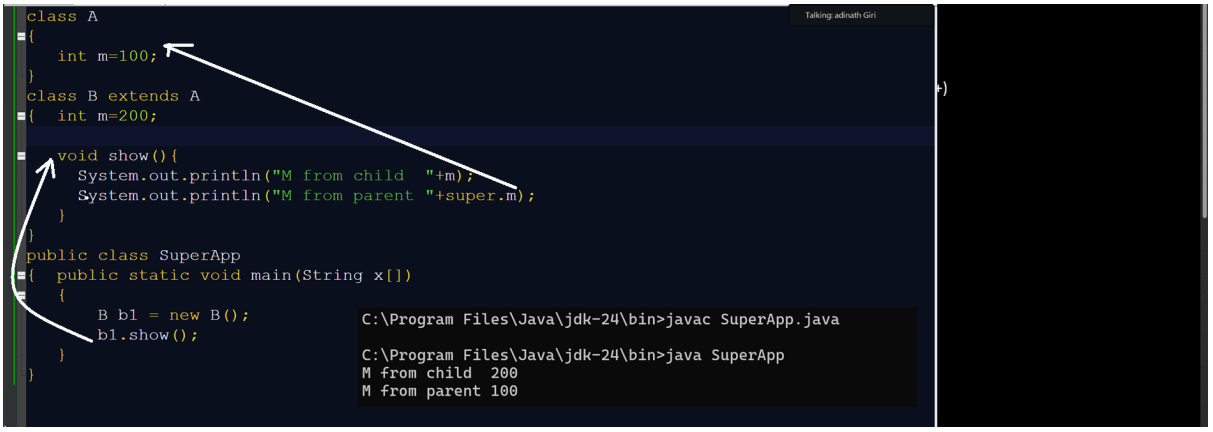
Example with source code

```
import java.util.*;
class Per
{  int agg=0;
   int per;
   void calPer(int marks[])
   {
      for(int i=0; i<marks.length; i++)
            { agg=agg+marks[i];
            }
            per=agg/marks.length;
   }
}
class Eight extends Per
{
    int getEightPer(){
            return per;
          }
}
class Nine extends Per
```

```
{   int getNinePer(){
              return per;
            }
}
class Ten extends Per{
            int year;
   void verifyAdmissionYear(int year)
   {  this.year=year;
   }
   void calPer(int marks[])
   {
     if(year<2010)
            { super.calPer(marks);
     }
            else{
      Arrays.sort(marks);
       for(int i=1; i<marks.length; i++)
              { agg=agg+marks[i];
              }
              per=agg/(marks.length-1);
            }
   }
   int getTenPer(){
      return per;
   }
}
public class MarkSheetApp
{   public static void main(String x[])
    {
                    int arr[]=new int[]{50,60,70,60,80,90};
                    Eight eight=new Eight();
                    eight.calPer(arr);
                    int result=eight.getEightPer();
                    System.out.println("Eight class Percentage is "+result);
                    Nine nine=new Nine();
                    nine.calPer(arr);
                     result=nine.getNinePer();
                    System.out.println("Nine class Percentage is "+result);
                              Ten ten=new Ten();
                              ten.verifyAdmissionYear(2026);
                      ten.calPer(arr);
                     result=ten.getTenPer();
                    System.out.println("Tenth class Percentage is "+result);

          }
}
```

Super reference: super is an inbuilt reference present in child class and which is used for accessing the member of parent in child class.
Normally super is recommended when we have same member name in parent and same member name in child means variable overriding or method overriding and we if we want to access parent member in child class then we can use super keyword



Using a method overriding we can achieve dynamic polymorphism:

Q. What is dynamic polymorphism?

Dynamic polymorphism means function or functionality bounded with an object at program runtime called dynamic polymorphism.
We can achieve it using a method overriding

Important points : if we want to understand the concept of dynamic polymorphism we need to know following important points

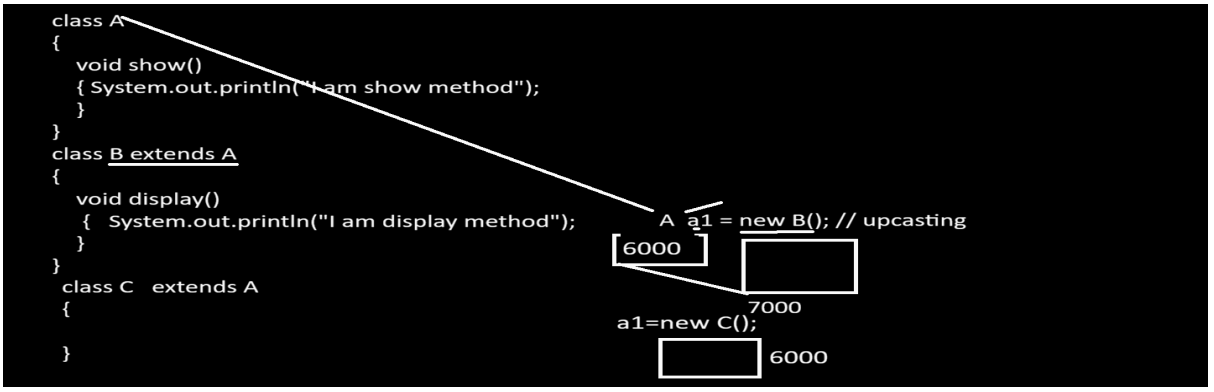
1. **Method overriding**
2. **Upcasting technique**

Q. What is the meaning of upcasting and downcasting?

These are type casting technique but normally applied on references or objects

a. **Upcasting:** upcasting means when we convert child object into parent class called as upcasting
In this case we have create reference of parent and store child class object address in parent reference

Syntax: Parentclassname ref = new childclassname();
Note: here parent class reference change child object at runtime or work with a different child objects at runtime



If we think about above we have three classes name as A, B and C here A is parent , B & C both are childs

We have statement A a1 = new B(); here we use reference of parent and store class B child object address in a1 i.e 7000 and again we have a1=new C() here we created object of child class C and store his address in a1 i.e 6000 means a1 reference leave the address of previous child class i.e B and point to new object i.e C but a1 change address of child class object at program runtime so the parent class reference can work with multiple child object at program runtime and this behaviour known as dynamic polymorphism

Means parent class reference can call different child class overridden method at program runtime

Example of dynamic polymorphism

We want to implement the dynamic polymorphism concept using method overriding Here we want to develop the calculator application which contain five classes

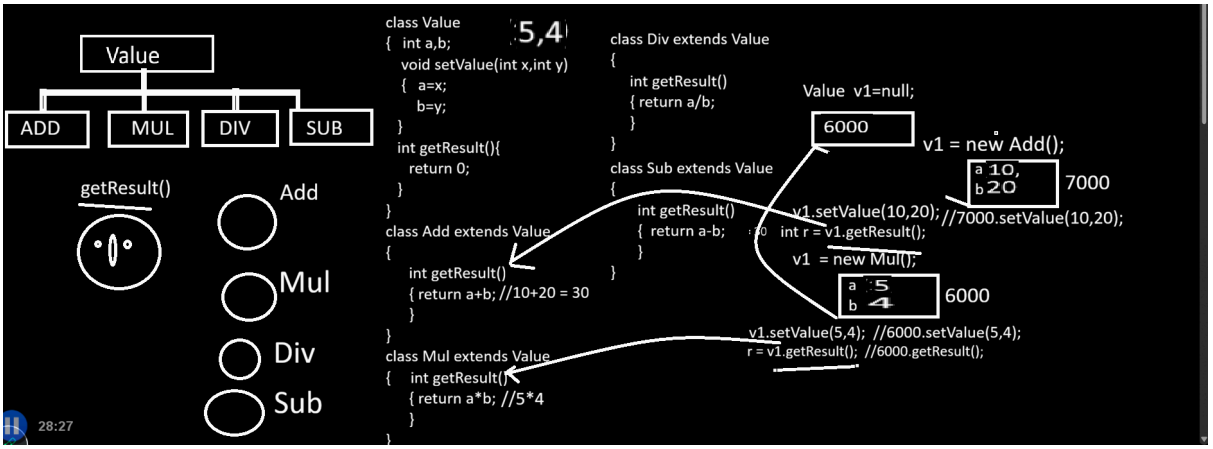
Value class with two methods void setValue(int x,int y) , int getResult() this method can return 0 in Value class and we have four child classes name as Add,Sub,Mul,Div And we override getResult() method in every child class and perform different operation like Add class return addition two values, Mul class can return multiplication of two values etc

Case 1: Calculate Addition

Case 2: Calculate Multiplication

Case 3: Calculate Substraction

Case 4: Calculate Division



```
import java.util.*;
class Value
{  int a,b;
   void setValue(int x,int y)
   { a=x;
     b=y;
   }
   int getResult()
   { return 0;
   }
}
class Add extends Value
{
    int getResult()
          { return a+b;
          }
}
class Mul extends Value
{int getResult()
          { return a*b;
          }
}
class Div extends Value
{int getResult()
          { return a/b;
          }
}
class Sub extends Value
{int getResult()
          { return a-b;
          }
}
public class DynamicPolyApp
{  public static void main(String x[])
   { Value v1=null;
     do{
            Scanner xyz = new Scanner(System.in);
            System.out.println("1:Addition");
            System.out.println("2:Multiplication");
            System.out.println("3:Division");
            System.out.println("4:Substraction");
            System.out.println("Enter your choice");
            int choice=xyz.nextInt();
            System.out.println("Enter two values");
            int a=xyz.nextInt();
            int b=xyz.nextInt();
           switch(choice)
           {
```

```
            case 1:
                    v1=new Add();
                    v1.setValue(a,b);
                    int r=v1.getResult();
                   System.out.println("Addition is "+r);
                   break;
                   case 2:
                    v1=new Mul();
                    v1.setValue(a,b);
                    r=v1.getResult();
                   break;
                   case 3:
                    v1=new Div();
                    v1.setValue(a,b);
                    r=v1.getResult();
                    System.out.println("Division is "+r);
                   break;
                   case 4:
                    v1=new Sub();
                    v1.setValue(a,b);
                    r=v1.getResult();
                   System.out.println("Substraction is "+r);
                   break;

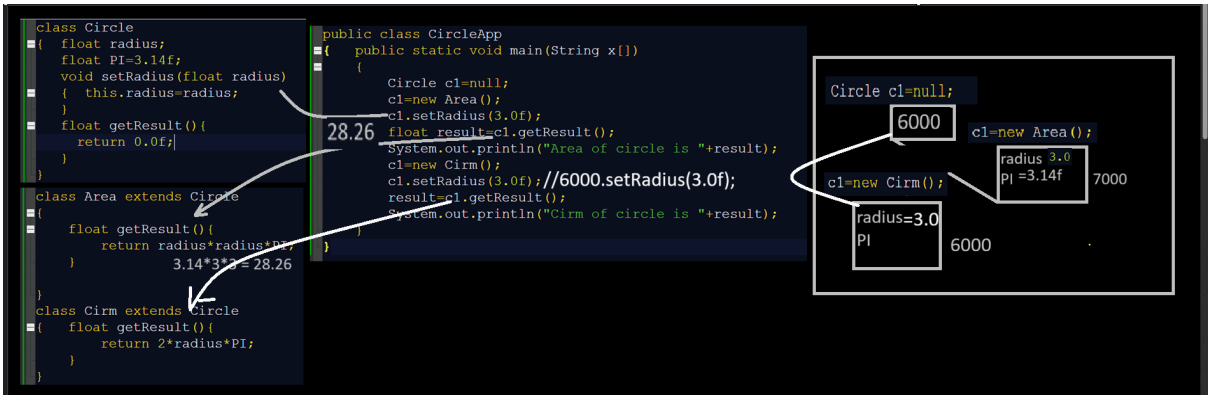
                   default:
                     System.out.println("Wrong choice");
               }
           }while(choice!=5);
     }
}
```

Example2: Suppose consider we have Circle class with two methods

void setRadius(float radius): this method is used for set the radius of circle

float getResult(): this method has two behaviour one is area, second is cirm

So we want to achieve two behaviour of same method via Circle so we need to create two classes name as Area and this class as child of Circle and Cirm also mark this class as child of Area and can override this method in both classes and write different logics



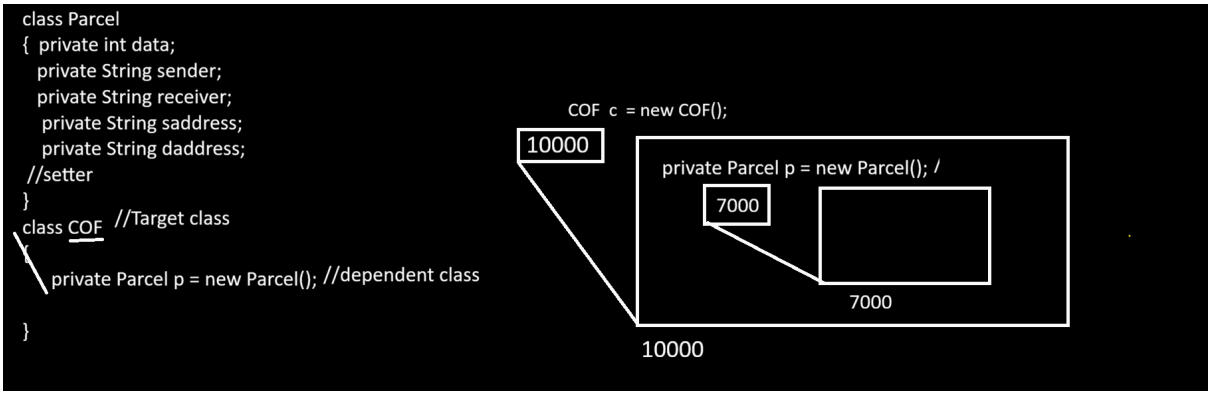
Q.What is the benefit of dynamic polymorphism or Why dynamic polymorphism is used?

The benefit of dynamic polymorphism is achieve loose coupling
Before that we need to know

Q. What is coupling?

Coupling means when one class object is dependent on another class called as coupling
Means we can say coupling works with object dependency
If we want to understand the coupling we need to clear two terms

1. **Target class :** target class means class using that we can create object of dependent class automatically
   Means when developer access target class then dependent class automatically create object
2. **Dependent class :** dependent class means can create its object when target class creates its object means dependent class object creation is dependent on target class.



Q. What is tight coupling?
Tight coupling means when the target class is 100% dependent on a dependent class called tight coupling.
Q. What is loose coupling?
Loose coupling means when target class is partially depend on dependent class called as loose coupling
Note: we will discuss this concept at the time of abstract class and interface

Q. Is function overriding beneficial or not?

The answer is we cannot say yes or no
Because sometimes function is beneficial or sometimes not.
It is dependent on the situation and requirement of the application .
If parent class allow child class to modify its logic then overloading is beneficial but if parent do not allow child to modify its logic then it is not beneficial then it is problematic

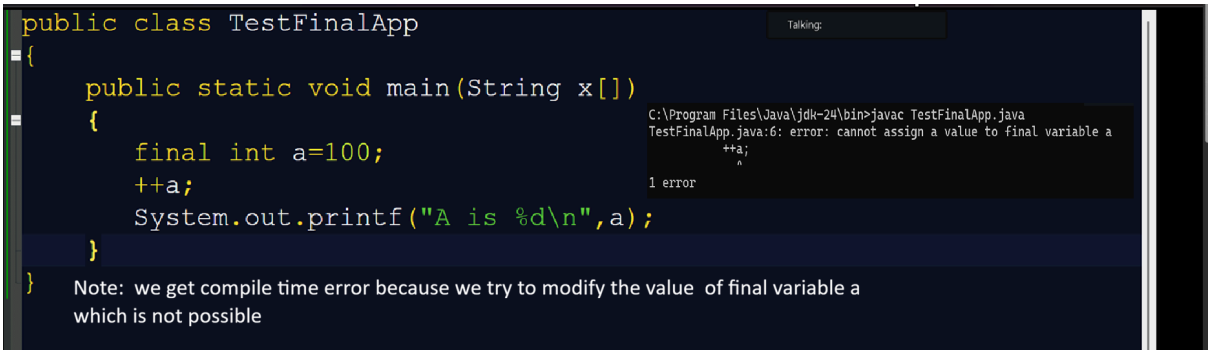
Q. How can we avoid method overriding?

If we want to avoid method overriding then we can define your method as final.

Q. What is final?

Final is a keyword which we can use with variable, function or class

Final variable: final variable means variable cannot modify its value once we initialize it. means in the case of java if we want to declare any variable as constant then mark it as final



Final method : final method cannot override in child class means final method specially used for avoid method overriding
If parent want to restrict child modify his logic then developer can define parent method as final method

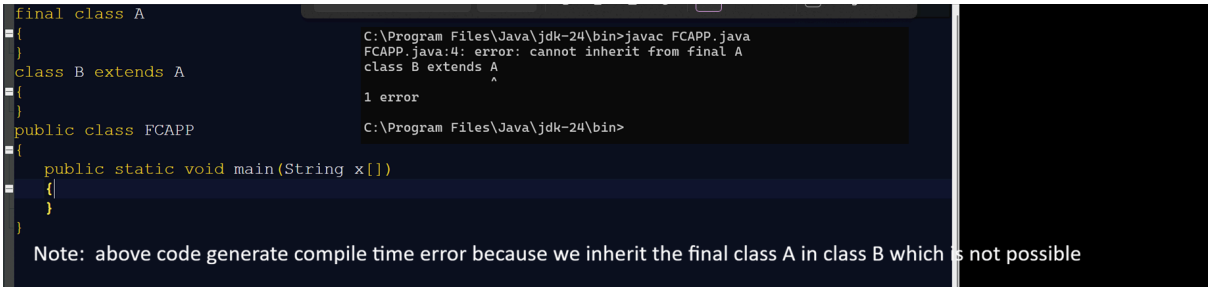
Example with source code

```
class Loan
{    int m;
          void applyInterest(){
                    m=0;
          }
          final void docReq(){
                    System.out.println("Adhar/PAN/salary sleep or 3 year return file");
          }
}
class Cooperative extends Loan
{
          void applyInterest(){
                    m=9;
          }

}
class Nationalize extends Loan {
          void applyInterest(){
                    m=7;
          }
```

```
}
public class TestFinalApp
{
   public static void main(String x[])
         {
           final int a=100;
           ++a;
           System.out.printf("A is %d\n",a);
         }
}
```

Final class: final class means class cannot inherit in another class and it is used for create immutable classes in java
Means if we try to inherit the final class in any other class there may be a compile time error.



Q. How can I use the final class?
You can use final class by creating only object

Q. Why do we use final class in application?
Final class is used for create the immutable classes in java

Q. What are immutable classes in JAVA?

Immutable classes means we cannot modify its object or change its object once we create object Or immutable means once we initialize value we cannot change it called as immutable
E.g String is a immutable class in java

Q. Can we create a user defined class as an immutable class?
Yes we can create a user class as an immutable class in JAVA.
If we want to create user class as an immutable class in JAVA we have some important rules

a. **All variable declared within class must be final**

b. **Class should contain parameterized constructor**
c. **No setter method should be exist in class**
d. **Just define getter methods for access data**

Example with source code

---

class Test

```
{
  private final int a;

  Test(int x){
   a=x;
  }

  int getA(){
      return a;
  }
}
public class TestFinalApp
{
   public static void main(String x[])
         {
           Test t = new Test(100);
           int result =t.getA();
           System.out.println("Result is  "+result);
         }
}
```

Q. Can the final method be inherited?

Means final method cannot modify by child object but can use by child object

Example with source code

```
class A
{         final void show(){
                  System.out.println("I am parent show method");
          }
}
class B extends A
{
}
public class FCAPP
{
  public static void main(String x[])
  {  B b1 = new B();
     b1.show();
  }
}
```

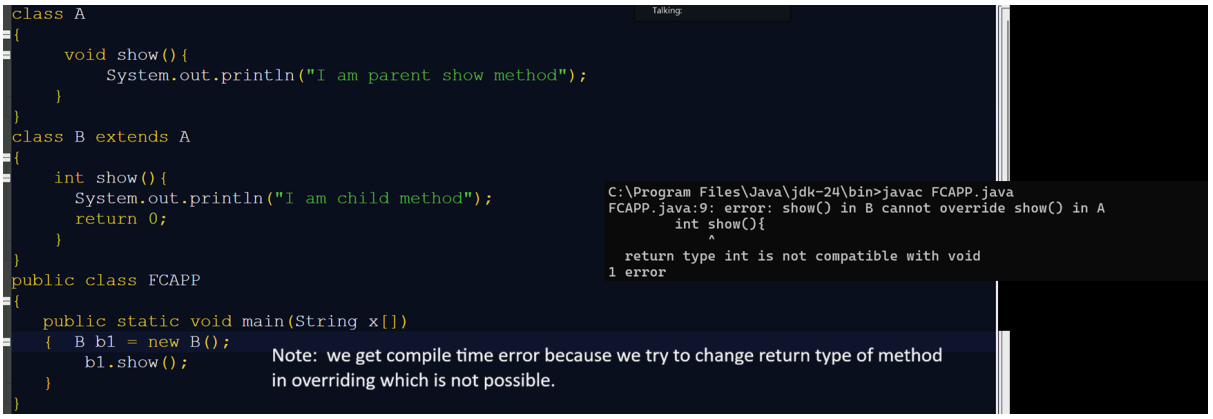
Rules of method overriding

---

a. Signature of parent method and child method must be same
   Signature means return type of method, parameter type of method , datatype of method must be same in parent class and in child class

```
class A
{
        void show(){
                System.out.println("I am parent show method");
        }
}
class B extends A
{
        void show(){
    System.out.println("I am child method");
        }
}
public class FCAPP
{
  public static void main(String x[])
  {  B b1 = new B();
     b1.show();
  }
}
```

Q. What will be the output of given code?



Q. What will be the output of given code?

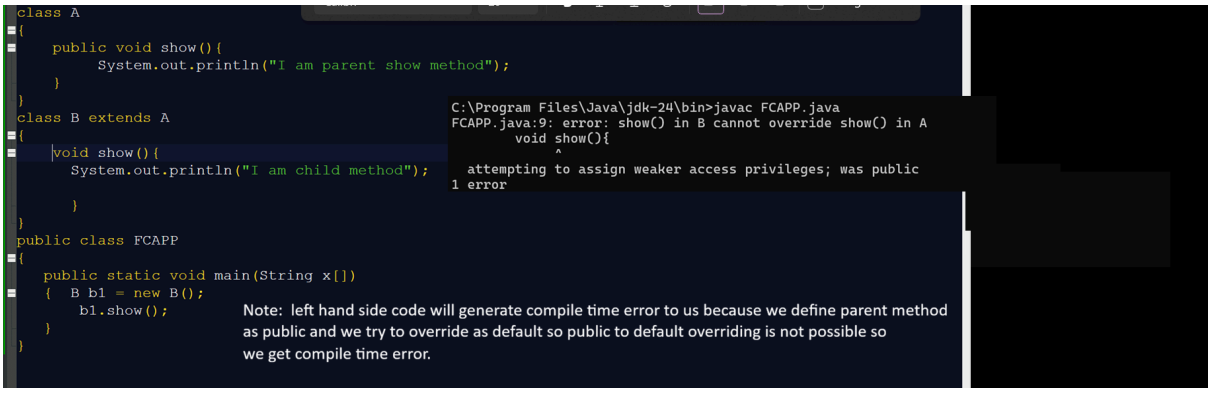


Note: In above code we change parameter type in overriding so internally it is considered as method overloading not overriding so there is no compile time error.

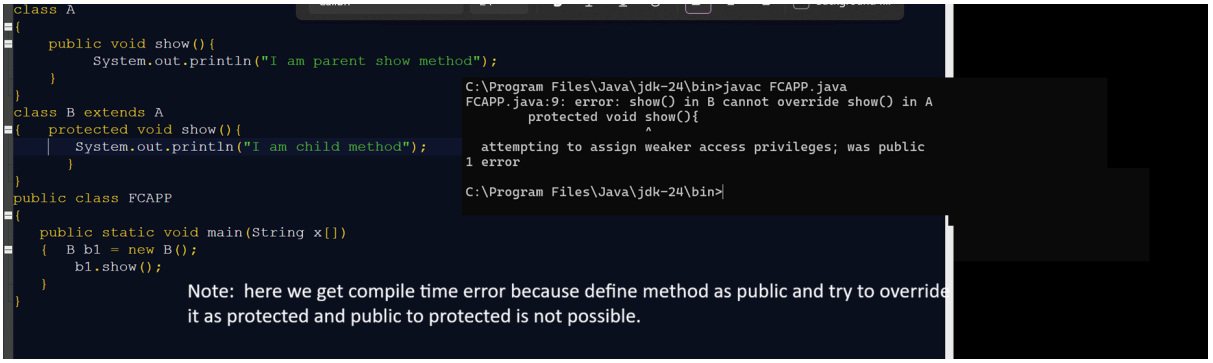
Access specifier rules in overriding

---

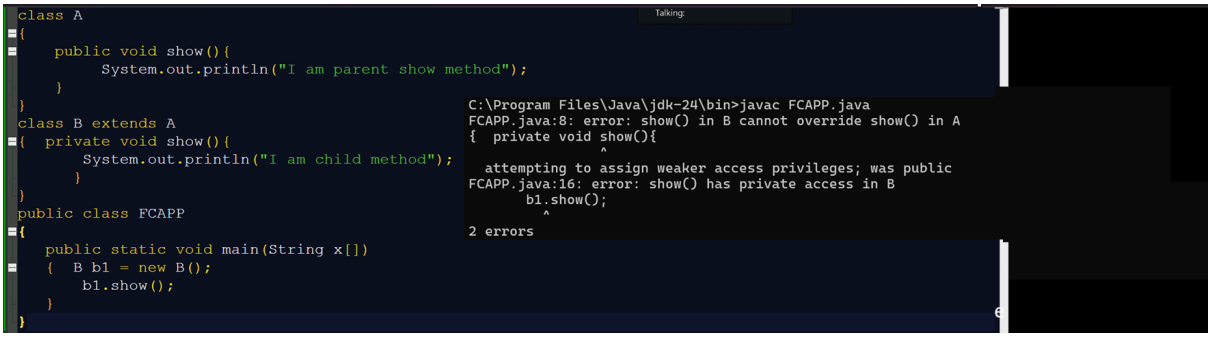
1. If we define method in parent class as public then we cannot override it as protected or private or default



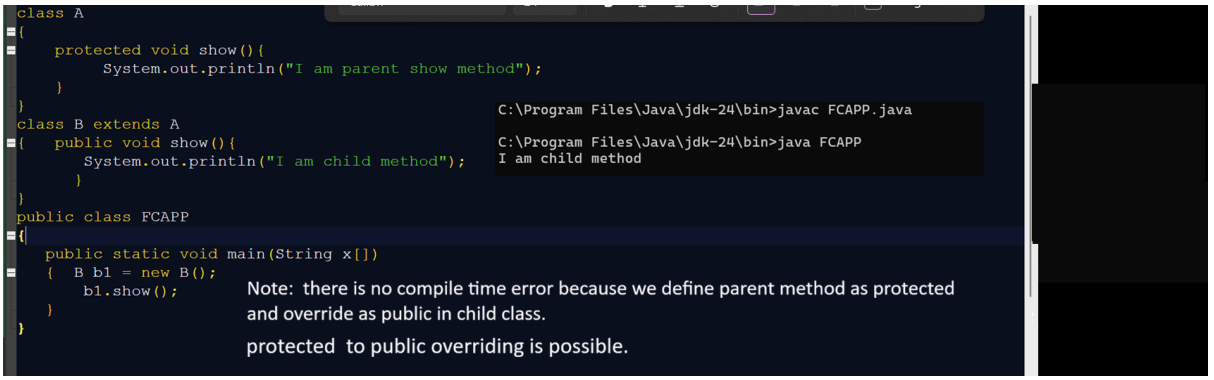
Or



Or



2. If method is protected then can override as public or protected but not default or private



Or

```java
class A
{
    protected void show(){
        System.out.println("I am parent show method");
    }
}
class B extends A
{     void show(){
       System.out.println("I am child method");
      }
}
public class FCAPP
{
   public static void main(String x[])
   {   B b1 = new B();
       b1.show();
   }
}
```

Note: if we think about left hand side code we get compile time error because we define parent method as protected and override it as default

protected to default is not possible.

```
C:\Program Files\Java\jdk-24\bin>javac FCAPP.java
FCAPP.java:8: error: show() in B cannot override show() in A
{     void show(){
            ^
  attempting to assign weaker access privileges; was protected
1 error

C:\Program Files\Java\jdk-24\bin>
```

Or

```java
class A
{
    protected void show(){
        System.out.println("I am parent show method");
    }
}
class B extends A
{    private void show(){
       System.out.println("I am child method");
      }
}
public class FCAPP
{
   public static void main(String x[])
   {   B b1 = new B();
       b1.show();
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac FCAPP.java
FCAPP.java:8: error: show() in B cannot override show() in A
{     void show(){
            ^
  attempting to assign weaker access privileges; was protected
1 error

C:\Program Files\Java\jdk-24\bin>javac FCAPP.java
FCAPP.java:8: error: show() in B cannot override show() in A
{    private void show(){
                 ^
  attempting to assign weaker access privileges; was protected
FCAPP.java:16: error: show() has private access in B
      b1.show();
        ^
2 errors
```

3. If method is default in parent class then we can override as protected, public but not private

```java
class A
{
    void show(){
        System.out.println("I am parent show method");
    }
}
class B extends A
{    public void show(){
       System.out.println("I am child method");
      }
}
public class FCAPP
{
   public static void main(String x[])
   {   B b1 = new B();
       b1.show();
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac FCAPP.java

C:\Program Files\Java\jdk-24\bin>java FCAPP
I am child method
```

Or

```java
class A
{
    void show(){
        System.out.println("I am parent show method");
    }
}
class B extends A
{    protected void show(){
       System.out.println("I am child method");
      }
}
public class FCAPP
{
   public static void main(String x[])
   {   B b1 = new B();
       b1.show();
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac FCAPP.java

C:\Program Files\Java\jdk-24\bin>java FCAPP
I am child method
```

Or

```
class A
{
    void show(){
        System.out.println("I am parent show method");
    }
}
class B extends A
{   private void show(){
      System.out.println("I am child method");
    }
}
public class FCAPP
{
   public static void main(String x[])
   {  B b1 = new B();
      b1.show();
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac FCAPP.java
FCAPP.java:8: error: show() in B cannot override show() in A
{   private void show(){
                 ^
  attempting to assign weaker access privileges; was package
FCAPP.java:16: error: show() has private access in B
      b1.show();
        ^
2 errors
```

4. We cannot override private method in child means if we define parent method as private we cannot override it
   Note: private method cannot inherit in child class and override is not possible without inheritance so we cannot declare parent method as private

```
class A
{
    private void show(){
        System.out.println("I am parent show method");
    }
}
class B extends A
{   private void show(){
      System.out.println("I am child method");
    }
}
public class FCAPP
{
   public static void main(String x[])
   {  B b1 = new B();
      b1.show();
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac FCAPP.java
FCAPP.java:16: error: show() has private access in B
      b1.show();
        ^
1 error
```

Abstract class and abstract methods

Abstract class means class cannot create its object and abstract method means cannot have logics

How to declare the abstract class and abstract methods

If we want to declare the abstract class and abstract method we have abstract keyword

Syntax of abstract class

---

abstract class classname{
}

Example: abstract class Demo{
}

```
abstract class Demo
{
}
public class AbsDemoApp
{
    public static void main(String x[])
    {
        Demo d=new Demo();
    }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac AbsDemoApp.java
AbsDemoApp.java:8: error: Demo is abstract; cannot be instantiated
            Demo d=new Demo();
                   ^
1 error
```

Note: above code generate the error to us because we try to create object of abstract class

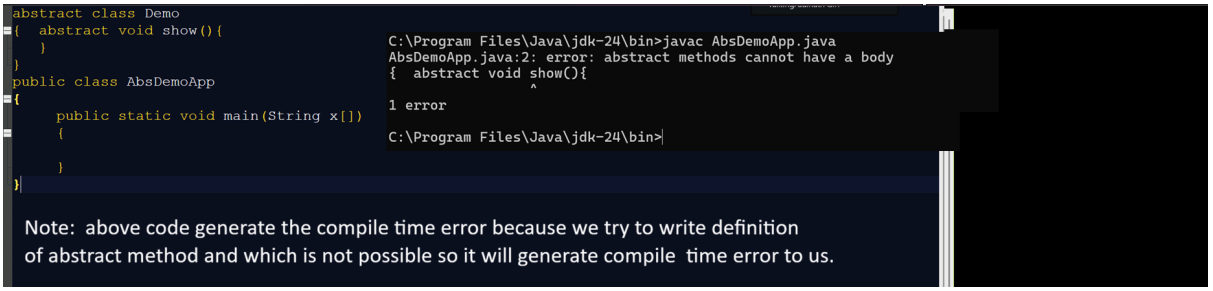
it is not possible

Syntax of abstract method

```
abstract class classname{
   abstract returntype functionname(datatype variablename);
}
```

Example:

```
abstract class Demo
{  abstract void show();
}
```



Q. What is the benefit of abstract class and abstract methods?

---

1. It is used for implement the abstraction
2. To it helps to achieve dynamic polymorphism
3. It help us to achieve loose coupling

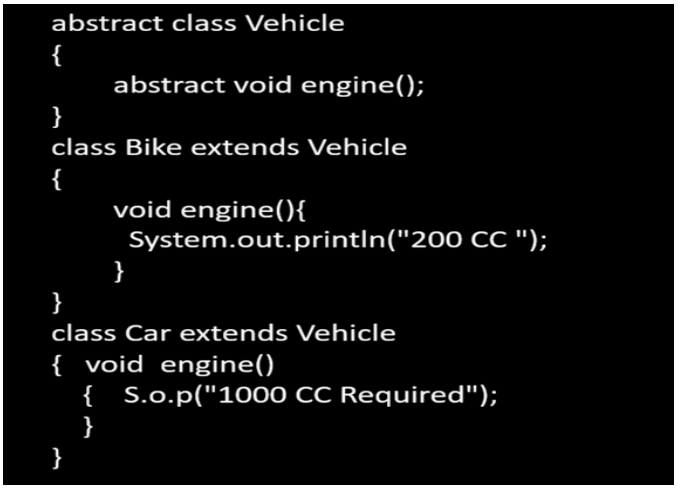
Note: if we want to use the abstract class and abstract methods we have to inherit the abstract class in child class and override its method and write its logic

Q. What is abstraction & how can we achieve it?

Abstraction is the process of hiding unnecessary details and showing only the essential features of an object or system. It helps reduce the complexity and make things easier to understand and use

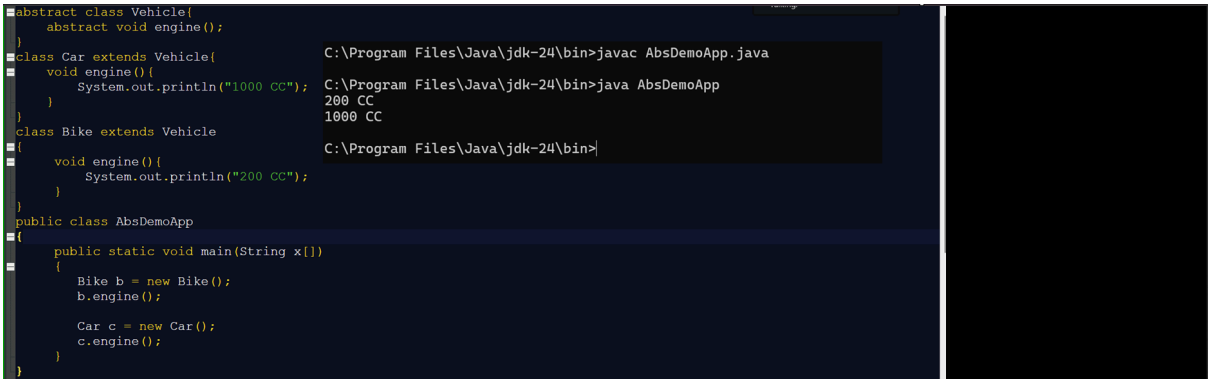
Abstraction means hide the implementation detail from end user at design level and implement the feature in future as per the requirement of system or application or object or child

Example: Suppose consider we want to implement the application Vehicle Manufacturer or company and company manufacture different kinds of vehicle like as Car, Bike etc



If we think about example at the time Vehicle just we abstraction of vehicle means just mention Vehicle required engine but if we think about Bike and Car we provide implementation to vehicle method as per their required means engine() method is implement by Car object as per their need and Bike object implemented engine() method as per their need.

If we want to use the engine() method of Vehicle we have to create object of Bike and Car class

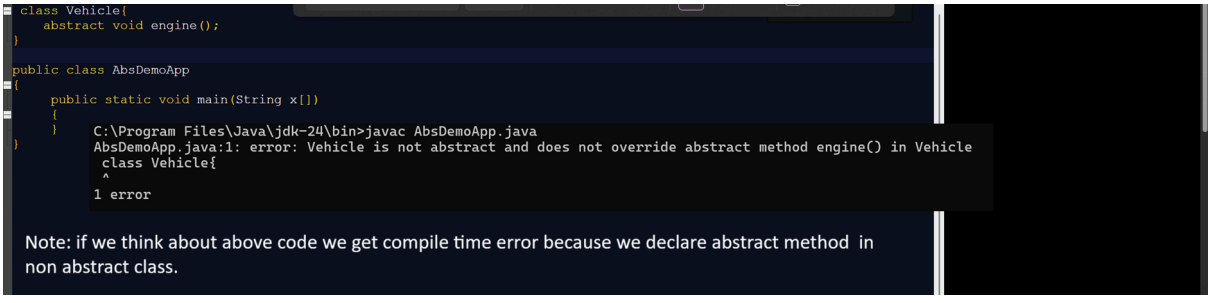


If we want to work with abstract class and abstract methods we have some important points

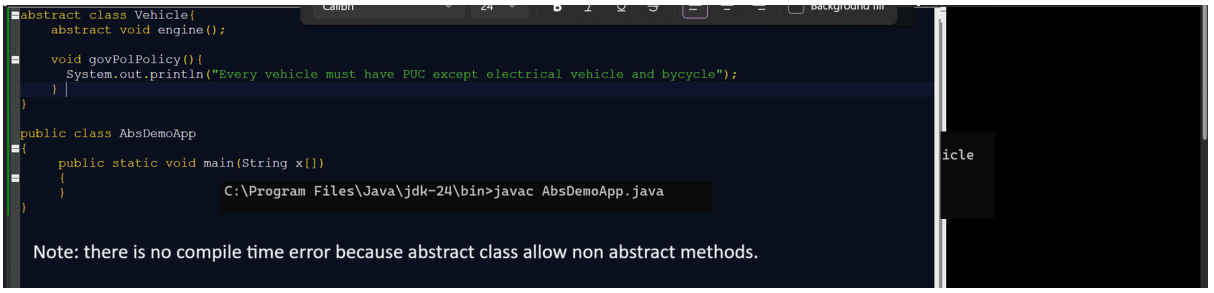
1. **Abstract class cannot create its object**
   Q. Why can we not object to abstract class?
   Because if we think about abstract class it contain abstract method and non abstract method also suppose consider OOP allow us create object of abstract class using that object there is possible developer can call abstract method but in abstract class abstract method has no definition or logical block
   And without definition we cannot call any method for avoiding this possible OOP restrict developer to create object of abstract class

2. **Abstract method cannot have logics**
   Using abstract method developer can achieve abstraction in system or application means abstract method give flexibility to developer write its logics differently in every child class as per need by using overriding technique means abstract method cannot have logic in abstract class but abstract method can have logic in its child classes

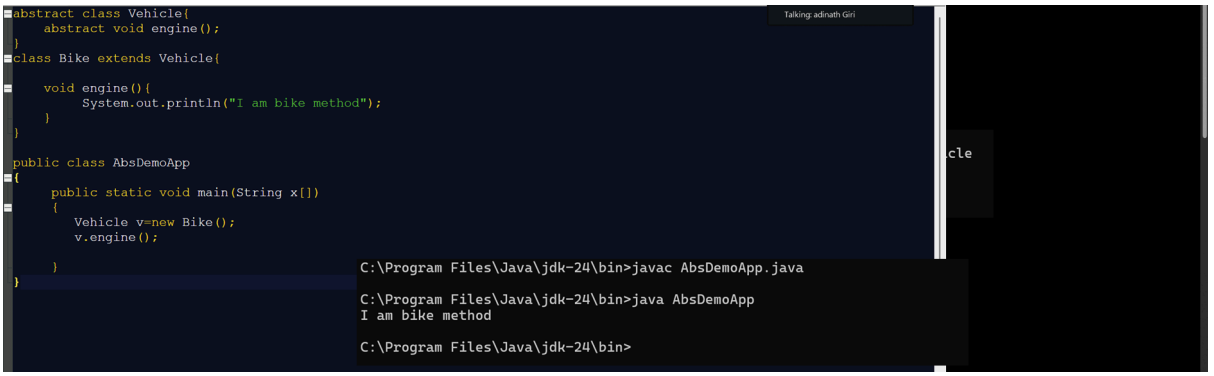
3. A**bstract method must have abstract class**
   If we want to declare the abstract method your class must be abstract you cannot declare the abstract method in non abstract class.



4. **Abstract class can allow non abstract methods**
   The meaning we can declare non abstract as well as abstract method in abstract class



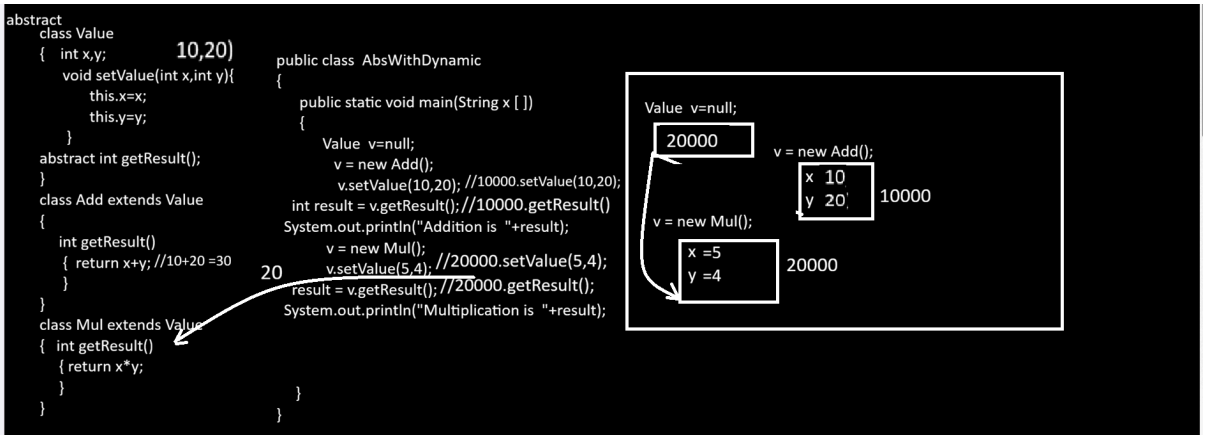
5. **Abstract class cannot create object but can its reference**
   If we want to create reference of abstract class we must have to create object of its child class by using upcasting technique



Q. Why does OOP allow the creation of references to abstract classes or What is the benefit of abstract class reference?
The major benefit of abstract class reference is

a. To achieve dynamic polymorphism
b. To achieve loose coupling

Example with source code



abstract class Value
{ int x,y;
void setValue(int x,int y)

```
  {  this.x=x;
     this.y=y;
  }
  abstract int getResult();
}
class Add extends Value
{
  int getResult(){
     return x+y;
  }
}
class Mul extends Value
{ int getResult(){
     return x*y;
  }
}
public class AbsDynamicApp
{
  public static void main(String x[])
  {
      Value v=null;
            v=new Add();
                  v.setValue(10,20);
          int result=v.getResult();
          System.out.println("Addition is  "+result);
            v=new Mul();
            v.setValue(5,4);
            result=v.getResult();
            System.out.println("Multiplication is "+result);
  }
}
```

Note:before understanding loose coupling we need to know what is coupling.

Q. What is coupling?

Coupling indicate the relationship between two objects with each other called as coupling

There are two types of coupling?

---

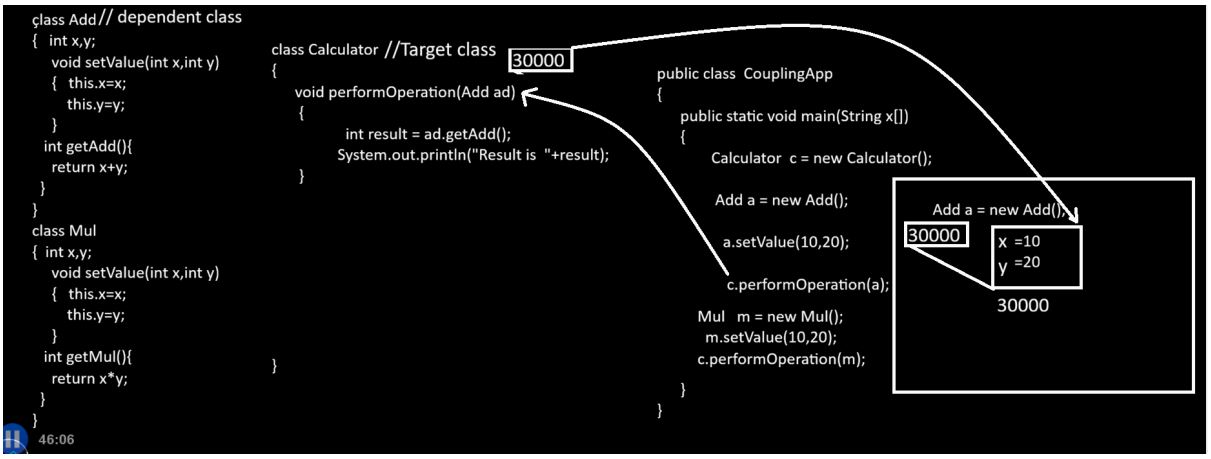
a. **Tight coupling:** tight coupling means when one object is creation or used 100% dependent on another object called tight coupling.

b. **Loose coupling :** object creation partially dependent on another object called as loose coupling

   Note: if we want to understand the terms of tight coupling and loose coupling we need to clear the two concepts or basic terms

a. **Target class :** target class means class using that we can create object of dependent class or use dependent class functionality
b. **Dependent class:** dependent class means class whose object use is dependent on target class object

Example: Suppose consider we want to develop the Calculator application and Calculator has four types of Operation Add, Mul etc and we develop the class structure by using tight coupling concept



Note: if we think about above code it is example of tight coupling because if we think about performOperation(Add ad) this method cannot accept any other class reference except Add means this method is 100% dependent on Add class reference and if we try to pass reference of any other class we get compile time error means in our code we get compile time error because we try to Mul reference in performOperation() method

Example with source code

```
class Add
{
  int x,y;
  void setValue(int x,int y)
  {this.x=x;
   this.y=y;
  }
  int getAdd(){
     return x+y;
  }
}
class Mul
{ int x,y;
  void setValue(int x,int y)
  {this.x=x;
   this.y=y;
  }
  int getAdd(){
     return x*y;
  }
}
```

```
class Calculator
{
   void performOperation(Add ad)
   { int result =ad.getAdd();
      System.out.println("Addition is  "+result);
   }
}
public class CouplingApp
{
    public static void main(String x[])
          {
             Calculator  c =new Calculator ();
                   Add a=new Add();
                   a.setValue(10,20);
                    c.performOperation(a);

                   Mul m= new Mul();
                   m.setValue(5,4);
                   c.performOperation(m);

          }
}
```

Output

```
C:\Program Files\Java\jdk-24\bin>javac CouplingApp.java
CouplingApp.java:40: error: incompatible types: Mul cannot be converted to Add
                c.performOperation(m);
                                   ^
Note: Some messages have been simplified; recompile with -Xdiags:verbose to get full output
1 error

C:\Program Files\Java\jdk-24\bin>
```

Q. How can we solve this problem or the above code problem?

We can solve this problem by using two approaches

a. **Using compile time polymorphism i.e function overloading**
b. Using runtime polymorphism i.e function overriding with abstract class or interface

Solution using compile time polymorphism

```
class Add
{
  int x,y;
  void setValue(int x,int y)
  {this.x=x;
   this.y=y;
  }
  int getAdd(){
     return x+y;
```

```
  }
}
class Mul
{ int x,y;
  void setValue(int x,int y)
  {this.x=x;
   this.y=y;
  }
  int getMul(){
     return x*y;
  }
}
class Calculator
{
   void performOperation(Add ad)
   { int result =ad.getAdd();
      System.out.println("Addition is  "+result);
   }
   void performOperation(Mul m)
   { int result =m.getMul();
      System.out.println("Multiplication is  "+result);
   }
}
public class CouplingApp
{
    public static void main(String x[])
          {
             Calculator  c =new Calculator ();
                   Add a=new Add();
                   a.setValue(10,20);
                    c.performOperation(a);

                   Mul m= new Mul();
                   m.setValue(5,4);
                   c.performOperation(m);

          }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac CouplingApp.java

C:\Program Files\Java\jdk-24\bin>java CouplingApp
Addition is  30
Multiplication is  20

C:\Program Files\Java\jdk-24\bin>
```

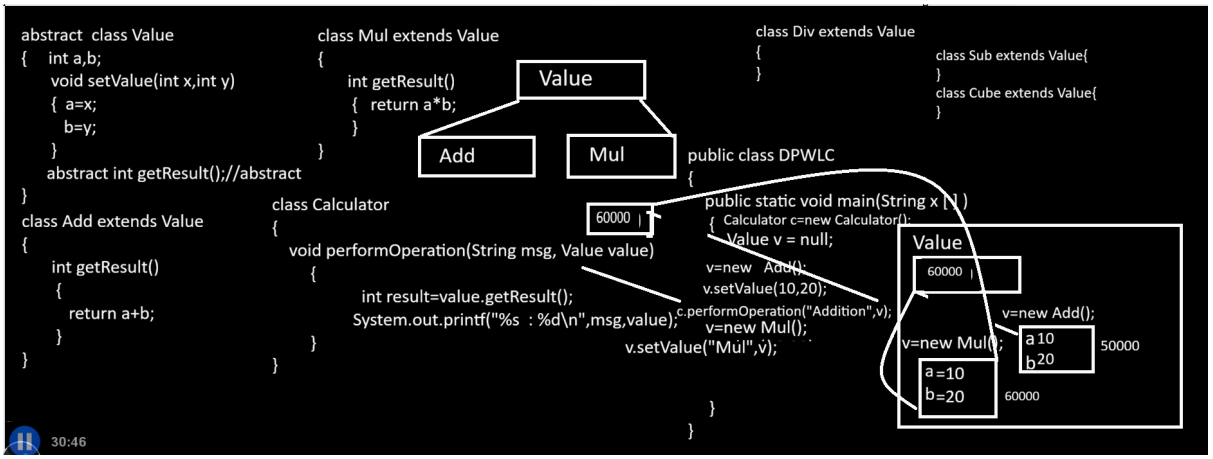
Note: if we think about above code we solve it using a compile time polymorphism
But there is limitation suppose if we have 100 operation in Calculator and if we develop the 100 classes for perform operation then we need to overload 100 methods in Calculator class

It is not feasible solution in real life but suppose if we define single method and method has ability to manage 100 operation or single method can behave like as 100 methods then Dynamic polymorphism and loose coupling comes in picture but we can manage loose coupling by using two ways

a. Abstract class
b. Interface

Now we want to implement the loose coupling by using abstract class

---



Example with source code

```
abstract class Value
{  int a,b;
   void setValue(int x,int y)
   { a=x;
     b=y;
   }
   abstract int getResult();
}
class Add extends Value
{
   int getResult()
   {  return a+b;
   }
}
class Mul extends Value
{  int getResult()
   { return a*b;
   }
}
class Calculator
{
   void performOperation(String msg,Value value)
   {
      int result=value.getResult();
     System.out.printf("%s\t%d\n",msg,result);
```

```
  }
}
public class DPWLC
{
   public static void main(String x[])
        {
          Calculator c= new Calculator();
           Value v=null;
                v=new Add();
                v.setValue(10,20);
                c.performOperation("Addition is ",v);
                v=new Mul();
                v.setValue(5,4);
                c.performOperation("Multiplication is ",v);
        }
}
```

6. **Abstract method cannot declare as final**

   Because final method cannot be overridden in child class but abstract method must have override so the behaviour of final and abstract are opposite of each other so abstract method cannot declare as final and if we try to declare as final we get compile time error.

```
abstract class Test
{
  final abstract void show();
}
class TestImpl extends Test{

 void show()
 { System.out.println("I am show from Test");
 }
}
public class TestImplApp
{
   public static void main(String x[])
        {  TestImpl ti=new TestImpl();
           ti.show();
        }
}
```

7. **Abstract method cannot declare as static**

   Reason static method cannot work with abstract

   a. Static method cannot support to overriding and abstract must be override
   b. Static method cannot support dynamic polymorphism and abstract method specially used for dynamic polymorphism implementation purpose.

These points indicate behaviour of static and abstract opposite of each other so we cannot declare static method as private and if we try to declare it as private then we get compile time error

Example with source code

```
abstract class Test
{
   static abstract void show();
}
class TestImpl extends Test{
  void show()
  { System.out.println("I am show from Test");
  }
}
public class TestImplApp
{   public static void main(String x[])
        {  TestImpl ti=new TestImpl();
           ti.show();
        }
}
```

8. **Abstract method cannot be declared as private :** private method cannot support inheritance and overriding and abstract must work with inheritance and overriding so we can not declare abstract method as private.
   If we declare it we get compile time error

   Example with source code

```
abstract class Test
{
   private abstract void show();
}
class TestImpl extends Test{

  void show()
  { System.out.println("I am show from Test");
  }
}
public class TestImplApp
{
    public static void main(String x[])
        {  TestImpl ti=new TestImpl();
           ti.show();
        }
}
```

9. **Abstract class can use without inheritance**

Yes we can use the abstract class without inheritance with the help of anonymous inner class.

Q. What is an anonymous inner class?

Anonymous inner class is a local inner class where we can create objects of the class as well as define its body and override its method called as anonymous inner class.

Example with source code

Syntax of anonymous inner class

```
classname  ref = new classname(){
 //override here methods
};
```

Example with source code

```
abstract class Test
{  abstract void show();
}
public class TestImplApp
{
   public static void main(String x[])
         {  Test t= new Test(){
                  void show(){
                           System.out.println("I am show methods");
                  }
            };
                  t.show();
         }
}
```

Interface

Interface is same like as abstract class and which is used for achieve abstraction,dynamic polymorphism and multiple inheritance

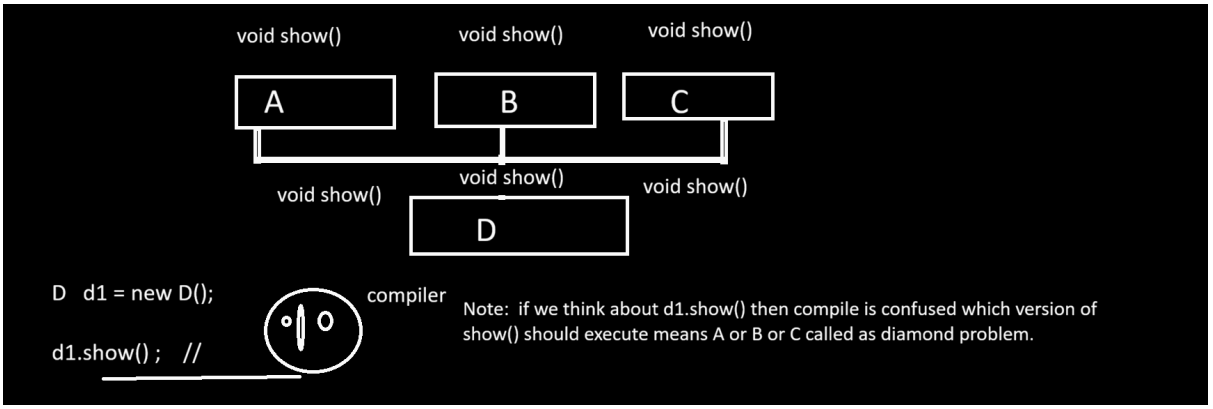
Q. Why use an interface if we have an abstract class already?

1. Interface can achieve 100% abstraction
2. Interface can implement multiple inheritance and solve diamond problem
3. Interface can support to loose coupling also

Interface can achieve 100% abstraction : because in interface by default methods are abstract not need to mention it using abstract keyword and interface also internally work as abstract

Q. Why does JAVA not use classes to achieve multiple inheritance?

Because of diamond problem here diamond problem means if we have multiple parent classes and if different parent contain same name method so the method name is replicated in child classes and if we create object of child class and try to call method whose name same in different parent then your compiler get confused for which parent should execute called as diamond problem



So if we want to solve this problem we can use interface

How to use interfaces practically in JAVA?

Steps to use interface practically in JAVA

1. **Declare the interface**

Syntax:

interface interfacename
{
}

Example:

interface Test {
}

2. **Declare method within interface :** if we want to declare method then not need to define its body because by default interface method is public abstract

Syntax:

Example:

interface Test
{ void show();
}

3. **Create user define class and implement interface in it and override its method**

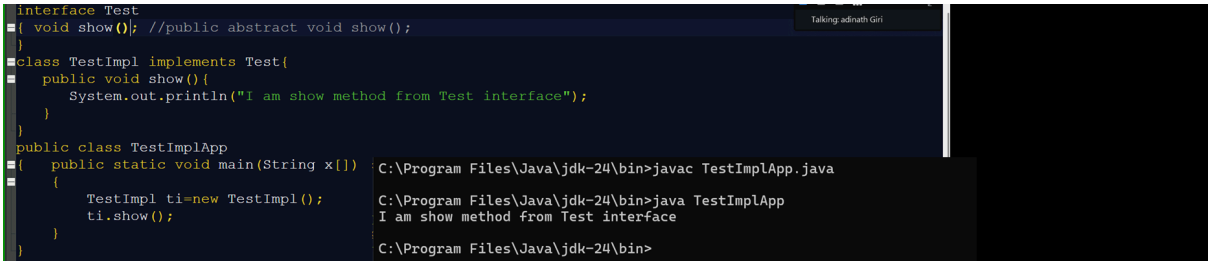
If we want to use interface we have to implement interface in any user define class or implementer class so if we want to implement interface in user define class we have to use implements keyword

Syntax: class classname implements interfacename{
public returntype methodname(datatype variablename)
{ //write here your logics
}
}

Example with source code

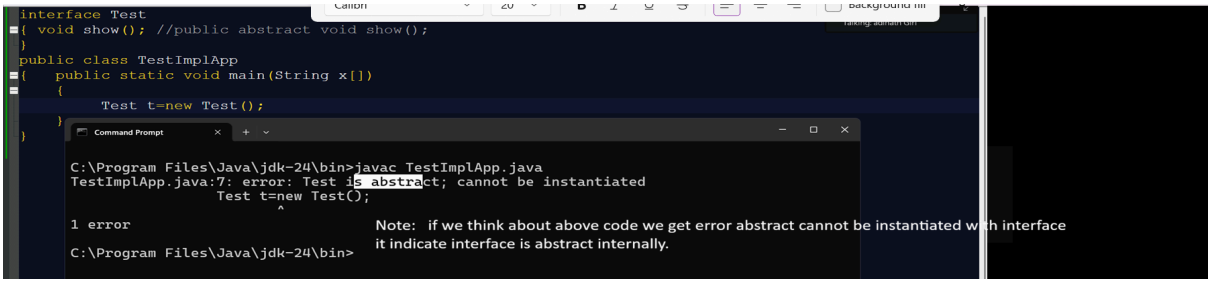
```
interface Test
{ void show(); //public abstract void show();
}
class TestImpl implements Test{
  public void show(){
    System.out.println("I am show method from Test interface");
  }
}
```

4. **Create object of implementer class and call its overridden method**

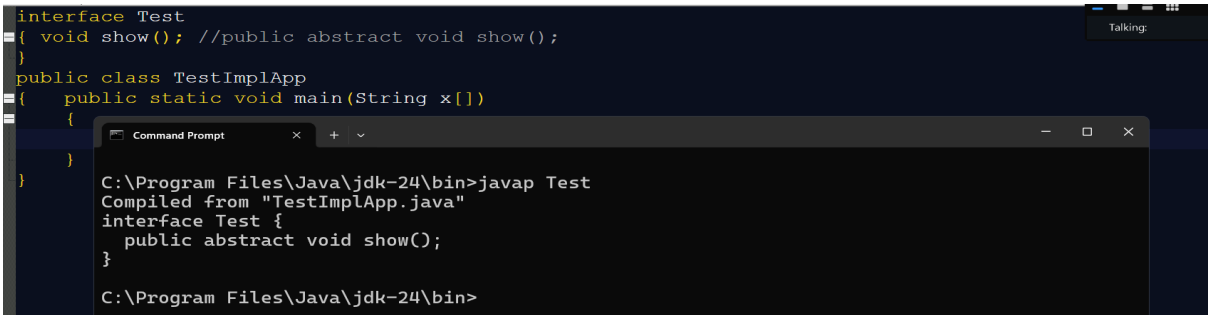


If we want to work with interfaces in JAVA we have some important points.

1. **Interface cannot create its object**
   Because interface is an internally abstract class and we cannot create object of abstract class so interface cannot create its object and if we try to create its object we get compile time error.

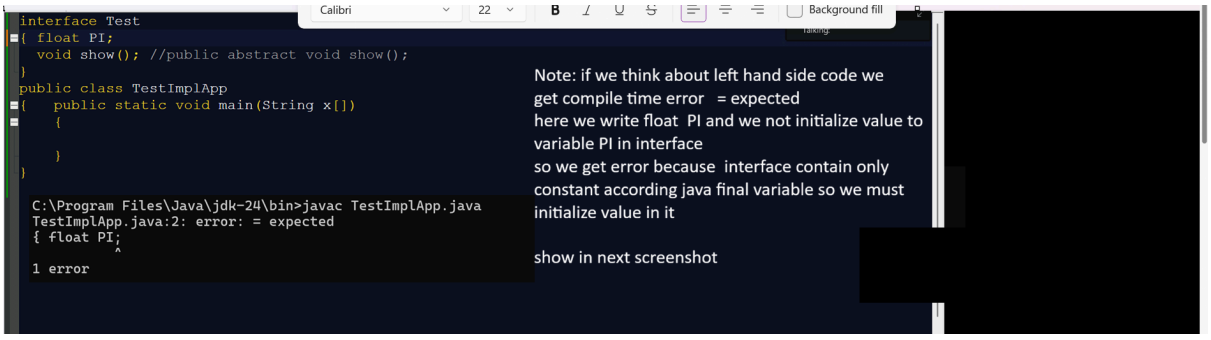


2. **Interface method cannot have logics :** because interface methods are public abstract internally and we cannot write definition of abstract methods

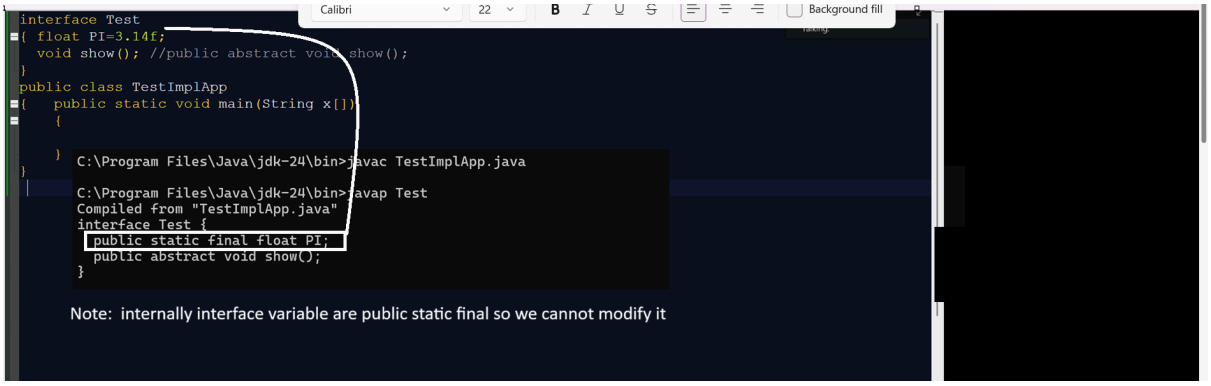


3. **Interface variables are by default final**

Means if we want to declare a variable within an interface we must initialize some value in it. If we not initialize value in interface variable we get compile time error means interface use constant values only



Without error screenshot give below



Q. Why are interface variables static?

Interface cannot create its object and if we want to access its variable without object then we can access it using interface name so they are internally static and static member can access via interface name or classname



4. **Interface method cannot mark as final:** because interface method is internally public abstract and abstract method cannot mark as final so interface method cannot mark as final

```java
interface Test
{ float PI=3.14f;
  final void show(); //public abstract void show();
}
public class TestImplApp
{   public static void main(String x[])
    {
        System.out.println("PI is  "+Test.PI);
    }
}
```

```
Command Prompt

C:\Program Files\Java\jdk-24\bin>javac TestImplApp.java
TestImplApp.java:3: error: modifier final not allowed here
  final void show(); //public abstract void show();
             ^
1 error
```

5. **Interface method cannot mark as static :** because interface method internally abstract and abstract cannot be static so interface method cannot declare as static and if we try then we get compile time error.

```java
interface Test
{ float PI=3.14f;
  static void show(); //public abstract void show();
}
public class TestImplApp
{   public static void main(String x[])
    {
        System.out.println("PI is  "+Test.PI);
    }
}
```

```
Command Prompt

C:\Program Files\Java\jdk-24\bin>javac TestImplApp.java
TestImplApp.java:3: error: missing method body, or declare abstract
  static void show(); //public abstract void show();
              ^
1 error
```

6. **Interface method cannot mark as private:** private cannot support inheritance and overriding and interface method cannot work without inheritance or overriding as well as interface method is public and if we declare it as private then internally compiler consider private public at same time and in java there is not private public access specifier.

```java
interface Test
{ float PI=3.14f;
   private void show(); //public abstract void show();
}
public class TestImplApp
{   public static void main(String x[])
    {
        System.out.println("PI is  "+Test.PI);
    }
}
```

```
Command Prompt

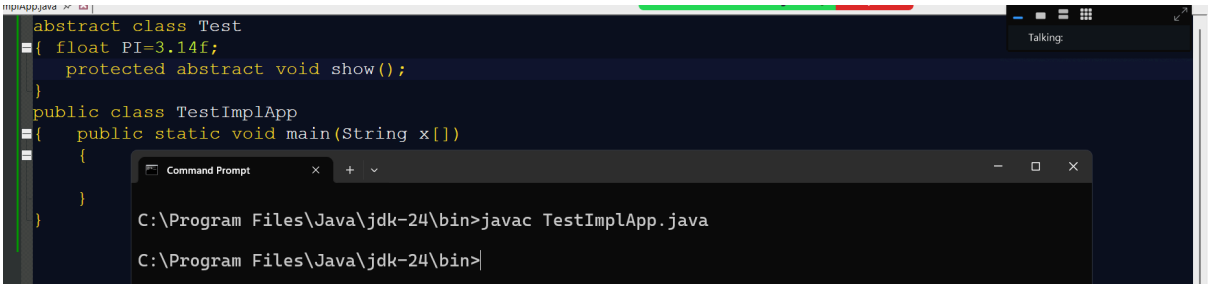
C:\Program Files\Java\jdk-24\bin>javac TestImplApp.java
TestImplApp.java:3: error: missing method body, or declare abstract
   private void show(); //public abstract void show();
                ^
1 error

C:\Program Files\Java\jdk-24\bin>
```

7. **Interface method cannot declare or mark as protected**
   Before solve this point we want to discuss some question

Q. Can we declare abstract methods as protected in abstract class?

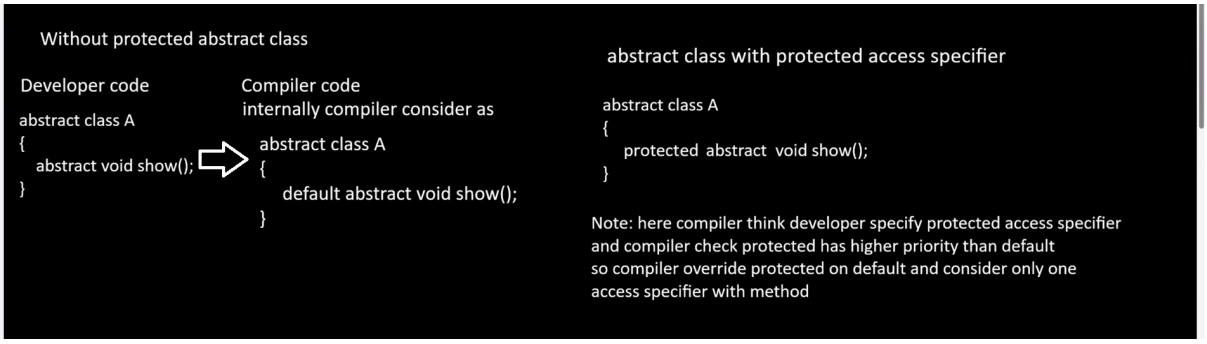
Yes we can declare abstract method as protected in abstract class



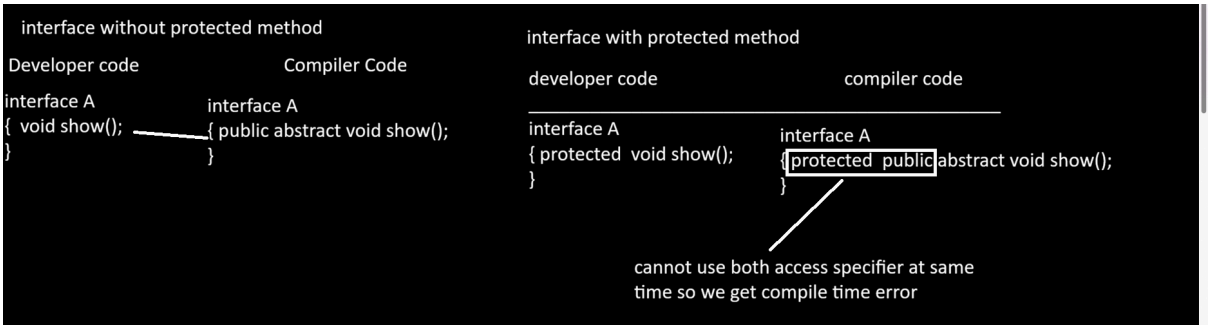
Q. If abstract class abstract method declare as protected and interface method is also abstract so why does interface method not declare as protected?

The reason if we think about abstract class abstract method if we not specify any access specifier with abstract method in abstract class then method by default use default access specifier and protected has higher priority than default so when we declare abstract method in abstract class as protected then protected get override on default so we can declare abstract method in abstract class as protected.

Shown in following diagram



If we think about interface abstract method internally it is public abstract and when we try to declare it as protected then compiler consider method protected public returntype methodname() means compiler use two access specifier at time on method and it is not possible because protected has less priority than public and when developer add protected with interface method then compiler try to override protected on public but it is not possible so compiler use protected before public so it will protected public access specifier in java there is protected or public not combination so we get compile time error shown in following screenshot



```
interface Test
{
   protected abstract void show();
}
public class TestImplApp
{    public static void main(String x[])
     {

     }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac TestImplApp.java
TestImplApp.java:3: error: modifier protected not allowed here
   protected abstract void show();
                           ^
1 error
```

8. **Cannot create object of interface but can create its reference**

So if we want to create reference of interface we have to create object its implementer class using upcasting technique

```
interface Test
{
     void show();
}
class TestChild implements Test
{
       public void show()
       {  System.out.println("I am show from Test");
       }
}
public class TestImplApp
{    public static void main(String x[])
     {
           Test t=new TestChild();
           t.show();
     }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac TestImplApp.java

C:\Program Files\Java\jdk-24\bin>java TestImplApp
I am show from Test

C:\Program Files\Java\jdk-24\bin>
```

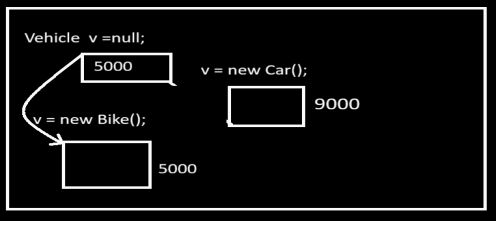
Note: The goal of interface reference is to achieve dynamic polymorphism and loose coupling.

```
interface Vehicle
{  void engine();
}
class Car implements Vehicle
{  void engine()
   { System.out.println("I am car engine");
   }
}
class Bike implements Vehicle
{ void engine()
  { System.out.println("I am bike engine");
  }
}

public class DynamicApp
{  public static void main(String x[])
   {
     Vehicle v =null;
     v = new Car();
     v.engine();

     v = new Bike();
     v.engine();

   }
}
```



Note: above code represent the dynamic polymorphism

```
interface Vehicle
{ void engine();
}
class Car implements Vehicle
{ public void engine(){
      System.out.println("I am Car engine method");
   }
}
class Bike implements Vehicle
{ public void engine()
   { System.out.println("I am bike engine method");
   }
}
public class DynamicApp
{  public static void main(String x[])
   {    Vehicle v=new Car();
        v.engine();
        v=new Bike();
        v.engine();
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac DynamicApp.java

C:\Program Files\Java\jdk-24\bin>java DynamicApp
I am Car engine method
I am bike engine method

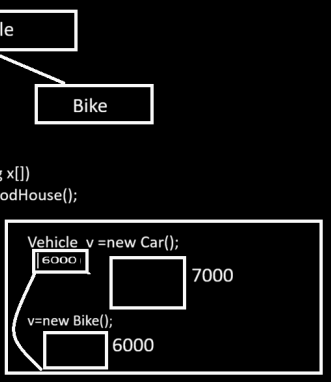
C:\Program Files\Java\jdk-24\bin>
```

Example of loose coupling

```
interface Vehicle
{   void  engine();
}
class Car implements Vehicle
{   void engine()
    { System.out.println("I am car engine");
    }
}
class Bike implements Vehicle
{ void engine()
  { System.out.println("I am bike engine");
  }
}
class  ProdHouse
{
    void manuVehicle(Vehicle  v )
    {
        v.engine();
    }
}
public class LooseCoupApp
{   public static void main(String x[])
    {     ProdHouse ph =new ProdHouse();
          Vehicle  v =new Car();
           ph.manuVehicle(v);

          v = new Bike();
          ph.manuVehicle(v);


    }
}
```



9. **Interface to interface inheritance use extends keyword**
   Note: when we perform inheritance between interface to interface we can use extends keyword

```
interface A
{ void show();
}
interface B extends A
{  void display();
}
class C implements B
{
   public void show(){
     System.out.println("I am show method");
   }
   public void display(){
     System.out.println("I am display method");
   }
}
public class InfInhApp
{   public static void main(String x[])
    {
        C c1 = new C();
        c1.show();
        c1.display();
    }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac InfInhApp.java

C:\Program Files\Java\jdk-24\bin>java InfInhApp
I am show method
I am display method

C:\Program Files\Java\jdk-24\bin>
```

10. **Interface can declare within class or class within interface also**

```
class T
{
   void disp()
   { System.out.println("I am display method of T");
   }
   interface M
   {  void show();
   }
}
class N extends T implements T.M
{
    public void show()
    {  System.out.println("I am M method");
    }
}
public class NestInfApp
{  public static void main(String x[])
   {
       N n1 = new N();
       n1.show();
       n1.disp();
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac NestInfApp.java

C:\Program Files\Java\jdk-24\bin>java NestInfApp
I am M method
I am display method of T

C:\Program Files\Java\jdk-24\bin>
```

Or

```
interface  T
{
  void disp();
  class  M
  {  void show(){
       System.out.println("I am M method");
     }
  }
}
class N extends T.M  implements T
{
    public void disp(){
         System.out.println("I am disp method");
    }
}
public class NestInfApp
{  public static void main(String x[])
   {
       N n1 = new N();
       n1.show();
       n1.disp();
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac NestInfApp.java

C:\Program Files\Java\jdk-24\bin>java NestInfApp
I am M method
I am disp method
```

Or

```
interface  T
{
  void disp();
  class  M implements T
   {  void show(){
            System.out.println("I am M method");
     }
          public void disp(){
                  System.out.println("I am T Method");
          }
   }
}
class N extends T.M
{

}
public class NestInfApp
{  public static void main(String x[])
   {
            N n1 = new N();
            n1.show();
            n1.disp();
   }
}
```

11. If an interface contains more than one method then all methods must be overridden where interfaces get implemented and need to override methods in implement if we are not required.

Example with source code

```
interface A
{
   void s1();
   void s2();
   void s3();
}
class B implements A
{
   public void s1()
   { System.out.println("I am s1 method");
   }
   public void s2(){
   }
   public void s3(){
   }
}
class C implements A
```

```
{
   public void s2()
   { System.out.println("I am s2 method");
   }
   public void s1(){
   }
   public void s3(){
   }
}
class D implements A
{  public void s3()
   { System.out.println("I am s3 method");
   }
   public void s1(){
   }
   public void s2(){
   }
}
public class InfApp
{
   public static void main(String x[])
   {

   }
}
```

Note: this is the limitation of abstract methods in abstract class or in interface because this technique generate the boilerplate code
So if we want to avoid this situation we have to use adapter class concept

Q. What is an adapter class?

Adapter class is intermediate class which contain all blank methods of abstract class or interface and this class is able to give specific method to child of interface or adapter class
Means implementer class of interface not need to override all methods

```
interface A
{
   void s1();
   void s2();
   void s3();
}
class ADP implements A
{
        public void s1(){
        }
        public void s2(){
        }
```

```
        public void s3(){
        }
}
class B extends ADP
{
  public void s1()
  { System.out.println("I am s1 method");
  }

}
class C extends ADP
{
  public void s2()
  { System.out.println("I am s2 method");
  }

}
class D extends ADP
{  public void s3()
  { System.out.println("I am s3 method");
  }

}
public class InfApp
{
  public static void main(String x[])
  {
                B b1 =new B();
                b1.s1();
                C c1=new C();
                c1.s2();
                D d2 =new D();
                d2.s3();
  }
}
```

12. We can define static method within interface from JDK 1.8 version

Note: you cannot declare the static method within interface but we can define it and we can use the static using interface name directly

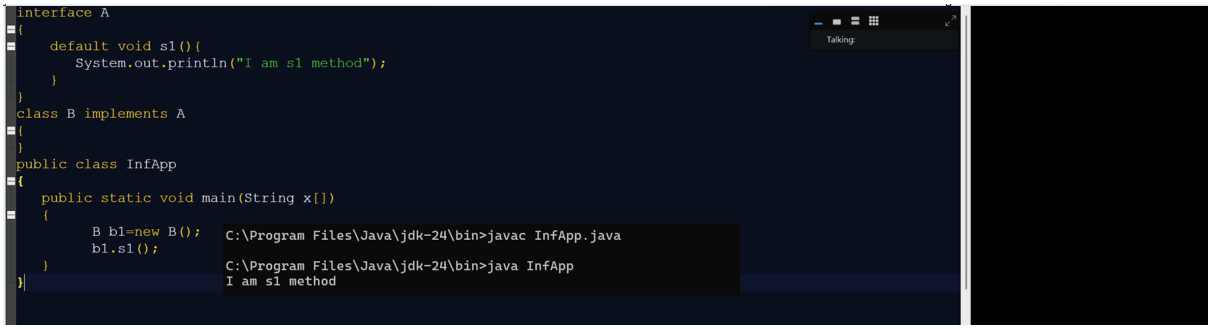
```
interface A
{   static void s1(){
          System.out.println("I am s1 method");
  }
}

public class InfApp
{    public static void main(String x[]) {
                A.s1();
  }
```

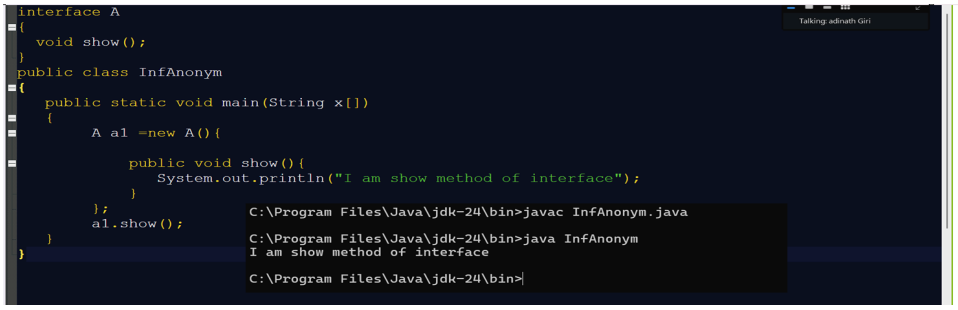
}

13. We can define default method within interface from JDK 1.8 version

If we use the default keyword with method in interface then we can define method in interface but default method can use only object of implementer class of interface



14. We can use the interface without implementation using anonymous inner class



15. From JDK 1.8 there is functional interface concept

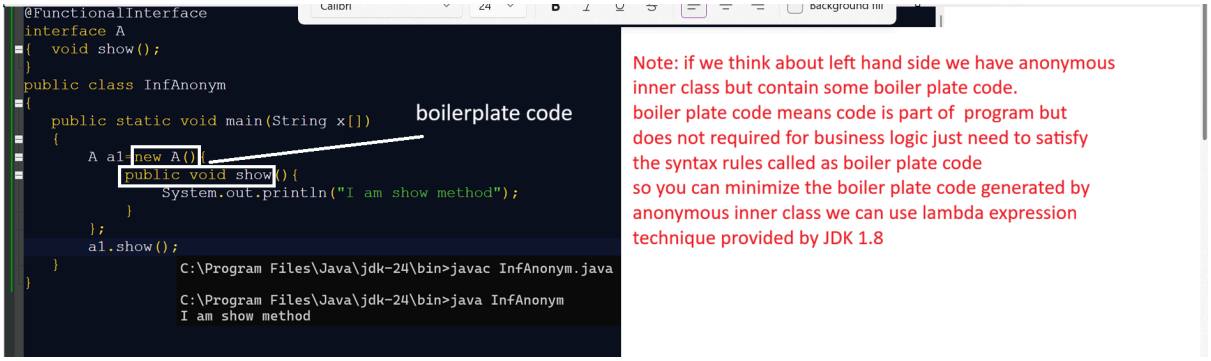
Functional interface means an interface containing only one abstract method and more than one static or default method called a functional interface and if we want to create functional interface in java we have annotation @FunctionalInterface

@FunctionalInterface
interface A
{ void show();
}

Q. What is the goal of functional interface in JAVA?

The goal of the functional interface is to work with lambda expressions. Lambda expression is short cut of anonymous inner class because anonymous generate the boiler plate and if we have only one abstract method in interface we can use lambda expression as short of anonymous inner

Syntax: Interfacename ref=(parameter)=>{ write here logics }



Example using lambda expression

```java
@FunctionalInterface
interface A
{  void show();
}
public class InfAnonym
{
   public static void main(String x[])
   {
       A a1= ()->{
              System.out.println("I am show method");
          };

       a1.show();
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac InfAnonym.java

C:\Program Files\Java\jdk-24\bin>java InfAnonym
I am show method

C:\Program Files\Java\jdk-24\bin>
```

If we want to work with lambda expression we have some important rules

1. If lambda contain single line of logic then not need to use opening and closing brace
2. If lambda contain single line of logic not need to write return keyword
3. If anonymous inner class function use the throws clause and if we convert in lambda not need to write throws clause

```java
@FunctionalInterface
interface A
{  void show();
}
public class InfAnonym
{
   public static void main(String x[])
   {
       A a1= ()->System.out.println("I am show method");
       a1.show();
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac InfAnonym.java

C:\Program Files\Java\jdk-24\bin>java InfAnonym
I am show method

C:\Program Files\Java\jdk-24\bin>
```

Or

```java
@FunctionalInterface
interface A
{  void show();
}
public class InfAnonym
{
   public static void main(String x[])
   { ((A)()->System.out.println("I am show method")).show();
   }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac InfAnonym.java

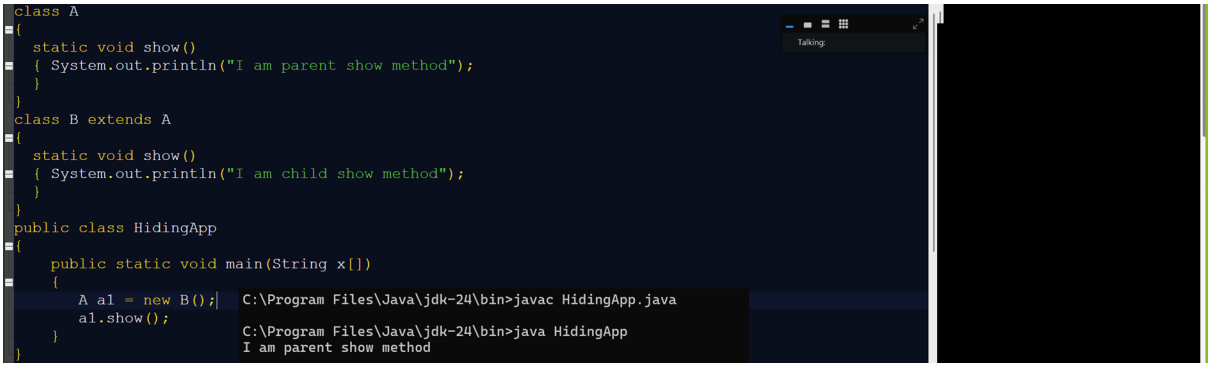
C:\Program Files\Java\jdk-24\bin>java InfAnonym
I am show method

C:\Program Files\Java\jdk-24\bin>
```

Q. What is Method hiding

Method hiding happens when we override the static methods

Method hiding indicate hide the method from overriding means in method hiding child class cannot completely modify the parent in child class



Q. What is the Covariant return type?

In Java a convenient return type means that when a subclass overrides a method from its superclass, the overriding method can return a more specific type than the return type declared in superclass method called as covariant return type.

```
abstract class Animal
{ abstract Animal reproduce();
}
class Dog extends Animal
{  Dog reproduce(){
    return new Dog();
  }
  public void feature(){
    System.out.println("Very loyal");
  }
}
class Cat extends Animal
{  Cat reproduce(){
     return new Cat();
  }
  public void feature(){
    System.out.println("Very lazy");
  }
}
public class CovarientApp
{  public static void main(String x[])
   {   Dog d1=new Dog().reproduce();
              d1.feature();
              Cat  c2 = new Cat().reproduce();
              c2.feature();
   }
}
```

Q. What is the difference between function overloading and function overriding?

Overloading	**Overriding**
Method name same with different parameter list ,number of parameter, sequence of parameter	Signature of parent and child method must be same means return type of method , name of method and type of parameters and list of parameter must be same
Can use without inheritance and with inheritance also	Must be use inheritance for overriding
Overloading support to implement compile time polymorphism	Overriding support to implement runtime polymorphism and loose coupling
Which method version should execute is dependent on number of parameter, its type and sequence in compile time	Which version should execute is dependent on child class object at run time

Q. What is the difference between compile time polymorphism and runtime polymorphism?
Q. What is the difference between abstract and final?
Q. What is the difference between static and abstract?
Q. What is the difference between static and final?
Q. What is the difference between abstract class and interface?
Q. What is the difference between abstraction and encapsulation?

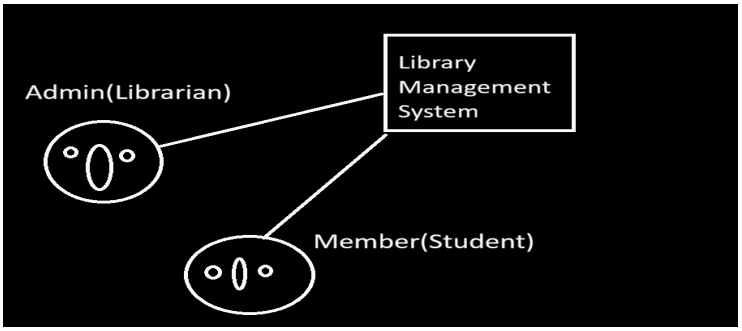
Library Management Project by using OOP?

Project Objective:
Develop console based library management system where librarians manage books and members,while members can search ,issue , return and reserve books. The Project should implement by using all OOP concept such as class, constructor,inheritance, polymorphism , encapsulation etc

1. **Login Module**

There are two types of users we have

a. Admin(Librarian)
b. Member(Student)



Admin

---

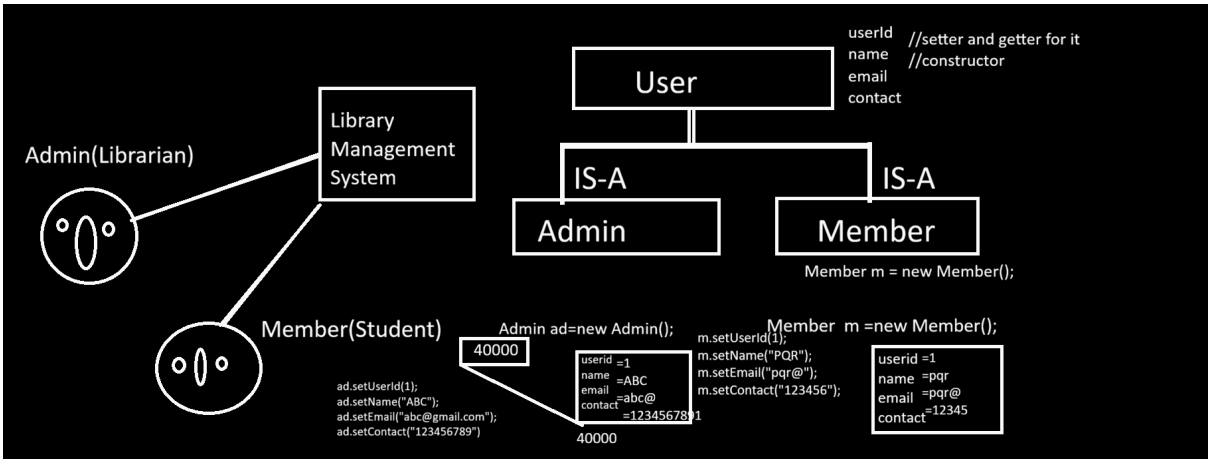
a. Login
b. Add New Book
c. Update Book
d. Delete Book
e. Search Book
f. View All Books
g. Register member
h. Allocate book to member
i. Remove member
j. View Members
k. View Issued Books
l. View Returned Books
m. Generate reports

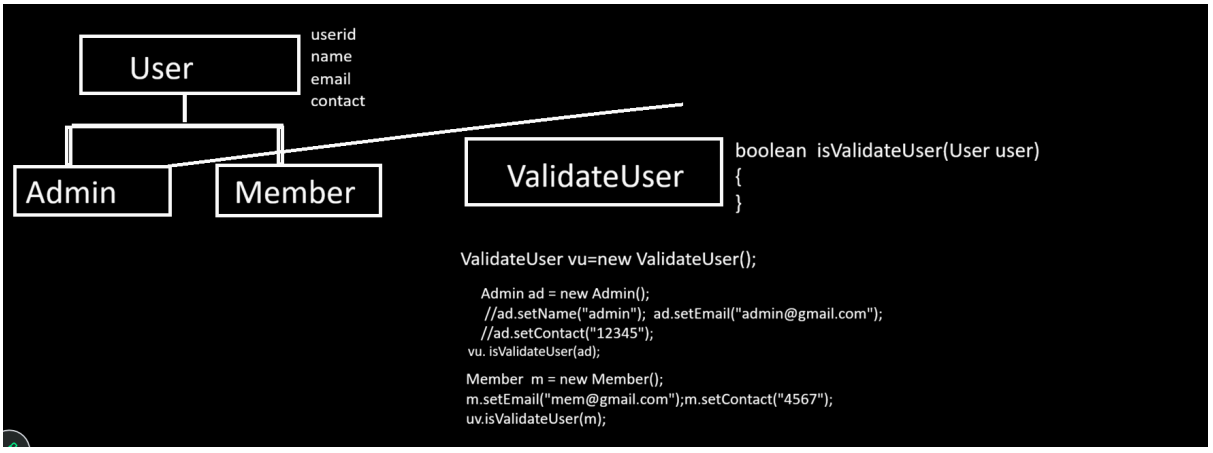
Member

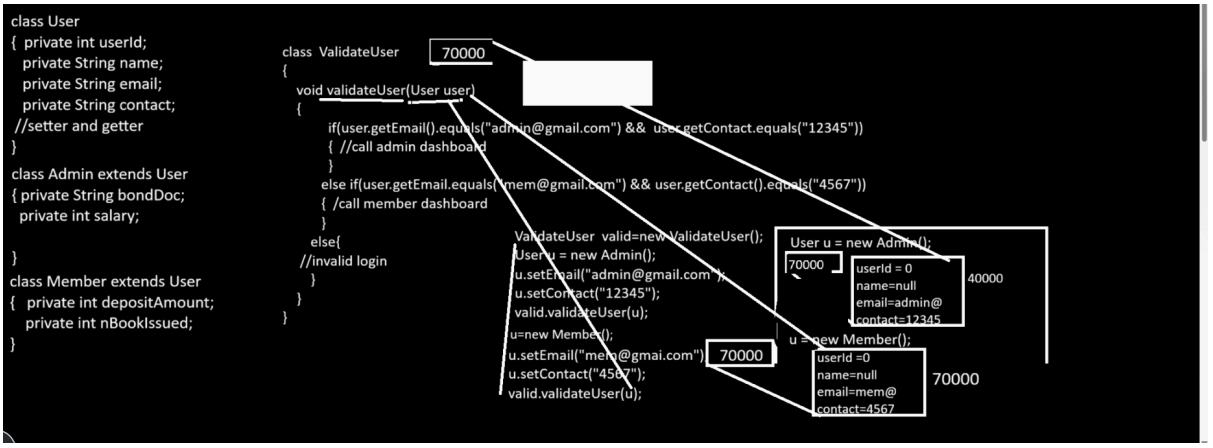
---

a. Login
b. View Available books
c. Search Book
d. Borrow Book
e. Return book
f. Reserve book
g. View Book History

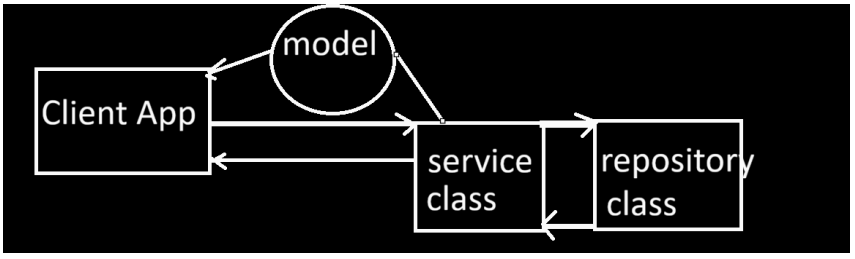
Login Module







If we want to work with any project in java we need to know the following folder structure



ClientApp/driver app : client application means class which contain main method from user provide the input and get results

Model class: model class is a class which contain setter and getter methods which is responsible for store data in object and pass in different layers

Service class: service class means class where user can write business logics

Repository class : repository class means a class where we can write database logics

Repository class object should be created in service class and service class object should be create in main method class or client application and model class object should be created in client application and pass from client application to service layer and service layer to repository class

In short we can say method of service layer should be call in client application and method of repository should be call in service layer means client application can send data to service class as input and service class send data to repository class as input and return result from repository to service and service to client

b. **Downcasting:** downcasting means when we convert a parent object into a child class object called downcasting.